

Artificial Intelligence

Complete Exam Notes

CT 710 (BEI, IV/I) · CT 653 (BCT, III/II) · CT 78506 (BEX, IV/II)
Institute of Engineering, Tribhuvan University

Built from the syllabus, seven lecturers' note sets and all **41 past papers (2069–2082 BS)** across the three course codes. Every topic carries a frequency tier counted from those papers, and every count is checked against the paper's own year tags.

Solved problems live in the companion volume, **Artificial Intelligence — Complete Solved Problems: 85 problems** over six chapters, each opening with the question exactly as its paper prints it. **Chapter 1 sets no problems** in 41 papers.

Short on time? Go straight to §8, Last-Minute Revision.

Ch	Topic	Hours	Papers	Marks	Problems
1	Introduction	7	41/41	337	—
2	Problem Solving	7	41/41	366	18
3	Search Techniques	9	41/41	392	5
4	Knowledge Representation, Inference and Reasoning	4	41/41	638	31
5	Structured Knowledge Representation	4	38/41	319	12
6	Machine Learning	6	40/41	436	10
7	Applications of AI	14	41/41	899	9
Total		51		3387	85 problems

Read the marks column, not the hours column. Chapter 4 is 4 syllabus hours and 18.8% of every mark ever set — 2.4 times its hour share — because resolution refutation alone is asked in 33 of the 41 papers. Chapter 7 is a quarter of the paper on its own. Chapter 3 is the only chapter that is under-asked for its hours.

Contents

1	Introduction	3
2	Problem Solving	17
3	Search Techniques	25
4	Knowledge Representation, Inference and Reasoning	32
5	Structured Knowledge Representation	43
6	Machine Learning	56
7	Applications of AI	70
8	Last-Minute Revision	88
	Abbreviations	94

1. Introduction

7 hrs · asked in **all 41 papers** · 6 syllabus sub-topics · 337 marks, 9.9 % of every mark ever set · no numerical component.

Read this first:

- **Every paper opens here.** Q1 is nearly always a two- or three-part Introduction question for 7 or 8 marks, and it is the cheapest 8 marks in the subject.
- Four questions carry the whole chapter:
 - **Turing test** **TOP 10/41** – what it is, when it is passed, what a machine needs to pass it.
 - **Applications of AI** **TOP 8/41** – almost always the tail of the Turing-test question, never a question on its own.
 - **Intelligent agents** **HOT 7/41** – define, then the types, with diagrams.
 - **PEAS** **HOT 7/41** – write the four rows for a named system.
- **The one idea everything hangs off is the four-approaches grid** (§1.1). Thinking Vs acting, humanly Vs rationally. Every *define AI* answer is that grid; the Turing test is the *acting humanly* box; the rational agent is the *acting rationally* box. Learn the grid and three separate questions collapse into one.
- Pairs the examiner reliably asks together: **Turing test** + **applications** (7 papers), **intelligent agent** + **PEAS** (6 papers), **knowledge** + **learning** (5 papers).
- **PEAS is worth memorising cold.** It is asked in 9 papers for a *named* system (taxi, medical diagnosis, self-driving car, robot in a manufacturing plant, medicine-delivery drone) and no lecture deck works a full example. Four rows, four columns, §1.4.
- Do not write history as prose. The examiner wants **year** → **event** lines, and one of those questions asks specifically which period was the “AI research failure” and why.

1.1 What Artificial Intelligence is

Define Artificial Intelligence. Describe AI and its subfields **1 2 4**

TOP 10/41 70 Bh, 69 Po, 75 Ba, **77 Ch**, 78 Ba, 79 Ash, 79 Bh, 81 Ba, **77 Ch**, **76 Bh** · opens the Turing-test question in almost every paper, so it is never worth more than 2 marks on its own, and it is never skippable

Definitions worth quoting

- **Elaine Rich:** the study of how to make computers do things at which, at the moment, people are better.
 - Best one to write: it explains why the field keeps moving. Once machines win at a task, it stops counting as AI.
- **Kurzweil:** the art of creating machines that perform functions requiring intelligence when performed by people.
- **Minsky:** the science of making computers do things that require intelligence when done by humans.
- **Working definition:** a branch of computer science that builds machines which solve complex problems in a human-like fashion.

Intelligence, unpacked

- Ability to **understand**, to **reason**, to **learn**, to **create**.
- No machine has all four. That gap is the whole subject.

Intelligent behaviour: three task classes

- **Everyday tasks:** recognise a friend, recognise a caller's voice, translate a language, interpret a photograph, talk, cook a meal.
 - Easiest for people, **hardest for machines**. Worth saying: this is Moravec's paradox.
- **Formal tasks:** prove a theorem, do calculus, play chess, checkers, Go.
- **Expert tasks:** engineering design, medical diagnosis, financial analysis.
 - Hardest for people, **easiest for machines**, because the knowledge is explicit and narrow.

Subfields of AI

- Search and problem solving · knowledge representation and reasoning · planning · machine learning · neural networks · expert systems · natural language processing · machine vision and image processing · robotics · fuzzy logic and evolutionary computing.

The four approaches to AI (the grid everything sits on) 4 7 8

74 Bh, 76 Ba, 82 Bh · asked as “justify that systems that think/act rationally are part of AI” and as “define AI from the dimension of behavioural process and thought process”

	Humanly (measured against people)	Rationally (measured against an ideal)
Thinking	Thinking humanly – the cognitive modelling approach. Build a precise theory of how the human mind works, then write it as a program. Needs psychology and introspection. Eg General Problem Solver, Newell and Simon, 1961. Criticism: we have no scientific theory of the brain's internal activity, so the model cannot be verified.	Thinking rationally – the laws of thought approach. Encode correct reasoning as formal logic, then let an inference engine derive conclusions. Eg “Socrates is a man; all men are mortal; therefore Socrates is mortal.” Criticism: informal, uncertain knowledge is very hard to state in logical notation, and solving a problem in principle is not solving it in practice.
Acting	Acting humanly – the Turing test approach. Model what people do, not how they think. Judged purely on external behaviour. Eg ELIZA. This is the box §1.2 lives in.	Acting rationally – the rational agent approach. Do whatever is expected to achieve the best outcome. Covers reflex action, which needs no reasoning at all. The approach modern AI actually uses, because it is general and it can be measured. This is the box §1.3 lives in.

How to answer “justify that X is part of AI”

- Name the box, give its yardstick, give an example system, give its criticism.
- **Thinking rationally** is part of AI because logic-based inference (FOPL, resolution, Ch 4) derives new true facts from known ones without any human in the loop.
- **Acting rationally** is part of AI because an agent that maximises its performance measure in an uncertain environment is doing what intelligence is for, whether or not it reasons the way people do.
- **Say which is preferred and why:** acting rationally, because rationality is mathematically well defined and generalises, while “humanly” needs a theory of mind we do not have.

Types of AI by capability

- **Artificial Narrow Intelligence (ANI, weak AI):** one task or one narrow domain. Everything that exists today. Eg spam filter, chess engine, a speech recogniser.
- **Artificial General Intelligence (AGI, strong AI):** any intellectual task a human can do, transferring skill between domains. Does not exist.
- **Artificial Super Intelligence (ASI):** exceeds human ability in every field including creativity and social skill. Hypothetical.

Weak Vs strong AI, the philosophical sense

- **Weak:** the machine *simulates* thinking.
- **Strong:** the machine genuinely *has* a mind.
- Searle's Chinese Room argues strong AI is impossible: symbol manipulation is not understanding. verified/added

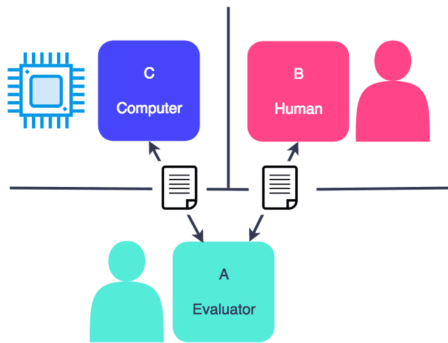
1.2 The Turing test

What is the Turing test? When is a machine said to have passed it? 2 4 5 7

TOP 10/41 70 Bh 5, 69 Po 7, 75 Ba 2, 77 Ch 3, 78 Ba 2, 79 Ash 4, 79 Bh 5, 81 Ba 6, 77 Ch 4, 76 Bh 3 · the single most-asked question in the subject's first chapter

The setup

- Proposed by **Alan Turing**, 1950, in *Computing Machinery and Intelligence*, to replace the unanswerable “can machines think?” with a testable question, “can machines behave intelligently?”
- Three rooms, no sight or sound between them:
 - a human **interrogator**,
 - a human **respondent**,
 - the **machine**.
- The interrogator questions both over a text-only channel (a teletype, so voice and handwriting give nothing away) and must say which is which.



Draw this: the arrangement is the definition. A is the interrogator, B the human respondent, C the machine. The wall and the text-only channel are the two things that must appear – they are what stops the test measuring anything except behaviour. A knows one of B, C is a machine and must say which.

When it is passed

- The machine passes when the interrogator **cannot do better than chance**, i.e. is wrong about half the time.
- Turing’s own operational figure: an average interrogator has **under a 70 % chance** of identifying correctly after **5 minutes** of questioning, so the machine fools them **30 %** of the time.
- **Write both:** the half-the-time principle, then the 5-minute / 30 % figure as the concrete criterion.

The four capabilities a machine needs to pass

- **Natural Language Processing** → to communicate in English at all.
- **Knowledge Representation** → to store what it knows and what it is told during the interview.
- **Automated Reasoning** → to answer questions and draw new conclusions from what is stored.
- **Machine Learning** → to adapt to new circumstances and detect patterns it was not given.

Total Turing test adds two more

- The plain test deliberately avoids physical contact, so it never tests perception or manipulation. The **total Turing test** puts a video link and a hatch for passing objects into the setup, adding:
 - **Computer vision** → to perceive objects,
 - **Robotics** → to manipulate them and move about.
- **6 capabilities in total.** (77 Ch and 76 Bh ask for the total test by name)

ELIZA, the standard example

- Written at MIT, **1964–66**, by **Joseph Weizenbaum**. First script **DOCTOR**, imitating a Rogerian psychotherapist.
- Mechanism: a table of **syntactic patterns**, each with a canned reply that echoes part of the input back after simple substitutions (*my* → *your*). **No understanding, no knowledge base, no memory.**
- Weizenbaum was disturbed by the reaction: psychiatrists thought it clinically useful and users confided in it.
- **The exam point:** ELIZA shows a machine can be *taken* for a human without any intelligence behind it, which is the standing objection to the test itself.

Importance of the Turing test

- Turns a philosophical question into an **operational, repeatable** one.
- Sets a **behavioural** yardstick: judge output, not internal mechanism, so it does not need a theory of mind.
- Defines the **research agenda**: its four required capabilities are four whole subfields of AI, and this syllabus is mostly those four.
- Still the benchmark by which conversational systems are reported.

Limitations

- Rewards **deception**, not competence. Passing needs human-like error and delay, so a machine that answers arithmetic instantly and correctly *fails*.
- **Not reproducible** and not mathematically definable; the result depends on the interrogator.
- Tests only what people happen to do, so it caps machine intelligence at human behaviour rather than measuring intelligence itself.

If the Turing test is passed, does that show computers exhibit intelligence? 2 3 4 7

HOT 4/41 72 Ma 7, 73 Bh 7, 81 Bh 4+3, 80 Ch 2+4 · **an argue-both-sides question**: take a side, but the marks are in giving both cases first. 81 Bh and 80 Ch tack the *reverse* Turing test on the end

Case for “yes”

- Intelligence has **no agreed internal definition**, so behaviour is the only thing that can be measured at all.
- We attribute intelligence to *other people* on exactly this evidence, and nothing else. Refusing the machine the same inference is inconsistent.
- Passing is not a trick: it requires NLP, knowledge, reasoning *and* learning working together, which is a real and general capability.

Case for “no” (the stronger answer)

- **Imitation is not cognition**. ELIZA fooled people with pattern substitution and knew nothing.
- **Searle’s Chinese Room**: a person following symbol-manipulation rules can answer Chinese questions perfectly while understanding no Chinese. Syntax does not give semantics. **verified/added**
- Tests only **one narrow channel**, text conversation. Says nothing about perception, physical skill, creativity or goals.
- **Recommended stance**: passing shows *intelligent behaviour*, not *intelligence*. It is a sufficient test of appearing intelligent and a necessary-but-not-sufficient test of being so.

Reverse Turing test 3 4

verified/added 81 Bh 3, 80 Ch 4 · 2 papers, both under 6 marks, so no tier chip · **not in any lecture deck in this folder**, researched and added, and it has been asked twice in the last three years

What it is

- The roles are swapped: **a machine interrogates, and a human must prove it is human**.
- The test succeeds when the machine can tell human from machine, which is the opposite goal to Turing’s.
- **CAPTCHA** is the everyday instance: *Completely Automated Public Turing test to tell Computers and Humans Apart*. A distorted-text or image-grid challenge that people pass easily and programs do not.

Turing Vs reverse Turing, side by side

- **Who judges**: human, Vs machine.
- **Who is on trial**: machine, Vs human.
- **Passed when**: the judge cannot tell them apart, Vs the judge *can*.
- **Purpose**: measure machine intelligence, Vs block bots from a web service.
- **Note the irony worth a mark**: every CAPTCHA solved by a program is a small Turing test passed, which is why CAPTCHAs keep getting harder.

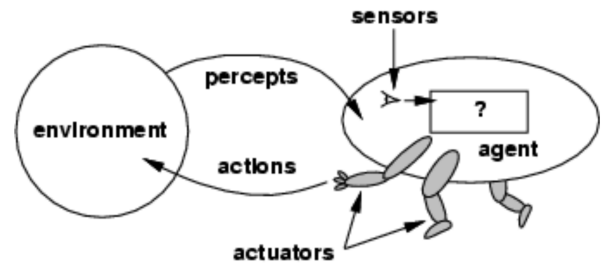
1.3 Intelligent agents

What is an intelligent agent? Explain the different types with examples 1 2 6 8

HOT 7/41 74 Bh 1+6, 80 Bh 2+6, 80 Ash 2+6, 79 Jth 1+6, 75 Bh 8, 76 Ba 8, 71 Bh 4+4 · 75 Bh asks specifically **how a learning agent works**, so do not stop at the four standard types

Definitions to write first

- **Agent:** anything that perceives its environment through **sensors** and acts upon that environment through **actuators**.
 - **Human agent:** eyes and ears as sensors; hands, legs and mouth as actuators.
 - **Robotic agent:** cameras and infrared range finders as sensors; motors as actuators.
 - **Software agent:** keystrokes and file contents as sensors; screen writes and network packets as actuators.
- **Percept:** what the agent perceives at one instant. **Percept sequence:** everything it has ever perceived.
- **Agent function** maps percept sequence to action, $f : P^* \rightarrow A$.
- **Agent program** is the concrete implementation that runs on the hardware.
- **Agent = architecture + program.** One line, and it is often a mark on its own.



The agent, its sensors and actuators, and the environment loop.

Draw this first in any agent answer; every later diagram is this one with a box added inside the agent.

Rational agent

- For each possible percept sequence, a rational agent selects the action expected to **maximise its performance measure**, given the evidence in the percept sequence and whatever built-in knowledge it has.
- **Performance measure:** the objective criterion for success, fixed by the designer, not the agent. Eg for a vacuum cleaner: dirt collected, time taken, electricity used, noise made.
- **Rationality is not omniscience.** An omniscient agent knows the actual outcome of its actions; that is impossible. Rationality maximises *expected* performance given what is knowable.
- **Autonomy:** an agent is autonomous when its behaviour is determined by its own percepts and experience rather than only by built-in knowledge. Autonomy requires learning.
- **Information gathering** and **exploration** are rational acts: it can be rational to act now purely to get a better percept later.

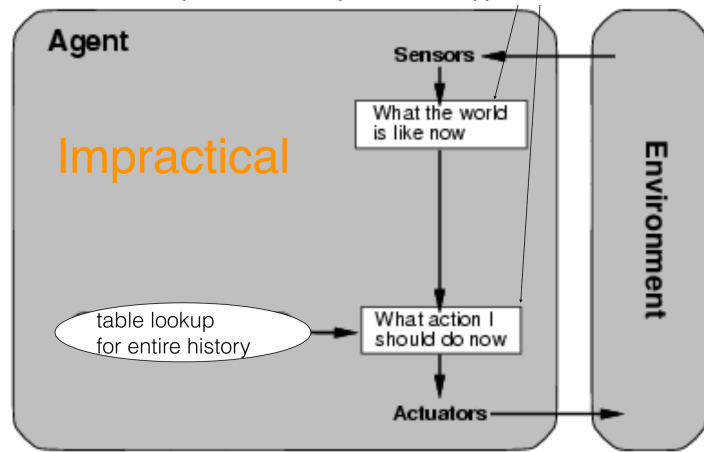
Environment properties (worth 2 marks when asked)

- **Fully Vs partially observable:** can the sensors see the whole relevant state?
- **Deterministic Vs stochastic:** does the next state follow from the current state and action alone?
- **Episodic Vs sequential:** does the current action affect later ones?
- **Static Vs dynamic:** does the world change while the agent deliberates?
- **Discrete Vs continuous:** countable states and actions, or not?
- **Single agent Vs multi-agent:** is anything else optimising against it?
- Eg a taxi driver's world is partially observable, stochastic, sequential, dynamic, continuous and multi-agent. **The hardest kind on every axis.**

The five agent types, in increasing generality

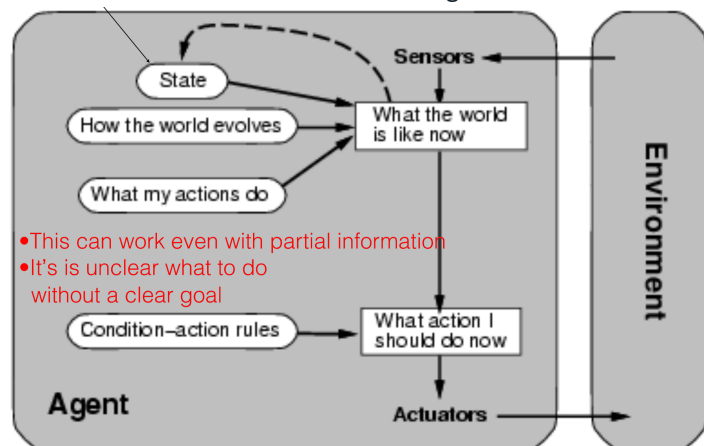
1. Table-driven agent

- Holds a lookup table indexed by the *entire percept history*, and reads the next action out of it.
- **Impractical**: the table grows exponentially with the length of the history. A taxi with a camera would need more entries than there are atoms in the universe.
- Included only to show why the later types exist.



3. Model-based reflex agent

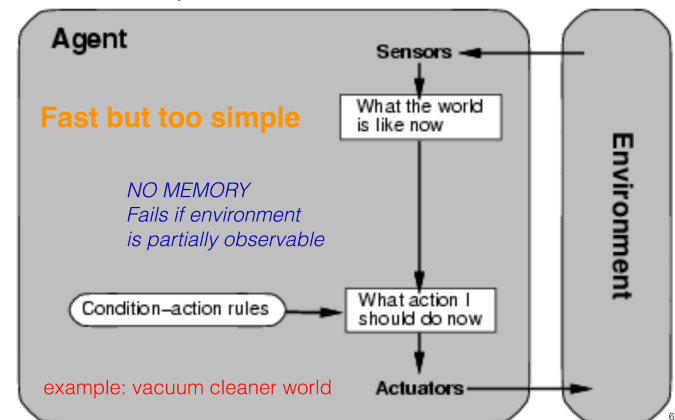
- Keeps **internal state** = the part of the world it cannot currently see.
- Needs two pieces of knowledge, together called the **model of the world**:
 - how the world evolves on its own,
 - what my actions do to it.
- **Fixes partial observability**. Eg a taxi remembering which cars are in the lanes it cannot see right now.



- This can work even with partial information
- It's unclear what to do without a clear goal

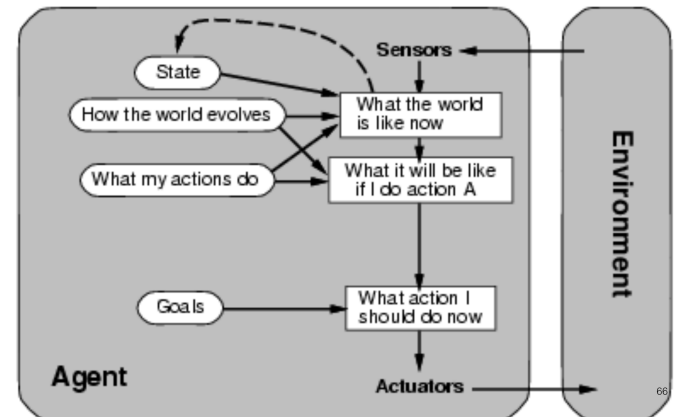
2. Simple reflex agent

- Acts on the **current percept only**, through **condition-action (if-then) rules**. Implemented as a production system (Ch 2).
- **Stateless**: no memory of past world states.
- Eg
 - if car-in-front-is-braking then initiate-braking;
 - the vacuum-cleaner agent that sucks whenever the current square is dirty.
- **Fails when the environment is partially observable**, and can loop forever in that case.



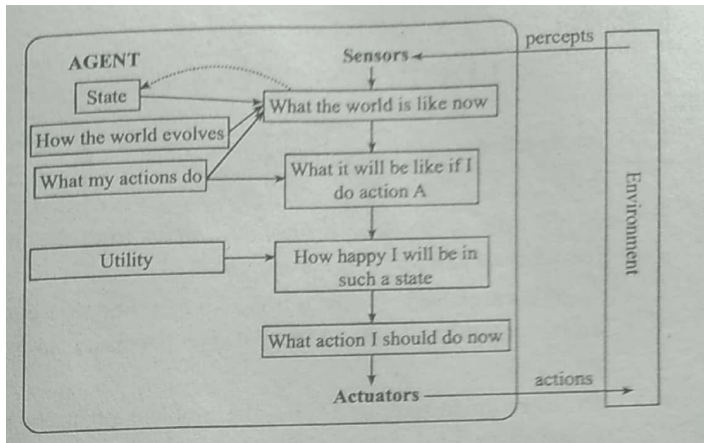
4. Goal-based agent

- Adds **goal** information describing desirable situations, and asks "what will it be like if I do action A?"
- Considers the **future**, so it can plan and search (Ch 3), not just react.
- **Far more flexible**: change the goal and the behaviour changes, with no rules rewritten. A reflex agent needs every rule rewritten.
- Eg a taxi told to go to a different destination.



5. Utility-based agent

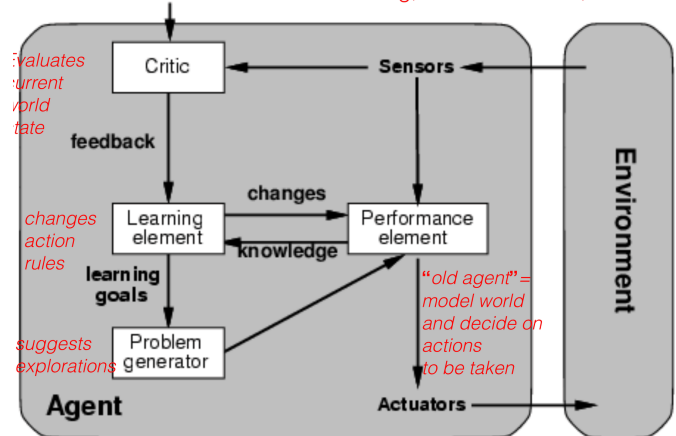
- A goal is binary: reached or not. A **utility function** maps a state to a number saying **how happy** the agent is there.
- Needed for two cases a goal cannot settle:
 - **conflicting goals** – fast Vs safe Vs cheap: utility trades them off;
 - **uncertain goals** – pick the action with the highest *expected* utility.
- Eg a taxi choosing between a quick risky route and a slow safe one.



Learning agent (asked by name, 75 Bh 8)

- Any of the four above, wrapped in machinery that **improves it with experience**. Four parts:
 - **Performance element** – the old agent: takes percepts, chooses actions.
 - **Critic** – judges the outcome against a fixed **performance standard** and reports feedback. The standard must be external, or the agent could just lower its own bar.
 - **Learning element** – uses that feedback to change the performance element.
 - **Problem generator** – deliberately suggests **exploratory** actions that look suboptimal now, so the agent discovers better behaviour later.
- **The exam point:** learning is what makes an agent truly autonomous, which is the link straight to §1.9 and to Ch 6.

Performance standard *better modeling, new action rules, etc.*



1.4 PEAS: specifying the task environment

Define PEAS. Give the PEAS information for a named system 2 3 5 6

HOT 7/41 80 Ba (medical diagnosis) 3, 78 Ch 4, 74 Bh 3, 81 Ash (taxi + robot) 5, 82 Ka (robot) 3, 82 Ba (self-driving car) 6, 82 Bh (medicine-delivery drone) 5 · **asked with a different system every single time**, so learn the four rows, not four memorised answers

What and why

- Before designing an agent, its **task environment** must be specified. PEAS is that specification:
 - P – Performance measure:** what counts as success. Must be about the *environment*, not the agent's internal state.
 - E – Environment:** what the agent operates in and on.
 - A – Actuators:** how it acts.
 - S – Sensors:** how it perceives.
- Why it must come first:** the performance measure decides what “rational” means, and the sensors and actuators decide what the agent can even represent. Design before PEAS is design without a specification.

Three mistakes that cost marks

- Writing the performance measure as “*work well*”. Give measurable terms: safe, fast, legal, comfortable, maximise profit.
- Putting an actuator in the sensor row. A horn is an actuator; a microphone is a sensor.
- Forgetting that a **display** is an actuator when the agent uses it to tell a person something.

Self-driving car (82 Ba 6)

- P:** reach the destination safely, obey traffic law, minimise journey time and fuel, ride comfort, zero collisions.
- E:** roads and lane markings, other vehicles, pedestrians and cyclists, traffic signals, road signs, weather, the passenger.
- A:** steering, throttle, brakes, gear selection, indicators, horn, headlights, passenger display and speaker.
- S:** cameras, LIDAR, radar, ultrasonic parking sensors, GPS, IMU and accelerometer, wheel-speed sensors, microphone.

Ready-made answers for the systems actually asked

- Taxi driver** (81 Ash)
 - P:** safe, fast, legal, comfortable trip, maximise profits.
 - E:** roads, other traffic, pedestrians, customers, weather.
 - A:** steering, accelerator, brake, indicator, horn, display.
 - S:** cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard.
- Medical diagnosis system** (80 Ba)
 - P:** healthy patient, minimise costs and lawsuits.
 - E:** patient, hospital, staff.
 - A:** screen display of questions, tests, diagnoses, treatments, referrals.
 - S:** keyboard entry of symptoms, findings and the patient's answers.
- Part-picking robot in a plant** (81 Ash, 82 Ka)
 - P:** percentage of parts placed in the correct bin.
 - E:** conveyor belt with parts, the bins.
 - A:** jointed arm, gripper.
 - S:** camera, joint angle sensors.

Medicine-delivery drone (82 Bh 5) verified/added

- No source note covers this one;** state your assumptions, as the paper explicitly allows, then give four clean rows.
- P:** medicine delivered intact, to the right address, within the time window, maximum payload per charge, no collision, no airspace violation.
- E:** air corridor, buildings, terrain and hills, trees and power lines, weather and wind, other drones and aircraft, the recipient, the dispatch centre.
- A:** rotors and motors, payload release hatch, landing gear, status lights, radio downlink to the dispatch centre.
- S:** GPS, camera, LIDAR or ultrasonic altimeter, IMU and gyroscope, barometer, wind and temperature sensors, battery gauge, radio uplink.

1.5 Importance and limitations of AI; AI Vs human intelligence

Differentiate natural and artificial intelligence. Can AI replace humans? 3 4 5 7

PIN 3/41 79 Ch 1+3+4, 78 Po 3+5, 73 Bh 7 · growth areas asked at 71 Bh 4, disadvantages at 78 Ba 2

	Human intelligence	Artificial intelligence
Origin	Biological, inherited and grown	Engineered, programmed and trained
Learns from	A few examples, plus intuition and context	Very large data sets, no context of its own
Speed	Slow, tires, needs rest	Constant, no fatigue, runs continuously
Accuracy	Errs when tired or emotional	Exact and repeatable within its domain
Breadth	General: transfers skill to a new task	Narrow: fails outside the trained task
Creativity	Original ideas, imagination	Recombines what it was trained on
Emotion	Empathy, judgement, ethics	None; only what is coded as a measure
Adapting	Handles the wholly unfamiliar	Needs retraining or reprogramming
Cost	Salary, training time	High to build, near-zero to copy

Importance and advantages of AI

- **Automates repetitive work** → people move to judgement work.
- **Better decisions:** finds patterns in data volumes no person can read.
- **Available always:** no fatigue, no shift limits, consistent output.
- **Works where people cannot:** deep sea, space, reactor cores, disaster zones, mine clearance.
- **Fewer human errors** in exact, repetitive tasks.
- **Reaches scarce expertise:** an expert system puts specialist knowledge in a village clinic.
- **Speeds research:** drug discovery, protein folding, materials.

Growth areas (71 Bh)

- Machine learning and deep learning · NLP and speech · computer vision · robotics and autonomous vehicles · expert and decision-support systems · recommender systems · medical imaging and diagnosis.

Can AI replace humans? Answer: in tasks, yes; in roles, no

- **Where it replaces:** repetitive, rule-bound, data-heavy work. Data entry, first-line support, defect inspection, routine diagnosis, translation drafts.
- **Where it cannot, and why:**
 - **No common sense** outside the training distribution.
 - **No moral or legal responsibility;** someone must be accountable.
 - **No genuine creativity or empathy,** both needed in teaching, care, negotiation, design.
 - **Depends on data** that people must collect, label and correct.
- **Conclusion to write:** AI is a tool that shifts human work up the skill ladder, not a replacement for the worker. The realistic risk is *displacement of specific tasks*, not obsolescence of people.

Disadvantages and limitations (78 Ba 2)

- **Cost:** high to build, and needs specialists to maintain.
- **No common sense:** fails absurdly on inputs slightly outside its training.
- **No creativity:** recombines, does not originate.
- **Job displacement** in routine occupations.
- **Opaque:** a neural network cannot explain its decision, which is unacceptable in medicine and law.
- **Inherits bias** from its training data, then applies it at scale.
- **No ethics or accountability,** so misuse is easy: surveillance, autonomous weapons, deepfakes.
- **Data-hungry and energy-hungry.**

Two named ethical positions worth a line

- **Weizenbaum (1976):** a real AI would be autonomous and would not share our motives or ethics, so the research is unethical.
- **Asimov's three laws of robotics:** do not harm a human or by inaction allow harm; obey humans; protect itself. Precedence runs lower number first.

1.6 AI and related fields

Describe AI and its subfields; which fields contributed to AI? 4

76 Bh 4 · asked once, but it is a two-minute answer and it recurs as a sub-part of "define AI"

What each field gave AI

- **Philosophy:** logic, the idea of the mind as a physical system, foundations of learning, language and rationality.
- **Mathematics:** formal representation and proof, algorithms, computation, decidability and intractability, probability, statistics, linear algebra.
- **Psychology:** perception, motor control, adaptation, and the experimental methods behind cognitive modelling.
- **Economics:** formal theory of rational decision, utility, game theory.
 - This is where the *utility* in a utility-based agent comes from.
- **Linguistics:** grammar, syntax and knowledge representation. NLP sits directly on it.
- **Neuroscience:** the physical substrate of mental activity. Neural networks are a deliberate, loose imitation of it.
- **Control theory:** stability, feedback, optimal agent design. The agent-environment loop is a control loop.
- **Computer engineering:** the machines fast enough to run any of it.

Fields AI overlaps with, and how it differs

- **Machine learning** is a subfield of AI, not a synonym: AI also covers search, logic and planning, which do not learn at all.
- **Data mining** (Ch 6 and the Data Mining elective) shares algorithms but aims at *discovering patterns in stored data*; AI aims at *acting well*.
- **Robotics** supplies the body; AI supplies the decision making.
- **Cognitive science** is the joint enterprise of AI and psychology: AI wants machines that work, cognitive science wants a theory of the mind.
- **Operations research** shares optimisation and search but assumes a fully specified model.

Programming languages of AI

- **LISP:** McCarthy, late 1950s. Functional, symbolic, interactive.
- **PROLOG:** France 1970, developed at Edinburgh 1975–79. Logic programming: a program is facts + rules, and running it is inference. **The syllabus lab is in one of these two.**

1.7 Brief history of AI

Discuss the brief history of AI with chronological development 2 5 6 7

PIN 3/41 70 Ma 2+6, 71 Bh 2+5, 72 Ma 7 · 72 Ma asks specifically **which period was the “AI research failure” period and why**, so learn the two AI winters, not just the timeline

The founding era, 1943–1979

Year	Event
1943	McCulloch and Pitts : Boolean circuit model of the neuron. The first neural network.
1950	Turing , <i>Computing Machinery and Intelligence</i> . The Turing test.
1956	Dartmouth meeting : the term “Artificial Intelligence” is adopted. The birth of the field.
1950s	Samuel’s checkers program, which learned and beat its author; Newell and Simon’s Logic Theorist; Gelernter’s Geometry Theorem Prover (1959).
1957	Rosenblatt’s Perceptron.
1958	McCarthy defines LISP and describes the Advice Taker.
1963	McCarthy starts the Stanford AI lab. Slagle’s SAINT solves first-year calculus.
1965	Robinson’s resolution : a complete theorem-proving algorithm for first-order logic. Ch4 is built on it.
1966	ELIZA , Weizenbaum.
1969–79	Knowledge-based and expert systems : DENDRAL , MYCIN , SHRDLU . Minsky’s frames (1975).

The “AI research failure” period

- Two collapses in funding and confidence, both called **AI winters**. The question almost always means the first.
- **First AI winter, 1966–73**, the “dose of reality”. In 1957 Simon predicted a computer chess champion and a major theorem proved within **10 years**; both took roughly **40**. Three reasons it failed:
 - **No subject knowledge**. Early programs succeeded by *syntactic manipulation* alone. Machine translation is the standard example: word-by-word substitution with no meaning behind it.
 - **Intractability**. Programs solved toy problems by trying combinations of steps. Scaling the problem exploded the search, and the 1965 discovery of NP-completeness explained why faster hardware could not fix it.
 - **Structural limits**. Minsky and Papert’s *Perceptrons* (1969) proved a single-layer perceptron cannot represent XOR, and neural network research nearly stopped for fifteen years. **This is the same XOR result you prove in Ch 7.**

Industry, science and the modern era, 1980–

Year	Event
1980	AI becomes an industry . XCON saves DEC an estimated \$40 million a year.
1986	Neural networks return , on back-propagation.
1987	AI becomes a science : results measured against benchmarks, and statistics and probability adopted.
1995	The intelligent agent becomes the unifying idea of the field.
1997	Deep Blue beats Kasparov at chess.
2011	IBM Watson wins <i>Jeopardy!</i>
2016	AlphaGo beats Lee Sedol at Go.
2020–	Large language models, GPT-3 onward.

- **Write it as a table, not as prose.** Year, then event, then one clause on why it mattered. Dates with no significance score badly.

The second winter, and what ended both

- **Second AI winter, late 1980s to early 1990s**: the expert-system market collapsed. The systems were **brittle** outside their narrow domain, **could not learn**, and cost more to maintain than they saved.
- **What ended each**:
 - the first ended with **knowledge-based systems** – stop searching blindly and put the domain knowledge in;
 - the second ended with **statistical machine learning**, then cheap parallel hardware and large data sets.
- **The pattern behind both**, and the sentence that earns the last mark: each winter followed a period of **over-promising on a technique that did not scale**, and each ended when the field adopted a *different* technique rather than more of the same.

1.8 Applications of AI

Discuss fields of daily life where AI has been applied 2 4 6 7

TOP 8/41 69 Bh 7, 72 Ash 7, 73 Ma 6, 81 Ch 2+4, 75 Ba, 77 Ch, 78 Ba, 78 Po · usually the tail of a Turing-test or define-AI question, which is why it is worth 4 marks and two minutes. Most papers ask for **any two**, explained

How to answer for full marks

- The question says *explain two fields*, not *list ten*. A list scores badly.
- For each field give: **the task**, **the AI technique behind it**, and **a named example**. Three sentences each is the whole answer.
- 81 Ch asks for applications **as a problem-solving tool**, so frame each one as “the problem, and how AI solves it”.

Two safe choices, worked

- **Healthcare**. Problem: expert diagnosis is scarce and slow. Technique: neural networks over medical images (Ch 7), plus rule-based expert systems for diagnosis. Examples: tumour detection on CT and MRI, IBM Watson for Oncology, diabetic-retinopathy screening from retinal photographs.
- **Transport**. Problem: driving needs continuous perception and decision under uncertainty. Technique: computer vision and sensor fusion feeding a utility-based agent, plus A* and its variants for route search (Ch 3). Examples: self-driving cars, Google Maps ETA prediction, Pathao and inDrive dispatch and pricing.

The wider list, grouped

- **Everyday software**: search-engine ranking and autocomplete, social-media feeds, Netflix and YouTube recommendation, spam filters, photo tagging.
- **Language**: machine translation, speech recognition, chatbots and virtual assistants, Nepali NLP tools (Ch 7).
- **Finance**: fraud and money-laundering detection, credit scoring, algorithmic trading.
- **Industry and agriculture**: robotic assembly and part picking, predictive maintenance, visual quality inspection, crop-disease detection from leaf images, yield prediction.
- **Games**: Deep Blue, AlphaGo, and the bots in Dota 2 and PUBG.
- **Science and space**: astronomical object classification, protein folding, space-mission monitoring, drug discovery.
- **Public services**: weather and flood forecasting, traffic signal control, disaster response, biometric identification.

Nepal-specific examples, if the question invites them

- Pathao and inDrive dispatch · eSewa and Khalti fraud detection · Nepali speech and OCR · landslide and flood early warning · crop-disease apps.

1.9 Knowledge and learning

What is knowledge and learning? Why are they important? 2 4 5 8

HOT 5/41 73 Bh 4+4 , 74 Ma 2+2+2+4 , 72 Ash 2+2+2+4 , 74 Bh 4+4 , 79 Jth 4+4 · also inside the Turing question at 79 Ash 2+2 and 71 Ma 2
 · **the quotable line the examiner wants back:** "Learning is an essential characteristic for intelligent agents"

	Data	Information	Knowledge
What	Raw facts, unorganised	Data processed into a meaningful, usable form	Information organised and understood, ready to act on
Form	Numbers, symbols, a database	Tables, reports, summaries	Rules, procedures, models
Answers	nothing on its own	who, what, where, when	how and why
Example	$V = 12, I = 6, R = 2$	$V = I \times R$ holds for these readings	Use $V = IR$ to design a circuit

- **Intelligence** sits one step above knowledge: the ability to use knowledge to solve new problems and adapt to new situations.
- **Write the chain:** data → information → knowledge → intelligence. Each step adds structure and context.

Knowledge, and why AI needs it

- **Knowledge:** the condition of knowing something with familiarity gained through experience or association; what lets us explore and decide.
- Stored by people in **natural language**, by machines as **symbols, strings and structures**.
- Forms it takes: simple facts, relations between concepts, **associations, inheritance hierarchies**, procedures.
- **Importance:** enough knowledge is what produces intelligence. A search algorithm with no domain knowledge is blind (Ch 3); the same algorithm with a heuristic is informed and vastly faster.
Knowledge is what turns search into intelligence.
- **The three designer problems** (they run through Ch 4 and Ch 5):
 - **Knowledge acquisition** – getting it out of the domain and its experts.
 - **Knowledge representation** – encoding it: logic, semantic nets, frames, rules.
 - **Knowledge manipulation** – inferring new conclusions from it.

Learning

- **Herbert Simon (1983):** "Learning is the phenomenon of knowledge acquisition in the absence of explicit programming."
- Fuller form: learning denotes **changes in the system that are adaptive**, in the sense that they let it do the same task, or a task drawn from the same population, more efficiently and more effectively next time.
- **Three factors in any learning system:**
 - **Changes** – what in the system is altered, and how it is represented.
 - **Generalisation** – performance must improve on *similar* tasks, not only the one seen. Without this it is memorisation.
 - **Improvement** – the change must not degrade performance; the system must detect and prevent that.
- **Knowledge Vs learning in one line:** knowledge is the *content* an agent holds; learning is the *process* that changes that content.

"Learning is an essential characteristic for intelligent agents"

- **Agree, and justify in four steps:**
 - **The designer cannot foresee every situation.** A fixed rule table is only as good as its author's imagination, and the table-driven agent shows how quickly enumeration becomes impossible.
 - **Environments change.** Roads are rebuilt, spam changes its wording, prices move. A non-learning agent's performance decays with no fault of its own.
 - **Autonomy requires it.** An agent whose behaviour comes only from built-in knowledge is running the designer's intelligence, not its own. Learning is exactly what makes it autonomous.
 - **The Turing test demands it.** Machine learning is one of the four capabilities needed to pass, so an agent that cannot learn cannot be judged intelligent on the field's own benchmark.
- **Example to close with:** a spam filter shipped with fixed rules is beaten within weeks; one that learns from each user's marking of messages keeps working. Same architecture, and learning is the only difference.

Belief, hypothesis and knowledge 4

71 Bh 4 · asked once, as the tail of the growth-areas question. A three-row table is the whole answer

Term	Means	Confidence
Belief	Any statement an agent holds to be true. It need not be verified and may turn out false.	Subjective, unverified.
Hypothesis	A belief <i>put forward for testing</i> , backed by some evidence and held provisionally until confirmed or refuted.	Justified belief, not yet confirmed.
Knowledge	A belief that is <i>true</i> and <i>justified</i> . Verified, and safe to reason from.	Confirmed.

- **The relation, in one line:** knowledge $\subset$ hypothesis $\subset$ belief. Every hypothesis is a belief; only a confirmed, true one becomes knowledge.
- **Why AI cares:** an agent must act on beliefs, because it rarely has knowledge. That is precisely why Ch 4 needs probability and belief networks rather than plain logic.

2. Problem Solving

7 hrs · asked in **all 41 papers** · 4 syllabus sub-topics · 366 marks, 10.8 % of every mark ever set · **heavy solved-problem chapter**: see Ch2 – Problem Solving – Solved Problems.

Read this first:

- **One question dominates this chapter and the whole subject: solve a crypt-arithmetic puzzle by constraint satisfaction** **TOP 31/41**. **31 of the 41 papers** set it, for 6 to 8 marks, and it is almost always preceded by a 1 or 2 mark “*what is a CSP*” **TOP 12/41**. Together that is **8 to 9 marks in nearly every paper**, from one technique.
 - Thirteen different puzzles have been set. They are all worked in the companion file. **BASE + BALL = GAMES** (8 papers) and **SEND + MORE = MONEY** (6) lead.
 - **Seven of the thirteen puzzles have more than one valid answer**; **ONE + ONE + TWO = FOUR** has **107**. So the marks are in **the column-by-column constraint reasoning**, not in the digits. Write the variables, the domains, the constraints and the carries, then reason. An answer with no working scores badly even when the arithmetic is right.
 - The rest of the chapter is four small, reliable topics: **well-defined problem** **HOT 7/41**, **state space and the steps of problem solving** **HOT 5/41**, **production system** **HOT 5/41**, and the **water-jug** formulation **HOT 4/41**.
 - Pairs the examiner reliably asks together: **well-defined problem** + **a water-jug or crypt-arithmetic instance**, and “**what is a CSP**” + **the puzzle**. Almost nothing here is ever asked alone.
 - **Game playing is set up here and answered in Ch 3**. This chapter owes you only the *definition* of a game as a search problem; minimax and alpha-beta belong to §3.4.
-

2.1 Problem solving and the state space

Discuss the steps of problem solving. What is state space, successor function, goal test, path cost? 2

3

HOT 5/41 82 Bh 3, 82 Ba 2, 82 Ka 2, 81 Ch 3, 76 Bh 3 · a 2 or 3 mark opener, never more. Four steps, five terms, done

The four steps of problem solving

- **1. Goal formulation** – decide which world states count as success. Without it the agent has no way to prefer one action over another.
- **2. Problem formulation** – decide what states and actions to consider: the initial state, the successor function, the goal test and the path cost.
- **3. Searching** – find a sequence of actions leading from the initial state to a goal state, and choose the best such sequence. This is Ch 3.
- **4. Execution** – carry out the returned solution.
- **Order matters and is often the mark:** goal before formulation, and the whole search happens *before* any action is executed.

State space representation

- A **state space** is a set of **nodes**, one per state of the problem, joined by **arcs** representing the legal moves from one state to another, with a designated **initial state** and **goal state**.
- When a problem is defined with *all* its states, it is a **complete state space**.
- A state space is drawn as a **directed graph**, or a **tree** when no state repeats. Nodes are search states, arcs are the operators.

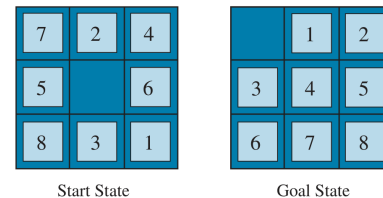
Why enumerating rules by hand does not work

- **The chess example** (*Insights* §2.1.1). A knight at (4, 4) can move to (2, 3), (2, 5), (3, 2), (3, 6), (5, 2), (5, 6), (6, 3) and (6, 5).
- Writing that out as a rule for every square gives an enormous rule set, and a chess position has around 10^{120} complete game paths.
- **The fix, and the point of the example:** describe the move as a **pattern** rather than a list, i.e. current position $\pm$ (2 vertical + 1 horizontal) or (1 vertical + 2 horizontal). One rule replaces sixty-four.
- This is exactly why the next section exists: a **production system** is a set of patterns, not a table of cases.

The five terms, with the 8-puzzle as the example

Term	Means, and for the 8-puzzle
State	A configuration of the world the agent may be in. <i>The positions of the eight tiles and the blank.</i>
Initial state	Where the agent starts. <i>The given starting board.</i>
Successor function	Given a state, returns the set of (action, resulting state) pairs reachable in one step. <i>Move the blank left, right, up or down.</i>
Goal test	Decides whether a state is a goal. <i>Does the board match the given goal board?</i>
Path cost	The sum of the step costs along a path. <i>1 per move, so the cost is the number of moves.</i>

- A **solution** is a sequence of actions from the initial state to a goal state. An **optimal solution** is one of lowest path cost.



Draw **both boards**: the initial state and the goal test are two of the five terms and neither means anything without them. The shortest solution from this pair is 26 moves.

Problem space, and how to answer “define a problem”

- **Problem space** = every state reachable, plus every action that can be taken, for a given goal. Eg the highways and cities of Nepal, when the problem is “get from Kathmandu to Dharan”.
- **The four-line answer that scores full marks** for “what are the criteria for defining a problem” (82 Ka 2):
 - define the **state space** the problem lives in,
 - specify the **initial state**,
 - specify the **goal state** or a goal test,
 - specify the **operators**, with their preconditions and effects.
- Add **path cost** if the question mentions optimality or “best”.

2.2 Well-defined problems

What makes a problem well defined? List the characteristics 2 3 4 8

HOT 7/41 71 Bh 3+4, 75 Ba 2, 80 Ash 2, 78 Ch 2, 81 Ba 2, 75 Bh 2, 81 Ash 8 · almost always a 2-mark opener to a water-jug or crypt-arithmetic question, so answer it in four bullets and move on. Only 81 Ash makes it the whole 8-mark question

Definition

- A problem is **well defined** when the agent knows, before it starts, exactly what the problem consists of. Formally, it is composed of:
 - an **initial state**,
 - the **actions**, given by a successor function,
 - a **goal test**, and
 - a **path cost** function.
- Equivalently: the starting position, the allowable operations and the goal state are all clearly specified.

Characteristics (81 Ash asks for these by name, 8)

- **Unambiguous**: one reading of what is being asked.
- **Complete**: everything needed to solve it is stated; nothing has to be assumed.
- **Verifiable**: the goal test can decide, mechanically, whether a state is a solution.
- **Finite and discrete** state space, so the states can be enumerated.
- **Deterministic operators** with known preconditions and effects.
- **Solvable in principle**: at least one action sequence reaches the goal.

Well defined Vs ill defined, with examples

	Well defined	Ill defined
Goal	Stated exactly	Vague or subjective
States	Enumerable	Open-ended
Operators	Fixed and known	Not fully known
Test	Mechanical	Needs judgement
Example	Water jug, 8-puzzle, chess, crypt-arithmetic	“Write a good poem”, “design a beautiful building”, medical diagnosis from an unclear history

- **The 5-litre / 2-litre trap** (81 Ash): the paper asks “*is this a well-defined problem? Justify.*” The answer is **yes** – the jugs, the pump, the operators and the target are all given, so every clause above is satisfied. Say that, then solve it. The companion works the solution.

2.3 Problem types and classification

Problem types and how a problem is classified 2 3

syllabus, no PYQ yet as a question of its own · but “problem types” is a printed syllabus line, and the *ignorable* / *recoverable* / *irrecoverable* split is worth knowing because it explains why the water jug is easy and tic-tac-toe is not

By whether a move can be undone (*Insights* §2.2.1)

- **Ignorable** – intermediate steps can simply be ignored; a wrong move costs nothing. *Eg* theorem proving, the **water jug**.
 - Solvable with a simple control strategy, no backtracking needed.
- **Recoverable** – steps can be undone to get back to an earlier state. *Eg* the **8-puzzle**.
 - Needs backtracking, so the control strategy must keep a history.
- **Irrecoverable** – steps cannot be undone. *Eg* **tic-tac-toe**, chess.
 - Needs **planning**: consider the consequences before moving, because there is no second chance. This is why games need adversarial search (Ch 3).
- **Decomposable** – the problem breaks into independent sub-problems whose answers combine. *Eg* symbolic integration, word puzzles.

By what the agent can see and predict

- **Single-state problem** – deterministic and fully accessible. The agent always knows exactly which state it is in. *Eg* chess with the board in view.
- **Multiple-state problem** – deterministic but inaccessible. The agent knows the rules but not its exact state, so it reasons over a *set* of possible states. *Eg* walking across a dark room. It may have no sensors at all.
- **Contingency problem** – non-deterministic and inaccessible. The environment is partially observable or the actions are uncertain, so the agent **must sense during execution** and cannot plan the whole path in advance.
- **Exploration problem** – the state space itself is unknown. The agent discovers states and operators as it acts. *Eg* a robot in an unmapped maze. **The hardest of the four**, and the one that needs learning.

2.4 Production systems

Define a production system. What are its essential terms? Explain its role with an example 2 4 6

HOT 5/41 70 Ma 2+6, 71 Bh 4, 82 Ka 2, 77 Ch 2, 75 Bh 2+6 · 70 Ma pairs it with *why is searching necessary in AI*; 75 Bh pairs it with *well-defined problems*. It is the machinery behind the water-jug rules, so answer it with those rules as the example

Definition and the four components

- A **production system** is a set of **rules**, each of the form $C_i \rightarrow A_i$:
 - **LHS** C_i , the **pattern** or condition, which decides whether the rule applies;
 - **RHS** A_i , the **action** to perform if it is applied.
- The right-hand side is read as an **action recommendation**, not as a logical conclusion. That is what separates a production rule from an implication.
- **The four components**, and the answer to “essential terms”:
 - **Rule base** (production rules / rule memory) – the rules themselves.
 - **Working memory** (database) – the current state of the world, updated as rules fire.
 - **Control strategy** (inference engine) – decides which applicable rule to fire when several match, and when to stop.
 - **Rule applier** – carries out the chosen action.
- **The recognise-act cycle**: **match** the rules against working memory → **conflict resolution** picks one from the matching set → **act**, i.e. fire it and update working memory → repeat until the goal test passes or nothing matches.

Two requirements of a good control strategy

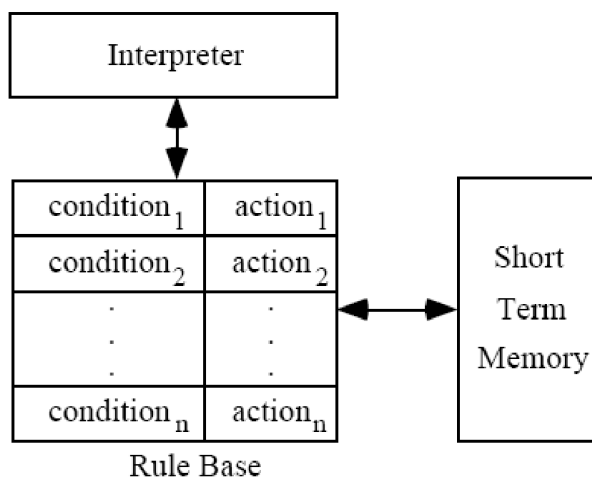
- **It must cause motion.** A strategy that can pick the same rule forever never leaves the initial state.
- **It must be systematic.** Choosing at random causes motion but wanders, and may revisit states endlessly.
- Both are one-line answers and both are asked.

Example: the vacuum world

Pattern (LHS)	Action (RHS)
[A, clean]	move Right
[A, dirty]	Suck
[B, clean]	move Left
[B, dirty]	Suck

- **The example that earns the marks** is the **water jug**, whose eight production rules are in the companion. A question asking for a production system with an example almost always means that one.
- **Practical uses**: expert systems (Ch 7), autonomous robots, and cognitive architectures modelling human reasoning such as ACT and SOAR.

The architecture, which is what to draw



- **The same four components under other names.** This drawing says **Interpreter** for the control strategy and inference engine, **Short Term Memory** for working memory, and folds the rule applier into the interpreter. Either set of names is accepted; **use one set consistently** and say which.
- Read the two arrows as the **recognise-act cycle**. The interpreter matches the rule base against short term memory, resolves the conflict set down to one rule, and fires it; firing rewrites short term memory, and the cycle repeats.
- **Both arrows are double-headed**, which is the part worth noticing: the rules read the state and the state is rewritten by the rules. Nothing else in the system holds any information, so **the whole state of the computation is the contents of that one box**.

Problems solvable by production rules 6

75 Bh 2+6 · asked as “explain about problems that can be solved using production rules with an example”

Which problems suit a production system

- The state can be written down **compactly** – the water jug’s whole state is the pair (x, y) .
- The legal moves are **few, uniform and pattern-like**, so a handful of rules covers every case.
- Progress is made by **repeatedly rewriting the state**, not by evaluating a formula.
- The goal is a **recognisable pattern**, so the goal test is a match.

The standard examples, and what each shows

- **Water jug** – 8 rules, state (x, y) . The canonical answer.
- **8-puzzle** – 4 rules (move the blank up, down, left, right), state = the board. Shows a recoverable problem needing backtracking.
- **Missionaries and cannibals, farmer/wolf/goat/cabbage** – rules with a **safety precondition**, which is what makes them interesting.
- **Tower of Hanoi** – shows a decomposable problem.
- **Tic-tac-toe** – shows an irrecoverable one, where rules alone are not enough and search over an opponent is needed.

2.5 Constraint satisfaction problems

What is a Constraint Satisfaction Problem? Discuss it and list its constraints 1 2 3

TOP 12/41 81 Bh, 80 Bh, 80 Ba, 78 Ba, 80 Ch, 74 Bh 2, 79 Jth 1, 79 Ch 3, 74 Ma, 72 Ash, 78 Po, 70 Bh · **never asked alone**: it is the 1–3 mark preamble to a crypt-arithmetic puzzle in every one of these papers, so keep it short and get to the puzzle

Definition

- A **constraint satisfaction problem** is a search procedure that **operates in a space of constraints** rather than a space of states.
- States are defined by the **values assigned to variables**, and the goal test is a set of **constraints those values must obey**.
- Formally, a CSP is the triple:
 - **Variables** $X = \{X_1, X_2, \dots, X_n\}$, the unknowns to be assigned.
 - **Domains** $D = \{D_1, D_2, \dots, D_n\}$, the set of values each variable may take.
 - **Constraints** $C = \{C_1, C_2, \dots, C_m\}$, the restrictions on which combinations of values are allowed.
- **Solution**: an assignment of one domain value to every variable that violates no constraint. **Write these three lines first in any crypt-arithmetic answer** – they are worth marks on their own.

How it is solved: constraint propagation

- Constraints are **discovered and propagated as far as possible** through the system. Where propagation stalls, a **guess** is made and added as a new constraint, and propagation resumes.
- If the guess leads to a contradiction, **backtrack** and guess otherwise.

The two termination conditions

- A **contradiction is detected** – no solution is consistent with the constraints now known.
- **Propagation has run out of steam** – no further change can be made from current knowledge. Either a solution has been found, or a guess must be made.
- Both are asked as a pair. Two lines, two marks.

Types of constraint

- **Unary** – restricts one variable. *Eg* a leading letter is not 0.
- **Binary** – relates two. *Eg* two regions differ in colour.
- **Higher-order** – relates three or more. *Eg* a crypt-arithmetic column, $D + E = 10C_1 + Y$.
- **Global / AllDiff** – all variables take different values. Every crypt-arithmetic puzzle carries this one.

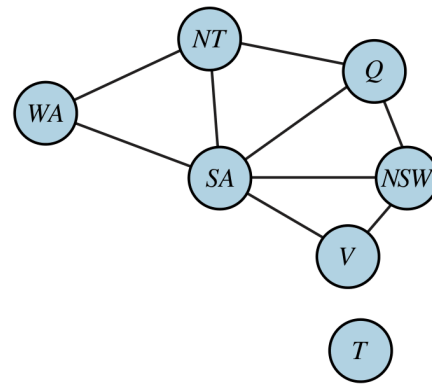
Other standard CSP examples

- **Map colouring** – no two adjacent regions share a colour.
- **n -queens** – no queen attacks another.
- **Class scheduling, VLSI layout, Sudoku**.

Map colouring, and the same problem as a constraint graph



(a)



(b)

(a) the seven regions, (b) the CSP. Variables become **nodes**, binary constraints become **edges**; here every edge means “these two must differ”. The domain of each variable is {red, green, blue}. · **Two things to read off (b) that (a) hides**: *SA* touches five of the other six, so it is the variable to assign first and the one a most-constrained-variable heuristic picks; and *T* has **no edge at all**, so Tasmania is an independent sub-problem that can be coloured any colour whatever the rest does. · That is the whole point of drawing the graph: the structure of the problem is visible in it and is invisible in the map.

The crypt-arithmetic rules, and how to set the answer out 6 7 8

TOP 31/41 · the highest-value single technique in the subject · every puzzle is worked in Ch2 – Problem Solving – Solved Problems

The rules, as every paper states them

- Different letters denote different digits, and identical letters denote the same digit. This is the AllDiff constraint.
- No leading letter may be 0.
- Digits come from $D = \{0, 1, 2, \dots, 9\}$, so a puzzle with more than **10 distinct letters has no solution at all**.
- Any extra restriction the question prints overrides these.

The five-step method

(Insights §2.3)

1. Write the sum in columns with a **carry variable** $C_1, C_2, \dots$ above each column boundary.
2. List X , the set of variables.
3. List D , the domain.
4. List C : the leading-letter constraints, the AllDiff constraint, and **one equation per column**, $(\text{column sum}) + C_{\text{in}} = 10 C_{\text{out}} + (\text{result digit})$.
5. Reason column by column, redrawing the sum with the digits filled in after each deduction, until every letter is fixed. Close with the arithmetic check.

The three deductions that crack most puzzles

- **A carry out of the leftmost column can only be 1**. If the answer is one digit longer than the addends, that leading digit is 1. This is what gives $M = 1$ in $\text{SEND} + \text{MORE} = \text{MONEY}$ immediately.
- **A letter added to itself**. $2N$ ends in the same letter only when that letter is 0 or 5, which is how $\text{TEN} + \text{TEN} + \text{FORTY} = \text{SIXTY}$ is cracked.
- **A column where a letter repeats**. If $E + O + C_2 = N$ and also $N + R + C_1 = E + 10$, substitute one into the other; the unknowns cancel.

Two things that lose marks

- **Giving only the digits**. Most of these puzzles have many solutions, so an unjustified assignment cannot be marked. Show the columns.
- **Forgetting the AllDiff check at the end**. It is the single commonest way a nearly-right answer fails, and it is what rules out $B = 9$ in the $AB + CD = AAA$ question (76 Bh).

2.6 Game playing

Formulate a game as a search problem 2 7

PIN 1/41 72 Ma 7, tic-tac-toe state space · the algorithms live in §3.4, not here. This section only owes you the formulation; minimax and alpha-beta are Ch 3, where they are asked 14 times

A game, defined as a search problem

- **Initial state** – the board as set up, plus whose turn it is.
- **Operators** – the legal moves.
- **Terminal test** – says when the game is over.
- **Utility / payoff function** – a number on each terminal state saying who won and by how much. Eg +1 win, 0 draw, -1 loss.
- **What makes it different from ordinary search:** there is an **opponent**, choosing to make things as bad as possible for you. So a plan cannot be a fixed sequence of moves; it must be a **strategy**, an answer for every reply.
- Two-player, perfect information, zero-sum games are solved by **minimax** (§3.4), made efficient by **alpha-beta pruning**.

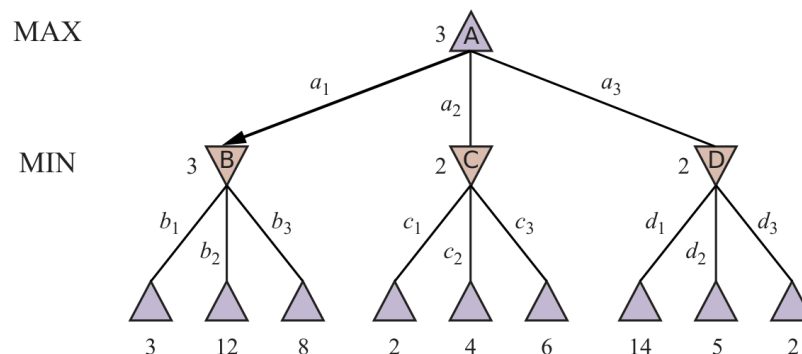
Why games were chosen as an AI testbed

- Rules are **simple and unambiguous**, so no knowledge acquisition problem gets in the way.
- The state is **fully observable**, so no perception problem either.
- Yet the **search space is enormous**, so brute force fails and real technique is needed. That combination is rare and useful.
- Performance is **measurable** against human players.

Production systems Vs game playing

- A production system fires rules against **its own** working memory.
- A game must additionally model **another agent choosing against it**, which no rule set alone can express. That is precisely why Ch 3 needs a separate family of algorithms.

The formulation above, drawn



MAX moves from the root, MIN replies, and the nine leaves carry the **utility**. The number written on each internal node is backed up from its children, and the arrow marks MAX's choice. · **The tree is the strategy**, which is the point of the whole section: MAX cannot commit to a sequence, only to an answer at every node MIN might send it to. · Computing those backed-up numbers is minimax, and it is §3.5, not here.

3. Search Techniques

9 hrs · asked in **all 41 papers** · 3 syllabus sub-topics · 392 marks, 11.6% of every mark ever set · worked traces in Ch3 – Search Techniques – Solved Problems.

Read this first:

- **The chapter is three comparisons and one algorithm.** Learn **BFS Vs DFS** **TOP 10/41**, **informed Vs uninformed** **HOT 4/41**, **A* Vs greedy** **HOT 7/41**, and the **A* algorithm** itself, and you have answered most of what this chapter has ever asked.
- **One table carries the whole of §3.2.** Completeness, optimality, time, space, for BFS and DFS. It is asked in **10 of the 41 papers** and it is four rows. Nothing else in the subject pays as well per line learnt.
- The other three reliable topics: **hill climbing's three problems and their cures** **HOT 6/41**, **minimax + alpha-beta** **HOT 6/41**, and **why searching is necessary in AI** **HOT 5/41**, which is a 2-mark opener bolted onto a bigger question five times.
- **Simulated annealing is asked four times and is not a syllabus line.** It only ever appears as “how does simulated annealing overcome the problems of hill climbing”, so learn it as hill climbing's cure, not as a topic.
- Five papers print a **figure** and ask you to run an algorithm on it: A* on a graph (81 Ch, 80 Ch), best-first on a tree (76 Bh), minimax with alpha-beta on a tree (79 Ch, 76 Ba). All five are worked in the companion. **Two of those papers print the same graph** with different heuristics.
- Pairs the examiner reliably asks together: **why search + BFS Vs DFS, informed Vs uninformed + hill climbing**, and **greedy's drawback + how A* fixes it**.

3.1 Why search, and how a search algorithm is judged

Why is searching necessary in AI? On what basis are search algorithms evaluated? **2 3 4**

HOT 5/41 72 Ma **2**, 74 Bh **2**, 78 Ch **2**, 80 Ash **2**, 82 Ka **3** · evaluation criteria alone in 3 papers, 79 Ash **4**, 75 Bh **4**, 81 Ash **2**, none over 4 marks so no tier chip · **always the 2-mark opener** to a BFS/DFS or hill-climbing question

Why searching is necessary

- Most AI problems have **no formula** that computes the answer. There is only a start state, a set of legal moves and a test for the goal, so the answer has to be **found** rather than calculated.
- Once a problem is written as a state space (Ch 2), **solving it is searching that space** for a path from initial state to goal.
- The space is far too large to enumerate: chess has around 10^{120} game paths, the 24-puzzle around 10^{25} states. **Brute force is not an option**, so the search must be directed.
- Search is the mechanism behind almost every other topic on this syllabus: game playing (§3.4), resolution refutation (Ch 4), rule chaining, planning, and the learning algorithms of Ch 6.
- **The one-line answer:** searching is how an agent converts knowledge of the rules into a sequence of actions, when it has no direct way to compute one.

The four evaluation criteria

- **Completeness** – is the algorithm guaranteed to find a solution when one exists?
- **Optimality** – when it finds one, is it guaranteed to be the *least-cost* solution?
- **Time complexity** – how many nodes are generated?
- **Space complexity** – how many nodes must be held in memory at once?
- **Say what the symbols mean** or the complexities score nothing:
 - b = **branching factor**, the maximum successors of any node;
 - d = **depth of the shallowest goal**;
 - m = **maximum depth** of the search tree, possibly infinite;
 - ℓ = the **depth limit**, for depth-limited search.
- Some papers add **admissibility** (does it find the optimal path when one exists) and **dominance** (does one heuristic expand fewer nodes than another). Mention both in a 4-mark answer.

3.2 Uninformed (blind) search

Explain BFS and DFS and compare their performance criteria 4 7 8 9

TOP 10/41 72 Ash 9, 69 Po 9, 70 Bh 4+4, 70 Ma 1+7, 73 Bh 4+5, 79 Bh 7, 79 Jth 7, 74 Bh 8, 71 Bh 8, 72 Ma 7 · the single most-asked question in the chapter · 71 Bh and 72 Ma additionally ask why these are called blind

What “uninformed” means

- The algorithm is given **nothing beyond the problem definition**: it can generate successors and test for the goal, and that is all.
- It has **no estimate of how close a state is to the goal**, so it cannot prefer one frontier node over another on merit. Hence **blind search**. That is the answer to “why are BFS and DFS called blind even though they search systematically”: systematic is not the same as informed.

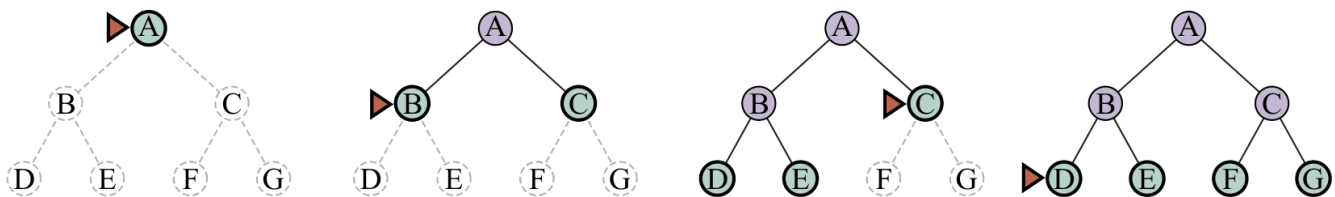
Breadth first search

- Expand **all nodes at depth k before any at depth $k + 1$** . **FIFO queue**.
- ✓ Complete, and ✓ optimal *when every step costs the same*.
- × Memory is the killer: it must hold the whole frontier, $O(b^d)$ nodes.

Depth first search

- Expand the **deepest** unexpanded node; go back only when stuck. **LIFO stack**, or plain recursion.
- ✓ Memory is linear, $O(bm)$: only the current path and its siblings.
- × Not complete (it can run down an infinite branch or loop), and × not optimal (it returns the first goal found, however deep).

The same tree, the same goal M , the two orders

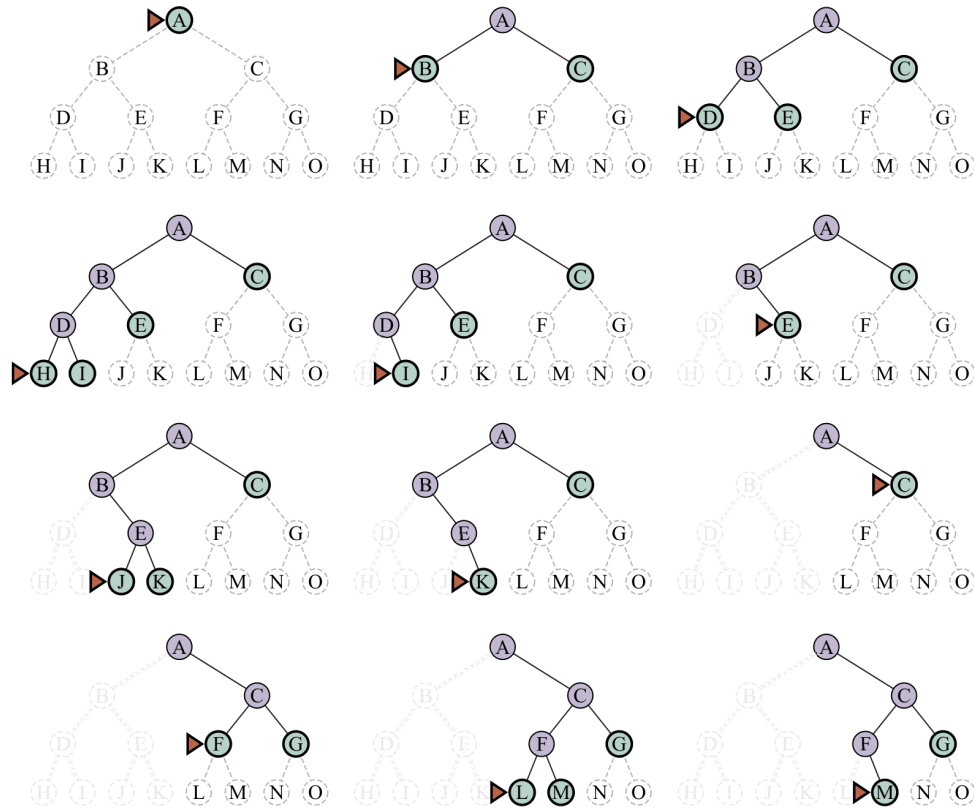


BFS: the frontier is a whole *level*. Four steps, and it has not gone below depth 2. Every node of a level is in memory at once, which is the $O(b^d)$ space.

The comparison table. Learn this.

	BFS	DFS	Why
Data structure	FIFO queue	LIFO stack	shallowest Vs deepest first
Complete	✓ yes	× no	DFS can descend forever
Optimal	✓ yes (unit step cost)	× no	BFS meets the shallowest goal first
Time	$O(b^d)$	$O(b^m)$	$m \geq d$, so DFS can be far worse
Space	$O(b^d)$	$O(bm)$	the real difference
Good when	the goal is shallow, memory is plentiful	the tree is deep, memory is tight, solutions are dense	

- **The sentence that earns the last mark**: BFS trades *memory* for a guarantee; DFS trades the *guarantee* for memory.
- With $b = 10$ and $d = 12$, BFS needs about 10^{12} nodes in memory and DFS about 120. That is the whole argument in one number.



DFS, twelve steps, left to right then top to bottom. The frontier is a single *path*, and the faded nodes have been dropped from memory already: that is the $O(bm)$ space. Note steps 7 and 10, where it **backs up** after a dead end. It reaches *M* at step 12, having never looked at *N* or *O*.

Depth limited search (DLS) (71 Bh 4+4, 79 Ash 4)

- DFS with a hard cut-off at depth ℓ : treat any node at depth ℓ as having no successors.
- **The advantage**, which is what the paper asks: it **fixes DFS's infinite-branch failure** while keeping DFS's $O(b\ell)$ memory.
- $\times$ Still not optimal, and $\times$ incomplete in a new way: if $\ell < d$ it reports failure even though a solution exists. If $\ell > d$ it is not optimal either.
- Choosing ℓ needs knowledge of the problem. Eg for a route between 20 cities, no path needs more than 19 steps, so $\ell = 19$.

Iterative deepening (IDS) verified/added

- Run DLS with $\ell = 0, 1, 2, \dots$ until a goal is found.
- **Gets BFS's guarantees at DFS's memory cost**: complete, optimal for unit steps, $O(b^d)$ time, $O(bd)$ space.
- The repeated work is negligible: re-expanding the top of the tree costs a constant factor, because the bottom level has more nodes than all the levels above it combined.
- **The preferred blind search** when the depth is unknown. Worth one line whenever a paper asks for a comparison of blind strategies.

Informed Vs uninformed search 3 4 8

HOT 4/41 74 Ma 8, 81 Ba 4+4, 81 Bh 3+6, 72 Ash 8 · in all four papers it is the **first half of a hill-climbing question**, so answer it in a table and spend the time on the second half

	Uninformed (blind)	Informed (heuristic)
Extra knowledge	None beyond the problem definition	A heuristic $h(n)$ estimating the cost from n to the goal
Chooses by	Position in the queue or stack	The value of an evaluation function
Efficiency	Explores blindly, generates many useless nodes	Explores towards the goal, generates far fewer
Completeness	BFS, UCS, IDS yes; DFS no	A* yes; greedy and hill climbing no
Optimality	BFS and IDS for unit cost, UCS always	A* with an admissible h ; greedy never
Cost	Cheap per node	Needs a heuristic, which costs domain knowledge to design and time to evaluate
Examples	BFS, DFS, DLS, IDS, uniform cost	Best first, greedy, A*, hill climbing, simulated annealing

- **The line that makes the difference concrete:** on the same problem an uninformed search may generate millions of nodes where an informed one generates hundreds. The heuristic is not a different algorithm, it is **knowledge added to the same algorithm**.

3.3 Informed search: best first, greedy and A*

Explain A* search. How does it overcome the drawback of greedy best-first search? 2 4 6 8

HOT 7/41 73 Ma 8, 77 Ch 8, 78 Po 8, 73 Bh 8, 82 Ba 2+6, 78 Ba 6, 82 Bh 4+5 · short note on A* alone at **HOT 5/41** 72 Ma, 79 Jth, 81 Bh 4, 80 Ba, 76 Ba 3 · **always phrased as a comparison**, so never explain A* without saying what greedy gets wrong

The three evaluation functions

- $g(n)$ = the cost **already paid** to reach n from the start.
- $h(n)$ = the **estimated** cost from n to the goal, the heuristic.
- **Best first** search is the family: keep an **OPEN** list of generated but unexpanded nodes and a **CLOSED** list of expanded ones, and always expand the OPEN node with the best evaluation.
 - **Greedy best first:** $f(n) = h(n)$. Ignores what has already been spent.
 - **Uniform cost:** $f(n) = g(n)$. Ignores the goal. This is blind search.
 - **A*:** $f(n) = g(n) + h(n)$. **Uses both.**

The A* algorithm, as it should be written

1. Put the start node on OPEN with $g = 0$, $f = h(\text{start})$.
2. If OPEN is empty, fail.
3. Remove the node of **lowest** f from OPEN, call it n , and put it on CLOSED.
4. If n is the goal, stop; trace the parent pointers back for the path.
5. Expand n . For each successor m : compute $g(m) = g(n) + \text{cost}(n, m)$ and $f(m) = g(m) + h(m)$; record n as its parent; and if m is already on OPEN or CLOSED with a **larger** g , **replace the old entry and reopen it**.
6. Re-order OPEN by f and go to 2.

Admissibility and optimality of A* 4+5

PIN 1/41 82 Bh 4+5 · asked as “define admissibility and optimality; under what conditions is A* guaranteed to be both”, so the two **conditions** are the answer

The two properties of a heuristic

- **Admissible:** $h(n) \leq h^*(n)$ for every n , where h^* is the true cost to the goal. The heuristic **never overestimates**; it is optimistic.
 - Guarantees A* is optimal in **tree search**.
- **Consistent (monotone):** $h(n) \leq \text{cost}(n, m) + h(m)$ for every edge, and $h(\text{goal}) = 0$. A triangle inequality on the heuristic.
 - Guarantees A* is optimal in **graph search**, i.e. with a CLOSED list that is never revisited. Consistency implies admissibility; the converse is false.
- **Conditions for A* to be admissible and optimal:** h admissible, every step cost > 0 , and the branching factor finite. Add *consistent* if a closed list is used.

Greedy's drawback, and A*'s fix

- Greedy asks only “which node looks nearest to the goal?”. It therefore **walks into whatever the heuristic flatters**, and a long detour that starts with a promising-looking step will be taken in full.
- $\times$ Greedy is **not optimal** and **not complete**: it can loop between two nodes that each look good from the other.
- A* adds $g(n)$, so **a node is judged on the whole journey, not the remainder**. A cheap-looking node that cost a fortune to reach is ranked accordingly.
- **The worked case in the companion:** on the 80 Ch graph, greedy takes $S \rightarrow C \rightarrow D \rightarrow F \rightarrow G$ at cost **25**, which is the *worst* of the four routes, because $h(C) = 5$ looks closest. A* takes $S \rightarrow B \rightarrow E \rightarrow F \rightarrow G$ at cost **18**, the optimum.

Comparison in one line each

- **Greedy:** fast, small memory, no guarantees.
- **A*:** optimal and complete with an admissible h , but stores every generated node, so **memory is its limit**.

Why this is not a pedantic point

- Both graphs printed in these papers carry heuristics that are **admissible but not consistent**. In 81 Ch, $h(B) = 13$ while $\text{cost}(B, E) + h(E) = 6 + 4 = 10$.
- The consequence is visible in the trace: A* first reaches E at $g = 12$ through A , and later finds E at $g = 11$ through B . **An implementation that refuses to reopen a closed node returns 19 instead of the optimal 18.**
- So step 5 of the algorithm above, the *replace and reopen* clause, is not decoration. Write it, and the trace comes out right.
- **Dominance:** if $h_2(n) \geq h_1(n)$ for all n and both are admissible, h_2 *dominates* h_1 and A* with h_2 expands no more nodes. A better heuristic is one that is larger while staying under h^* .

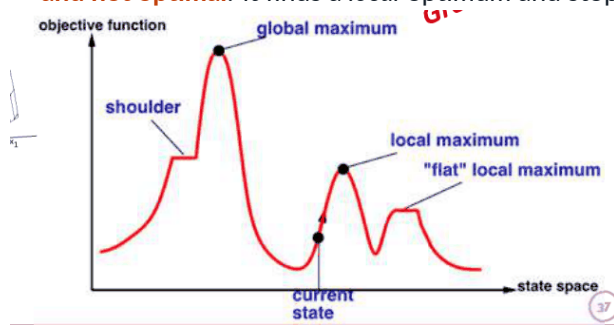
3.4 Hill climbing and simulated annealing

Discuss hill climbing, the problems associated with it, and their solutions 3 4 6 9

HOT 6/41 69 Bh 9, 75 Ba 6+2, 80 Ba 3+5, 75 Bh 4, 77 Ch 4, 79 Bh 4 · **three problems, three cures**, and the marks are split evenly between them. Four of these six papers then ask how simulated annealing fixes it

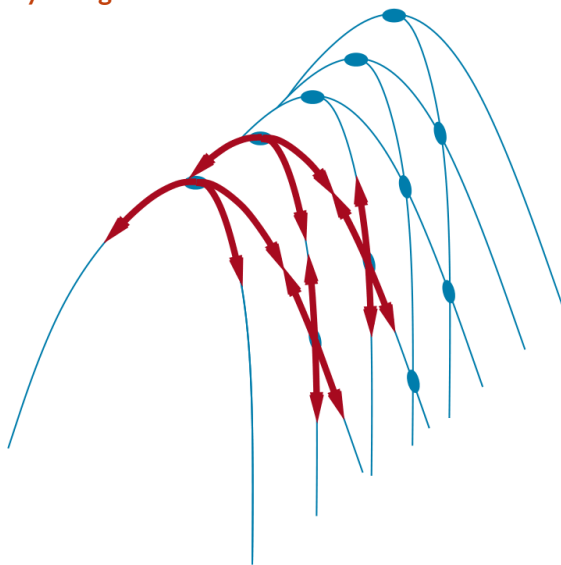
The algorithm

- A **local search**: keep only the current state, and move to a neighbour that improves the objective function. No frontier, no backtracking, no memory of where it has been.
- **1.** Evaluate the initial state; if it is the goal, stop. **2.** Generate a neighbour. **3.** If the neighbour is better, make it the current state. **4.** Repeat until no neighbour is better.
- **Variants**: *simple* hill climbing takes the first better neighbour; *steepest ascent* evaluates all neighbours and takes the best; *stochastic* picks randomly among the better ones.
- ✓ Almost no memory, and very fast. × **Not complete and not optimal**: it finds a *local* optimum and stops.



The whole answer in one picture: climbing from the current state reaches the local maximum, not the global one.

Why a ridge is the hard one

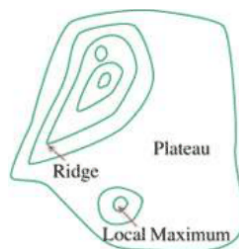


The three problems, and their cures

Problem	What happens	Cure
Local maximum	A peak higher than all its neighbours but lower than the global maximum. Every move looks worse, so the search stops on a suboptimal answer.	Backtrack to an earlier node and try a different direction. Or restart from a new random state (random-restart hill climbing).
Plateau	A flat region where all neighbours score the same, so the search has no gradient to follow and wanders at random.	Make a big jump in some direction, to reach a new region of the space. A single step is not enough to get off it.
Ridge	A sequence of local maxima along a slope where the ascent runs <i>across</i> the ridge rather than along it, so every single-step move goes down.	Apply two or more rules at once before testing, i.e. move in several directions at a time. Bidirectional search also helps.

- **Name all three and pair each with its own cure.** A generic “use random restarts” for all three loses half the marks.

- The dots are **states**; they sit on a ridge rising left to right. The dark arrows are the moves available from each state.
- **Every one of those moves goes down.** The ascent runs *across* the ridge, not along it, so no single step improves anything and hill climbing halts on a slope that is not a maximum at all.
- That is exactly why the cure is **apply two or more rules at once**: one step in any single direction cannot go up, but a combination can.
- **A ridge needs two dimensions to exist**, which is why the one-dimensional landscape on the previous page cannot show one. If a paper asks for “the problems of hill climbing with diagrams”, **this is the one people leave out**.



The same landscape as a contour map, which shows all three problems at once: the **plateau** is the wide gap between contours, with no gradient anywhere on it; the **ridge** is the narrow crest on the left; the **local maximum** is the small closed loop at the foot.

Simulated annealing 4 5 6

HOT 4/41 75 Ba 2, 80 Ba 5, 81 Ba 4, 81 Bh 6 · **not a syllabus line**, but asked four times and always as “how does it overcome the demerits of hill climbing”

The idea

- Named after **annealing** in metallurgy: heat a metal, then cool it slowly so its atoms settle into a low-energy crystal instead of freezing into a flawed one.
- Hill climbing accepts *only* improving moves, which is exactly why it gets stuck. **Simulated annealing sometimes accepts a worse move**, and does so less and less as the search proceeds.
- A move is picked at random:
 - if it **improves** the state, accept it always;
 - if it **worsens** the state by ΔE , accept it with probability $P = e^{-\Delta E/T}$.
- T is the **temperature**, lowered on a fixed **annealing schedule**.

Why that fixes hill climbing

- An accepted downhill move is exactly what **escapes a local maximum**: the search can go down one side of the small peak and up the big one.
- It also **crosses a plateau**, since a sideways or slightly worse move costs almost nothing in probability.
- **The role of T** , which is where the marks are:
 - **High $T \rightarrow P$ near 1**, most bad moves accepted, the search explores wildly. Nearly a random walk.
 - **Low $T \rightarrow P$ near 0**, bad moves refused, the search becomes **ordinary hill climbing**.
 - Starting hot and cooling slowly explores first and refines later.
- Note the shape of $e^{-\Delta E/T}$: a *slightly* worse move is much likelier to be accepted than a much worse one, at any temperature.
- **The theoretical guarantee worth quoting**: if T is lowered slowly enough, simulated annealing finds the global optimum with probability approaching 1.

3.5 Adversarial search: minimax and alpha-beta

Explain the min-max algorithm. What are its drawbacks, and how does alpha-beta pruning solve them? 2 3 4 6

HOT 6/41 71 Ma 2+4+3, 77 Ch 2+6, 82 Ka 6, 81 Ash 6, 76 Bh 4, 80 Bh 2+6 · short notes on the same at **HOT 6/41** 78 Ba, 74 Bh, 74 Ma, 72 Ash, 78 Po 4, 82 Ba 3 · **12 papers between them**, which makes this the second-biggest topic in the chapter

Minimax

- For a **two-player, zero-sum, perfect-information** game. One player (**MAX**) maximises the score, the other (**MIN**) minimises it.
- Build the game tree to the terminal states, score each with the utility function, then **propagate values upward**:
 - at a **MAX** node, take the **largest** child value;
 - at a **MIN** node, take the **smallest**.
- The root’s value is the best MAX can force **against an opponent playing perfectly**, and the move to play is the child that achieves it.
- Implemented as a **depth-first** traversal, so space is $O(bm)$; time is $O(b^m)$.

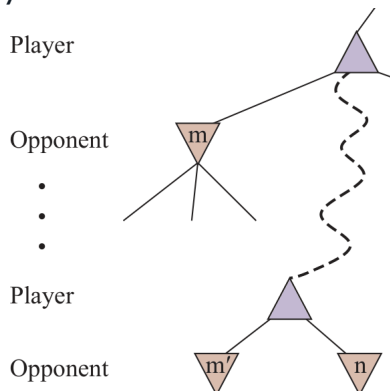
The drawbacks (the part the papers ask for)

- **Exponential time**, $O(b^m)$. Chess has $b \approx 35$ and $m \approx 100$, so the complete tree can never be built. **This is the drawback**; everything else is a consequence.
- It **examines every leaf**, including branches whose outcome cannot possibly change the root value.
- It assumes a **perfect opponent**, so it can play too cautiously against a weak one.
- In practice the tree must be **cut off at a fixed depth** and the cut-off nodes scored by an **evaluation function**, which makes the answer approximate.

Alpha-beta pruning

- **Same answer, fewer leaves.** It returns exactly the minimax value; it only skips work that cannot affect it.
- Carry two bounds down the tree:
 - α = the best value **MAX** can already guarantee. It only rises.
 - β = the best value **MIN** can already guarantee. It only falls.
- **The two cut rules:**
 - at a **MIN** node, if its value $\leq \alpha$, **stop** – MAX will never choose this branch (*alpha cut-off*);
 - at a **MAX** node, if its value $\geq \beta$, **stop** – MIN will never allow it (*beta cut-off*).

Why a cut is safe



The whole justification in one picture. Player is choosing at the top and can already reach m . Somewhere far below, n is a node Player would have to pass through to get anywhere in that branch, and Opponent can hold Player to m' there. · **If m or m' is better than n for Player, then Player will never be allowed to reach n in real play** – so whatever value n turns out to have cannot change the root, and the whole subtree under it can be skipped unexamined. · That is the sentence to write when a paper asks *why* pruning does not change the answer: a pruned branch is one a rational player above it would never let the game enter.

How to run alpha-beta in an exam, without losing the thread

- **Go strictly left to right, depth first.** The examiner's expected answer depends on that order.
- Write (α, β) beside every internal node as you enter it, starting $(-\infty, +\infty)$ at the root.
- A child **inherits** its parent's current α and β .
- After each child returns, update the node's own value, then update α (at a MAX node) or β (at a MIN node).
- Test the cut rule **immediately after each child**, not at the end. Cross out the skipped leaves so the pruning is visible.
- **State how many leaves were pruned** at the end. It is usually worth a mark and it proves the trace was actually run.

What pruning buys, and what it does not

- **Effect:** with a *perfect* move ordering the time complexity falls from $O(b^m)$ to $O(b^{m/2})$, i.e. **the same time searches twice as deep**, or an effective branching factor of $\sqrt{b}$. With random ordering it is about $O(b^{3m/4})$.
- × It does **not** change the value returned.
- × It does **not** help in the worst case: with the best move examined last everywhere, nothing is pruned and the cost is exactly minimax's.
- So it depends entirely on **move ordering**, which is why real game programs invest in ordering heuristics before they invest in searching deeper.

Worked in the companion

- 79 Ch: a 12-leaf, 5-level tree. Minimax value 3; alpha-beta prunes **2 of the 12 leaves**.
- 76 Ba: a 9-leaf, 2-level tree. Minimax value 3; alpha-beta prunes **4 of the 9**, because the first branch returns 3 and sets a high α straight away.
- **The contrast between the two is the lesson:** the second tree prunes twice as large a fraction, purely because its best branch happened to come first.

Other adversarial ideas worth a line

- **Horizon effect:** a fixed-depth cut-off can hide a disaster just beyond the last ply, so the agent plays a move that merely postpones it.
- **Quiescence search:** keep searching past the cut-off while the position is still volatile.
- **Expectiminimax:** for games with chance, add a *chance* node whose value is the expected value over its children.

4. Knowledge Representation, Inference and Reasoning

4 hrs · asked in **all 41 papers** · 4 syllabus sub-topics · 638 marks, **18.8 % of every mark ever set**, against a 7.8 % share of the teaching hours · 33 resolution proofs and 15 Bayes numericals worked in Ch4 – Knowledge Representation, Inference and Reasoning – Solved Problems.

Read this first:

- **This is the highest-paying chapter in the subject, by a factor of 2.4.** Four teaching hours, nearly a fifth of every mark. Whatever time the timetable suggests for it, spend more.
- **One question dominates everything: prove a goal from given premises by resolution refutation** **TOP 33/41**. It is in **33 of the 41 papers**, always for 6 to 10 marks. The premises change; the procedure never does. Learn the procedure once and the question is answered for the rest of your life.
- Its two feeder topics are asked in their own right: **conversion to CNF** **TOP 9/41** and the **rules of inference**. You cannot resolve without CNF, so this is one topic wearing two hats.
- The other two reliable topics: **forward Vs backward chaining** **TOP 11/41**, always a comparison with an example, and **belief / causal networks** **HOT 7/41**.
- **Fifteen papers set a Bayes numerical**, usually bolted onto a 2–3 mark theory opener. The arithmetic is one formula; all 15 are worked in the companion.
- Pairs the examiner reliably asks together: a resolution proof preceded by **the CNF steps, skolemization or the rules of inference**, and a Bayes numerical preceded by **prior/posterior**. The short part is always the definition; the long part is always the work.
- **The 5-mark saver:** if a resolution proof stalls in the exam, write the FOPL, write the clause form, write the negated goal, and show the resolution steps you do have. The marks are spread across those stages, not parked on the empty clause.

4.1 Knowledge, representation and reasoning

What is knowledge, representation and reasoning? What kinds of knowledge are there? **2 2+5**

72 Ash **2+5**, 77 Ch **2**, 72 Ma **7** · never asked alone: it is the opening 2 marks of a chaining or rule-based-reasoning question, so **answer it in four lines and move on**

The three words

- **Knowledge** = facts, concepts, rules and relationships about a domain that an agent can use. It sits above data and information:
 - **data** → raw symbols; **information** → data with context; **knowledge** → information plus how to use it; **intelligence** → knowledge applied to a new situation.
- **Representation** = writing that knowledge in a formal notation a machine can store and manipulate. Not any notation: one that **supports inference**.
- **Reasoning** = deriving new sentences from the stored ones, so the agent knows things nobody explicitly told it.
- The four questions KR exists to answer: how do we represent facts about the world, how do we reason about them, which representation suits the real world, and how do we put it in a form a machine can compute with.

The five kinds of knowledge

Kind	What it is	Example
Declarative	Knowing <i>that</i> . Facts, concepts, objects, stated without saying how to use them	“Kathmandu is the capital of Nepal”
Procedural	Knowing <i>how</i> . Rules, strategies, procedures, agendas	the steps to sort a list
Heuristic	Rules of thumb from experience: good, not guaranteed	in chess, keep the opponent's king exposed
Structural	Relationships between concepts and objects	“a cow <i>is-a</i> mammal”
Meta	Knowledge <i>about</i> knowledge: what the agent knows, and how reliable it is	“my map of Pokhara is out of date”

- **Name all five with one example each.** Papers award a mark per kind, and *meta-knowledge* and *structural* are the two students forget.

What a good representation must give you

- **Representational adequacy** – can it express everything in the domain?
- **Inferential adequacy** – can new knowledge be derived from it?
- **Inferential efficiency** – can the derivation be steered, so the search does not explode?
- **Acquisitional efficiency** – can new knowledge be added easily?

Why logic is the representation of choice

- It is concise, unambiguous, context-insensitive and expressive, and it comes with a **proof theory**, i.e. **inference is defined for you**. That last property is what the rest of this chapter is built on.
- Logic is fixed by three things, and papers ask for them by name: **syntax** (which strings are sentences), **semantics** (what a sentence means about the world) and **proof theory** (which new sentences may be derived from old ones).

What is a knowledge-based agent? How does it work? 2+3+2

PIN 1/41 82 Ba 2+3+2 · no lecture deck in the set covers this by name verified/added · answer it as the *three operations* plus the two levels

Definition and structure

- An agent whose behaviour comes from a **knowledge base** (KB) of sentences about the world, plus an **inference engine** that derives new sentences from it. It **acts on what it can prove**, not on hard-wired reflexes.
- Two levels, and papers like the distinction:
 - **Knowledge level** – *what* the agent knows and what goal it has, independent of any notation.
 - **Implementation level** – the sentences, the data structures and the algorithm that actually run.
- Three operations, in this order, once per time step:
 - **TELL** the KB what was perceived;
 - **ASK** the KB which action to take;
 - **TELL** the KB that the action was taken.

Why it matters, with the standard example

- The **Wumpus world**: an agent in a cave perceives *breeze*, *stench*, *glitter*. No single percept names a pit, but **breeze in (1,2) and (2,1) with none in (1,1)** proves where the pit is. That inference, not the percept, is the whole point.
- ✓ It can be **told new facts** without reprogramming: add a sentence and the behaviour changes.
- ✓ It handles **partially observable** worlds, because it reasons about what it has not seen.
- × It is only as good as its KB, and inference costs time: a large KB with an undirected engine is exactly the search-explosion problem of Ch 3.
- **Every expert system** (Ch 7) is a knowledge-based agent with a domain KB and a chaining inference engine. Say that sentence and the examiner sees you have connected the chapters.

4.2 Propositional logic: WFF, tautology and the rules of inference

Define a WFF with its properties. Prove that $(A \wedge (A \rightarrow B)) \rightarrow B$ is a tautology 4+4

PIN 1/41 71 Bh 4+4 · the proof is a 4-row truth table, worked in the companion · “short note: predicate logic / sentence validity” 75 Bh 4, 72 Ma 4 wants the same vocabulary

Proposition and well-formed formula

- A **proposition** is a declarative statement that is **either true or false, never both**. “5 is a prime number” is one; “close the door” is not.
- An **atomic** sentence is a single propositional symbol (P , Q). A **compound** sentence joins atomic ones with the connectives $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$.
- A **WFF** is a string built only by these recursive rules:
 - any propositional symbol is a WFF;
 - if α is a WFF then $\neg\alpha$ and (α) are WFFs;
 - if α, β are WFFs then so are $(\alpha \wedge \beta)$, $(\alpha \vee \beta)$, $(\alpha \rightarrow \beta)$, $(\alpha \leftrightarrow \beta)$;
 - **nothing else is a WFF**. That closing clause is worth a mark and is the one students leave out.
- Precedence, when brackets are missing: $\neg$, then $\wedge$, then $\vee$, then $\rightarrow$, then $\leftrightarrow$.

The four properties to name

- **Valid (a tautology)**: true under every interpretation. Eg $P \vee \neg P$.
- **Satisfiable**: true under *at least one*. Eg $P \wedge Q$.
- **Unsatisfiable (a contradiction)**: true under *none*. Eg $P \wedge \neg P$.
- **Equivalent**: $\alpha \equiv \beta$ when they have the same truth value in every row. Equivalently, $\alpha \leftrightarrow \beta$ is valid.
- **The link resolution depends on**: α is valid iff $\neg\alpha$ is unsatisfiable. That single line is why proving a goal is done by **refuting its negation** in §4.5.

Equivalences you must be able to quote

- $\alpha \rightarrow \beta \equiv \neg\alpha \vee \beta$ **(the workhorse)**
- $\alpha \leftrightarrow \beta \equiv (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$
- De Morgan: $\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$; $\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$
- Distribution: $\alpha \vee (\beta \wedge \gamma) \equiv (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$
- **Contrapositive** $\neg\beta \rightarrow \neg\alpha$ is equivalent to $\alpha \rightarrow \beta$; the **converse** $\beta \rightarrow \alpha$ and the **inverse** $\neg\alpha \rightarrow \neg\beta$ are **not**.

List the rules of inference. When can we use them? 2 3 2+3

78 Ch 3, 80 Ash 3, 81 Ch 2+3, 76 Ba 2, 70 Ma 3 · 5 papers, none over 3 marks on its own, so no tier chip · **it is always the opener to a resolution proof**, which is where the other 7 or 8 marks are

The eight propositional rules

Rule	From $\therefore$ conclude	Tautological form
Modus ponens	$P \rightarrow Q, P \therefore Q$	$[(P \rightarrow Q) \wedge P] \rightarrow Q$
Modus tollens	$P \rightarrow Q, \neg Q \therefore \neg P$	$[(P \rightarrow Q) \wedge \neg Q] \rightarrow \neg P$
Hypothetical syllogism	$P \rightarrow Q, Q \rightarrow R \therefore P \rightarrow R$	chains two implications
Disjunctive syllogism	$P \vee Q, \neg P \therefore Q$	eliminates a disjunct
Addition	$P \therefore P \vee Q$	weakens
Simplification	$P \wedge Q \therefore P$	splits
Conjunction	$P, Q \therefore P \wedge Q$	joins
Resolution	$P \vee Q, \neg P \vee R \therefore Q \vee R$	the only one §4.5 needs

The four quantifier rules

- **Universal instantiation**: from $\forall x P(x)$ infer $P(c)$ for any constant c .
- **Existential instantiation**: from $\exists x P(x)$ infer $P(c)$ for a **brand-new** constant c . Reusing an existing name is the classic mistake; this is skolemization in one line.
- **Universal generalisation**: from $P(c)$ for an arbitrary c infer $\forall x P(x)$.
- **Existential generalisation**: from $P(c)$ infer $\exists x P(x)$.

When can we use them (the second half of the question)

- Only when the premises **match the pattern exactly**, after unification. A rule is a *syntactic* licence: it never looks at meaning.
- They are **sound**: applied to true premises they can only produce true conclusions, so a derivation is a proof.
- They are used when the KB is in the right form: modus ponens needs implications, **resolution needs CNF** (§4.4), and forward/backward chaining (§4.6) need *definite* clauses.

4.3 First-order predicate logic

Why do we need FOPL? Explain its syntax and quantifiers 2 2+3 4

70 Ma 2+3, 75 Bh 4, 72 Ma 4 · translating English into FOPL is the first step of every resolution proof, so this section is really 33 papers wide · a translation set is worked in the companion (76 Bh 2)

Why propositional logic is not enough

- In PL a whole statement is **one indivisible symbol**. “All humans are intelligent” and “Some humans are intelligent” become two unrelated letters P and Q , with no way to see that one implies the other.
- It cannot express **objects, their properties or relations between them**, and it has **no quantifiers**, so a fact about a million individuals needs a million sentences.
- FOPL fixes exactly that: it breaks a sentence into **subject** and **predicate**, and quantifies over objects.
- The three-line answer: PL has **limited expressive power**; FOPL adds objects, predicates and quantifiers; so one FOPL sentence replaces an unbounded set of PL ones.

Syntax: the seven elements

- **Constants** – *John, Nepal*, 2. Named objects.
- **Variables** – x, y, z .
- **Predicates** – *Brother, Loves, >*. Relations, true or false of their arguments.
- **Functions** – *FatherOf, sqrt*. Return an *object*, not a truth value. **Predicate Vs function is a favourite one-mark trap.**
- **Connectives** $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$, **equality** $=$, and **quantifiers** $\forall, \exists$.

Skolemization (81 Ba 2, 69 Bh, short note)

- **Definition:** removing an existential quantifier by replacing its variable with a *new* symbol, so the sentence can be written as a clause.
- **Two cases, and the second is the one papers test:**
 - $\exists$ **not inside any** $\forall \rightarrow$ replace with a new **Skolem constant**. $\exists x \text{ rich}(x) \Rightarrow \text{rich}(A)$.
 - $\exists$ **inside** $\forall \rightarrow$ replace with a **Skolem function** of the enclosing universal variables. $\forall x \exists y \text{ loves}(x, y) \Rightarrow \text{loves}(x, F(x))$.
- **Why a function and not a constant:** the person loved depends on who is loving. A constant would assert that *everyone loves the same individual*, which is a different and stronger claim, and the proof built on it is wrong.
- The new symbol must be one **not already used** anywhere in the KB.

The two quantifiers, and the connective each takes

- $\forall$ (**universal**, “for all”) goes with $\rightarrow$:
 $\forall x \text{ man}(x) \rightarrow \text{drinks}(x, \text{coffee})$
- $\exists$ (**existential**, “there exists”) goes with $\wedge$:
 $\exists x \text{ student}(x) \wedge \text{fails}(x, \text{maths})$
- **Swap the connectives and the sentence is nonsense.**
 $\forall x (\text{man}(x) \wedge \text{drinks}(x, \text{coffee}))$ says *everything in the universe is a coffee-drinking man*;
 $\exists x (\text{student}(x) \rightarrow \text{fails}(x))$ is satisfied by any non-student. This single pairing earns or loses the translation marks.
- **Duality**, needed in every CNF conversion:
 $\neg \forall x P \equiv \exists x \neg P$ and $\neg \exists x P \equiv \forall x \neg P$.

Translation patterns worth memorising

- “All A are B” $\rightarrow \forall x A(x) \rightarrow B(x)$
- “Some A are B” $\rightarrow \exists x A(x) \wedge B(x)$
- “No A is B” $\rightarrow \forall x A(x) \rightarrow \neg B(x)$, or $\neg \exists x (A(x) \wedge B(x))$
- “A has at least one B” $\rightarrow \exists x B(x) \wedge \text{has}(A, x)$
- “Everyone who ... all animals” $\rightarrow \forall$ *inside* the antecedent of another $\rightarrow$; that nesting is what makes the standard CNF example in §4.4 hard.

Horn clause and definite clause (78 Ch 3, 70 Ma)

- A **clause** is a disjunction of literals: $\neg P \vee Q \vee \neg R$.
- **Definite clause:** a clause with **exactly one positive** literal. $\neg P \vee \neg Q \vee K$.
- **Horn clause:** a clause with **at most one positive** literal. So every definite clause is a Horn clause, and the extra case is the **goal clause** with no positive literal at all.
- **Why the restriction exists**, which is what the 3 marks are for: a definite clause is exactly an implication with a single conclusion, $(P \wedge Q) \rightarrow K$, so
 - inference collapses to **modus ponens**, and
 - **forward and backward chaining** (§4.6) become possible, in time *linear* in the size of the KB rather than exponential.
- **Prolog** is exactly this: a program is a set of definite clauses.

Unification and the most general unifier 2 3

asked inside resolution questions, never alone · **but no resolution step is legal without it**, and examiners take marks off a proof that unifies wrongly

What it does

- **Unification** finds a substitution θ that makes two atomic sentences identical: $\text{UNIFY}(\alpha, \beta) = \theta$ where $\alpha\theta = \beta\theta$.
- The substitution with the fewest commitments is the **most general unifier (MGU)**, and it is the one to use: a more specific one can lose proofs.
- Written $[\text{term}/\text{variable}]$, Eg $\theta = [\text{John}/x]$.
- **It fails**, and the resolution step is then illegal, when
 - the predicate symbols or the arities differ;
 - two different constants must match, Eg $\text{King}(\text{John})$ Vs $\text{King}(\text{Ram})$;
 - a variable would have to be bound to a term containing itself (the *occurs check*).

Worked, in the form the papers expect

- $\text{King}(x)$ and $\text{King}(\text{John}) \rightarrow \theta = [\text{John}/x]$.
- $P(x, y)$ and $P(a, f(z)) \rightarrow \theta = [a/x, f(z)/y]$.
- $P(x, f(y))$ and $P(a, f(g(x))) \rightarrow \theta = [a/x, g(a)/y]$: substitute a for x first, then **apply that binding inside the second term too**. Writing $g(x)/y$ and stopping is the common half-answer.
- $Q(a, g(x, a), f(y))$ and $Q(a, g(f(b), a), x) \rightarrow \theta = [f(b)/x, b/y]$.
- **Standardise apart first**: if both clauses use x , rename one of them. Two clauses that share a variable name will unify wrongly and the proof will be rejected.

4.4 Conversion to conjunctive normal form

Why is CNF required? Explain all the steps involved in converting to CNF, with an example 2+6 3 5

8 10

TOP 9/41 69 Po 10, 70 Bh 8, 75 Ba 2+6, 79 Ch 2+6, 75 Bh 2+6, 76 Bh 5, 80 Ch 3, 81 Ch 2+3, 76 Ba 4 · **four more papers** give a formula and ask for its CNF **HOT 4/41** 73 Bh 2+2+2+3, 74 Ma 6, 72 Ash 8, 79 Jth 2+2+4, all worked in the companion

Why CNF is required (the 2 marks, and the only part students skip)

- **CNF** = a conjunction of clauses, each clause a disjunction of literals: $(A \vee \neg B) \wedge (C \vee D \vee \neg E)$
- **Resolution is a single inference rule and it only operates on clauses**. Everything else – implications, biconditionals, nested negations, quantifiers – has to go before the machinery can start.
- Because the whole formula is one big **AND**, each clause can be **asserted separately** and stored as its own line in the KB.
- **Every** sentence of propositional or first-order logic has a CNF equivalent, so nothing is lost by insisting on it. That completeness is what makes resolution a *general* proof procedure.
- Contrast **DNF**, a disjunction of conjunctions. Papers sometimes ask for both: CNF is joined by AND, DNF by OR.

The eight steps, in this order

- 1. Eliminate $\leftrightarrow$** : $\alpha \leftrightarrow \beta \Rightarrow (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$.
- 2. Eliminate $\rightarrow$** : $\alpha \rightarrow \beta \Rightarrow \neg \alpha \vee \beta$.
- 3. Move $\neg$ inwards**, until it sits on atoms only: De Morgan, $\neg \neg \alpha \equiv \alpha$, and the quantifier duals $\neg \forall x P \equiv \exists x \neg P$, $\neg \exists x P \equiv \forall x \neg P$.
- 4. Standardise variables apart**: rename so that no two quantifiers bind the same variable name.
- 5. Skolemize**: drop each $\exists$, replacing its variable by a new constant, or by a **function** of the enclosing universal variables (§4.3).
- 6. Drop the universal quantifiers**: every remaining variable is understood to be universally quantified.
- 7. Distribute $\vee$ over $\wedge$** : $\alpha \vee (\beta \wedge \gamma) \Rightarrow (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$.
- 8. Split the conjunction into separate clauses**, and rename variables again so no two clauses share one.

Steps 4, 7 and 8 are the ones the local textbook leaves out, and a proof that skips 4 or 8 unifies two unrelated variables and derives nonsense. Write all eight; the marks are per step.

The worked example the papers use (79 Ch, 75 Bh, 73 Bh)

- “Everyone who loves all animals is loved by someone.”
- **FOPL:**
 $\forall x [\forall y \text{ Animal}(y) \rightarrow \text{Loves}(x, y)] \rightarrow [\exists z \text{ Loves}(z, x)]$
- **2. Eliminate $\rightarrow$, twice:**
 $\forall x \neg[\forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)] \vee [\exists z \text{ Loves}(z, x)]$
- **3. Move $\neg$ in, using the dual and De Morgan:**
 $\forall x [\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists z \text{ Loves}(z, x)]$
- **5. Skolemize.** Both $\exists$ are inside $\forall x$, so both become **functions of x :** $y \Rightarrow F(x)$, $z \Rightarrow G(x)$:
 $\forall x [\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))] \vee \text{Loves}(G(x), x)$
- **6. Drop $\forall$, 7. distribute, 8. split:**
 - $\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)$
 - $\neg \text{Loves}(x, F(x)) \vee \text{Loves}(G(x), x)$
- **Read the Skolem functions back:** $F(x)$ is “the animal x does not love, if there is one” and $G(x)$ is “someone who loves x ”. Saying that in one line shows the step was understood, not copied.

Where marks are actually lost

- **Skolemizing to a constant inside a $\forall$.** $\forall x \exists y$ becomes $F(x)$, never A . This is the single commonest error, and it silently changes the meaning.
- **Distributing where you must not.** $\neg(A \wedge \neg B) \vee C$ is $\neg A \vee B \vee C$ – one clause. It is **not** $(\neg A \vee C) \wedge (B \vee C)$. **The local textbook makes exactly this error** in its “likes all food” problem, then quietly uses the correct single clause in the proof underneath. The companion works that family properly.
- **Forgetting to standardise apart.** Two clauses both using x will unify on a variable they do not share in meaning.
- **Dropping $\forall$ before skolemizing.** The order in the list is not decoration: once the $\forall$ is gone there is nothing to make the Skolem function depend on.
- **Simplification is allowed and is worth doing:** a clause containing both P and $\neg P$ is a tautology and can be dropped, and duplicate literals in one clause collapse.

4.5 Resolution refutation**Prove the given goal from the given premises using resolution refutation** 6 7 8 9 10

TOP 33/41 69 Bh 10, 75 Ba 8, 77 Ch 8, 82 Ka 8, 81 Ash 2+6, 81 Bh 9, 82 Bh 6, 70 Bh 8, 71 Bh 8, 72 Ash 7, 74 Bh 8, 73 Bh 8, 71 Bh 8, 80 Bh 8, 80 Ba 8, 72 Ma 6+4, 72 Ma 7, 73 Bh 8, 78 Po 8, 74 Bh 8, 79 Jth 6, 79 Bh 8, 78 Ba 8, 73 Ma 10, 81 Ba 2+6, 81 Ch 8, 78 Ch 3+3, 80 Ch 3+5, 80 Ash 3+5, 79 Ash 7, 76 Ba 2+8, 76 Bh 7, 77 Ch 8 · **33 of 41 papers, 260 marks** · every one of them is worked in the companion

The procedure. This is the chapter.

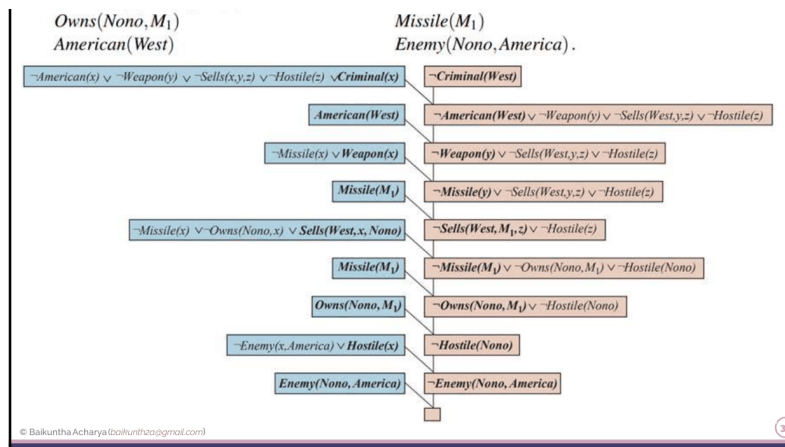
- **1. Convert every premise into FOPL.**
- **2. Convert every FOPL sentence into CNF (§4.4), and number the clauses.**
- **3. Negate the goal,** convert that to CNF too, and add it to the clause set.
- **4. Repeatedly resolve** two clauses containing complementary literals, unifying them with the MGU, and add the **resolvent**.
- **5. Stop when the empty clause $\square$ is derived.** The clause set is then unsatisfiable, so the negated goal is false, so **the goal is true**.

Why refute instead of prove directly

- Resolution is **refutation complete**, not complete: it is guaranteed to find a contradiction in an unsatisfiable set, but it is not guaranteed to derive an arbitrary true sentence.
- So the proof is turned round using $KB \models \alpha$ iff $KB \wedge \neg \alpha$ is unsatisfiable, the equivalence from §4.2.

How to write one that scores

- **Number every clause** and, beside each resolvent, write *which two clauses* it came from and *which substitution* was used. Examiners award those annotations.
- **Resolve towards the goal.** Start from the negated goal clause and work down: it prunes the search and it reads as a proof rather than a search log.
- **One complementary pair per step.** Cancelling two pairs in one line is unsound and is a standard trap.
- **Standardise variables apart** before each step (§4.4, step 8).
- Finish with the sentence “*the empty clause is derived, so the assumption contradicts the premises, hence the goal is proved*”. It is worth a mark and takes one line.



The shape the answer must take: clauses down the left, each resolvent to its right, the empty clause $\square$ at the foot. Premises are the Colonel West set, asked in 7 papers.

33 papers, but only 26 different premise sets, and four families cover 22 of them

- Likes-all-food / peanuts — 7 papers, five wordings of one puzzle (John/Bill/Sue, Ram/Krishna/Radha, Ram/Anil, Sita/mushroom, Bhogendra/chicken). **Learn this one first.**
- Charlie is a horse / a mammal — 6 papers, three wordings. Tests *inverse relations*, and **two of its six premises are never used.**
- Selling weapons to a hostile nation — 7 papers under six names, one asking for forward chaining instead. Tests *existential instantiation*.
- Light sleeper, hounds and mice — 2 papers. The hardest of the four: the goal is itself an implication, so **negating it gives three clauses.**
- Then 10 one-off sets, from Marcus and Caesar to Curiosity and the cat.

Contradiction Vs extraction (the textbook's two methods)

- **Answer by contradiction:** negate the goal, derive $\square$. Use this whenever the paper says *prove that ...*, which is 21 of the 23 sets.
- **Answer by extraction:** do not negate; chain the clauses forward until the goal literal itself is produced. Use it when the paper asks *who* or *what* rather than *prove*, Eg 72 Ma's "did Marcus hate Caesar?"
- **Same clauses, same resolution rule;** only the starting point differs.

The two things that go wrong in an exam

- **Negating a goal that is an implication.** "If John is a light sleeper then John has no mice" negates to "John IS a light sleeper AND John HAS a mouse", which after skolemization is **three** clauses: $LS(John)$, $Have(John, b)$, $Mouse(b)$. Students negate it to one clause and the proof cannot close.
- **"Has some" is existential.** "Nono has some missiles" is $\exists x Missile(x) \wedge Owns(Nono, x)$, which skolemizes to a **new constant** M_1 : two clauses, $Missile(M_1)$ and $Owns(Nono, M_1)$. Writing $Missile(x)$ with a free variable claims *everything* is a missile.
- Both are worked step by step in the companion, which prints each paper's premises exactly as the paper words them.

4.6 Rule-based reasoning: forward and backward chaining

Differentiate between forward and backward chaining with a suitable example 2 3 4 7 8

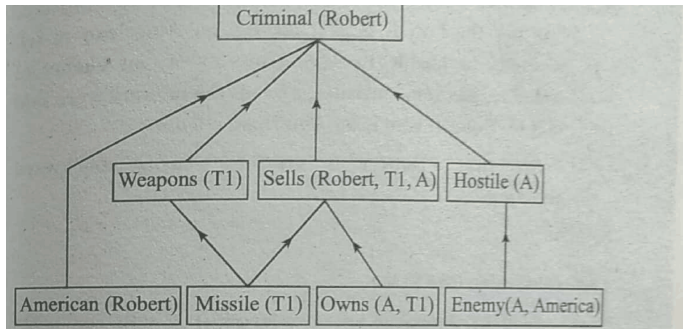
TOP 11/41 70 Ma 4+4, 74 Bh 4, 79 Bh 7, 81 Bh 7, 82 Ka 2+4, 81 Ash 2, 73 Bh 3, 80 Ch 4, 72 Ash 2+5, 78 Po 8, 77 Ch 8 · **always wants an example, not just the table** · in 81 Ash the example is the Colonel West proof, done forward

Forward chaining: data-driven

- Start from the **known facts**. Fire every rule whose premises are all satisfied, add its conclusion to the fact base, repeat until the goal appears or nothing new can be derived.
- **Bottom-up**, and it uses **modus ponens** on definite clauses.
- ✓ Derives **everything** that follows, so it suits monitoring, alarms and planning, where new data arrives and consequences must be found.
- × Does a lot of **irrelevant work**: it proves facts nobody asked for.

Backward chaining: goal-driven

- Start from the **goal**. Find a rule that concludes it, make that rule's premises the new sub-goals, and recurse until every sub-goal is a known fact.
- **Top-down, depth first**, and again modus ponens.
- ✓ Touches **only what the goal needs**, so it suits diagnosis, query answering and theorem proving. **Prolog** works this way.
- × Can loop on recursive rules, and it answers only the question asked.



Forward chaining on the Colonel West premises: facts on the bottom row, one new row per iteration, until *Criminal(Robert)* appears.

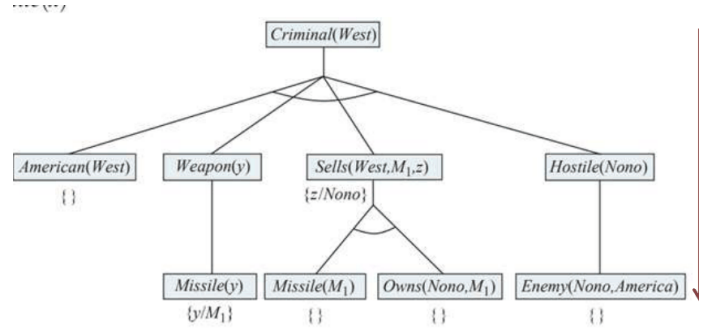


Figure 9.7 Proof tree constructed by backward chaining to prove that West is a criminal. The tree should be read depth first, left to right. To prove *Criminal(West)*, we have to prove the four conjuncts below it. Some of these are in the knowledge base, and others require further backward chaining. Bindings for each successful unification are shown next to the corresponding subgoal. Note that once one subgoal in a conjunction succeeds, its substitution is applied to subsequent subgoals. Thus, by the time FOL-BC-ASK gets to the last conjunct, originally *Hostile(z)*, *z* is already bound to *Nono*.

Backward chaining, same premises: the goal at the top, its four conjuncts below it, each expanded until it meets a fact. The binding that made each unification succeed sits under its node.

	Forward chaining	Backward chaining
Starts from	Known facts / data	The goal to be proved
Direction	Bottom-up, data-driven	Top-down, goal-driven
Question answered	"What can I conclude?"	"Can I prove this?"
Search	Breadth-first in effect: all consequences of the current facts	Depth-first: one sub-goal chain at a time
Work done	Derives many facts, most irrelevant to any one goal	Derives only what the goal needs
Suits	Monitoring, alarms, planning, real-time control, design	Diagnosis, query answering, theorem proving, Prolog
Number of goals	Handles many goals at once	Handles one goal at a time
Efficiency	Slow when the goal is specific	Slow when the fact base is small and rules are deep

- **The one-line discriminator to write:** forward chaining asks *what follows from what I know*, backward chaining asks *what would I need to know for this to be true*. Same rules, same modus ponens, opposite ends.

What is rule-based reasoning? 2 7

HOT 4/41 72 Ma 7, 70 Ma 4+4, 71 Bh 7, 77 Ch 2 · asked as the **2-mark opener** to a backward-chaining question in three of the four

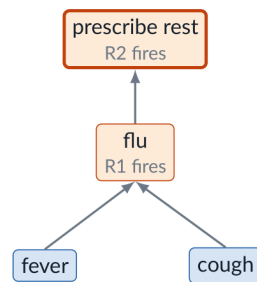
Definition and parts

- Knowledge stored as **IF condition THEN action** rules, and reasoning done by an **inference engine** that decides which rule to fire.
- Three components:
 - **Rule base** – the IF-THEN rules (long-term knowledge);
 - **Working memory** – the facts known right now;
 - **Inference engine** – match, resolve conflicts, fire.
- The engine's cycle is **match** → **conflict resolution** → **execute**, exactly the production system of Ch 2. **Say so:** the examiner set that question too.

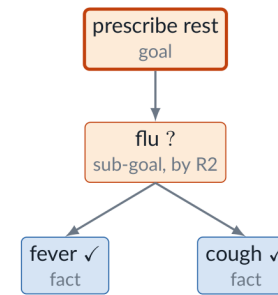
Why it is used, and the worked shape

- ✓ Rules are **modular**: add or remove one without touching the rest. ✓ They are readable by a domain expert who is not a programmer. ✓ The chain of fired rules is the explanation, which is how an expert system justifies itself (Ch 7).
 - × No learning, and × large rule bases become slow and can conflict.
 - **The example to give**, since every paper asks for one:
 - R1: IF *fever* AND *cough* THEN *flu*. R2: IF *flu* THEN *prescribe rest*.
 - Facts: *fever, cough*. Forward: R1 fires, *flu* added, R2 fires, *prescribe rest*. Backward from *prescribe rest*: needs *flu*, which needs *fever* and *cough*, both known.
- Two directions, one rule base.**

FORWARD: from the facts up



BACKWARD: from the goal down



4.7 Statistical reasoning: uncertainty, Bayes and belief networks

How is reasoning done under uncertainty? What are the approaches for measuring it? 2 3 3+6

80 Ash 3, 81 Ash 2, 79 Ash 3+6 · always the opener to a Bayes numerical, so keep it to half a page and spend the time on the calculation

Why logic is not enough

- FOPL is **monotonic and categorical**: a fact is true or false, and adding knowledge never retracts a conclusion. The real world is neither.
- Three sources of uncertainty, and papers want them named:
 - **incomplete** knowledge (not all facts are available);
 - **imprecise / unreliable** data (a sensor, a symptom);
 - **rules that are only usually true** (*a stiff neck suggests meningitis*).
- Writing *stiff neck* → *meningitis* in logic is simply false, and listing every exception is impossible. **So attach a number to the belief instead.**

The approaches for measuring uncertainty

- **Probability theory** – degree of belief in $[0, 1]$, updated by **Bayes' rule**. The mainstream answer, and the only one the numerals use.
- **Certainty factors** – an ad-hoc $[-1, +1]$ score attached to each rule, as used in MYCIN. Simple, but not probabilistically sound.
- **Dempster-Shafer theory** – assigns belief to *sets* of hypotheses, so it can represent *ignorance* separately from *disbelief*.
- **Fuzzy logic** – handles *vagueness* ("tall", "warm"), which is a different thing from uncertainty. Covered in Ch 6.
- **Non-monotonic / default reasoning** – assume the normal case, retract the conclusion when contradicted.
- **The distinction worth one mark**: probability is about *not knowing which case holds*; fuzzy logic is about *the case not having a sharp boundary*.

What is prior and posterior probability? Explain Bayes' theorem and its role in reasoning 2 3 2+6 3+5

TOP 15/41 71 Ma 3+5, 73 Bh 7, 73 Ma 8, 73 Bh 8, 72 Ash 8, 78 Ch 3+5, 80 Ch 3+5, 80 Bh 2+6, 80 Ba 5+3, 81 Ba 2+6, 80 Ash 3+5, 81 Ash 2+6, 78 Ba 2+6, 82 Ba 2+6, 82 Bh 7 · **15 papers carry a Bayes calculation**, every one of them worked in the companion · the theory part is never more than 3 marks

The vocabulary

- **Prior** $P(A)$ – belief in A **before** any evidence. Eg 1 person in 50,000 has meningitis.
- **Posterior** $P(A|B)$ – belief in A **after** evidence B is observed. Eg the chance of meningitis *given* a stiff neck.
- **Likelihood** $P(B|A)$ – how probable the evidence is if the hypothesis holds. **This is the number a doctor actually knows**, and it is the reason the theorem is needed.
- **Marginal** $P(B)$ – the probability of the evidence however it arose.

The theorem

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- When $P(B)$ is not given, expand it by the **law of total probability**, which is what most papers require:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B|A)P(A) + P(B|\neg A)P(\neg A)}$$

hypothesis is true, then we calculate the probability of evidence.

LIKELIHOOD
The probability of "B" being True, given "A" is True

probability of hypothesis before considering the evidence

PRIOR
The probability "A" being True. This is the knowledge.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

POSTERIOR
The probability of "A" being True, given "B" is True

MARGINALIZATION
The probability "B" being True.

Probability of hypothesis A when we have occurred an evidence B.

pure probability of an evidence

$$P(\text{cause} | \text{effect}) = \frac{P(\text{effect} | \text{cause})P(\text{cause})}{P(\text{effect})}$$

The doctor knows $P(\text{symptoms} | \text{disease})$ and wants to derive a diagnosis, $P(\text{disease} | \text{symptoms})$

$$P(A|B) = \frac{P(A) \cdot P(B|A)}{[P(A) \cdot P(B|A)] + [P(\neg A) \cdot P(B|\neg A)]}$$

How it aids inference (the marks in the theory half)

- It **reverses the direction of a conditional**. Causal knowledge runs cause $\rightarrow$ effect ($P(\text{symptom} | \text{disease})$, stable and measurable); diagnosis needs effect $\rightarrow$ cause. Bayes converts one into the other.
- It gives a **principled update rule**: posterior $\propto$ likelihood $\times$ prior. Evidence does not replace belief, it revises it.
- It makes the **base rate** explicit. A 99 %-accurate test for a 1-in-10,000 disease still leaves you far more likely to be healthy than ill, because the prior is tiny. **This is asked directly** (72 Ash) and it is the single most useful sentence in the section.
- It composes: a chain of evidence can be applied one piece at a time, which is exactly what a belief network does below.

Getting the arithmetic right

- Name the events **before** substituting. Half the lost marks are a $P(A|B)$ written where $P(B|A)$ was meant.
- If the question gives percentages of a population that split into groups (*factories, men and women*), the denominator is **always the total-probability sum** over those groups.
- $P(\neg A) = 1 - P(A)$ is almost always needed and almost never given.

What is a causal / belief network? How is reasoning done in it? 2+5 2+6 6 7

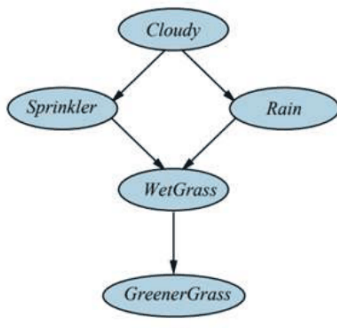
HOT 7/41 71 Ma 2+5, 79 Ch 2+6, 79 Ash 3+6, 77 Ch 6, 73 Bh 7, 80 Ba 5+3, 82 Bh 7 · causal net, belief network, Bayesian network and Bayesian belief network are **four names for one thing** in these papers · two papers print a network and ask for a joint probability (78 Ba, 82 Bh)

Definition and the two parts

- A **directed acyclic graph** in which
 - each **node** is a random variable, and
 - each **edge** $X \rightarrow Y$ says X has a **direct causal influence** on Y ; plus a **conditional probability table (CPT)** at every node giving $P(\text{node} \mid \text{its parents})$.
- A root node has no parents, so its CPT is just its prior.
- **The whole network in one equation**, and this is what the numerals use:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i \mid \text{parents}(x_i))$$

- **Why it is worth having:** a full joint distribution over n binary variables needs $2^n - 1$ numbers. The alarm network below has 5 variables and needs **only 10**, because each node depends on its parents alone. That compactness is the answer to *when do we use a Bayesian network*.
- **Conditional independence** is the property being exploited: a node is independent of its non-descendants given its parents.



A causal net: Cloudy influences Sprinkler and Rain, both influence WetGrass. The shape 78 Ba prints.

Reasoning in the network (the 5 or 6 marks)

- **Causal / top-down** – from cause to effect. *Burglary happened; how likely is John to call?* Uses the CPTs directly.
- **Diagnostic / bottom-up** – from effect to cause. *John called; how likely is a burglary?* This is where **Bayes' theorem enters**, and it is the answer to “how is Bayes' theorem used in a belief network” (77 Ch).
- **Intercausal** – between two causes of one observed effect. *The alarm rang and an earthquake is reported, so burglary becomes less likely: explaining away.* Worth naming; almost nobody does.
- **Mixed** – evidence above and below the query node at once.
- **The method for a numerical:** write the product form, substitute one CPT entry per variable, and read $P(\neg X) = 1 - P(X)$ off the same table.

✓ **Variables:** Burglar, Earthquake, Alarm, JohnCalls, MaryCalls

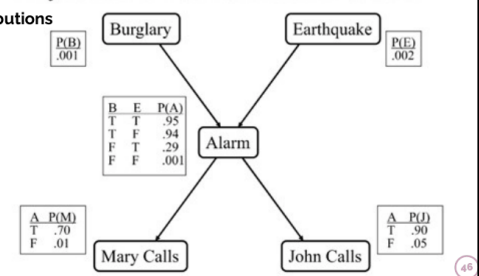
Bayesian Network for Alarm Domain

Retrieving Probabilities from the Conditional Distributions

- $P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i \mid \text{Parents}(x_i))$
- E.g.

$$\begin{aligned}
 &P(J \ \& \ M \ \& \ A \ \& \ \neg B \ \& \ \neg E) \\
 &= P(J|A)P(M|A)P(A|\neg B, \neg E)P(\neg B)P(\neg E) \\
 &= 0.9 \times 0.7 \times 0.001 \times 0.999 \times 0.998 \\
 &= 0.00062
 \end{aligned}$$

© Baikuntha Acharya (baikunthaa@gmail.com)



The standard alarm network, with a CPT at every node. *Burglary* and *Earthquake* both cause *Alarm*; the alarm makes *John* and *Mary* call. Five variables, and the graph plus these five tables hold the whole joint distribution in **10 numbers** instead of $2^5 - 1 = 31$. The worked line beside it is the product form applied straight down the network, which is exactly what 78 Ba and 82 Bh ask for.

- **The connection the papers ask for in one sentence:** a belief network stores *causal* conditional probabilities, and Bayes' theorem is what lets you run them *backwards* to a diagnosis. Without the network you would need the full joint distribution; without Bayes you could only reason in the direction the arrows point.

5. Structured Knowledge Representation

4 hrs · asked in **38 of 41 papers** · 5 syllabus sub-topics · 319 marks, 9.4% of every mark ever set · the 11 sentence sets and the net-to-frame conversion are worked in *Ch5 – Structured Knowledge Representation – Solved Problems*.

Read this first:

- **The chapter is two objects and one comparison.** A **semantic net**, a **frame**, and **net Vs frame**. Learn those three and you have answered nine tenths of what this chapter has ever set.
- **The most-asked question is a drawing:** “convert the given sentences into a semantic network”, in **12 papers** across **11 sentence sets**, for 5 to 7 marks each, and **six of the eleven are the same three figures with the names changed**. Answer with a drawing, never with prose. All worked in the companion.
- **Second-most-asked, and the safest marks here:** “explain semantic net and frame with examples, list their advantages and limitations” **TOP 13/41**. One comparison table plus two small drawings.
- Three reliable short answers, each a fixed list: **issues in KR** **HOT 7/41**, **approaches to KR** **HOT 6/41**, how a KR is evaluated (four adequacies).
- **CD and scripts are only 2 papers, but both times they carry the whole 8 marks** (71 Bh **2+5**, 77 Ch **3+5**).
- Pairs the examiner reliably asks together: **issues in KR + draw the semantic network**, **approaches + semantic nets with an example**, and **compare net and frame + convert one into the other**.

5.1 Knowledge, representation, and what makes a representation “structured”

What is knowledge representation? What do you understand by structured knowledge representation? **2 2+5**

75 Bh **2**, 76 Bh **2+5** · 2 papers at 2 marks each, so no tier chip · **always the opener** to a semantic-net question, so answer it in three lines and move on

Knowledge representation

- Encoding what an agent knows about the world in a form a computer can manipulate, chosen so that **inference is easy**.
- Two things exist and must not be confused:
 - **Facts** – truths in the world. The **knowledge level**.
 - **Representations** of those facts in some formalism. The **symbol level**.
- KR is the **mapping** between the two, and it must run *both ways*: *Tommy is a dog* $\rightarrow$ `dog(Tommy)`; add $\forall x : \text{dog}(x) \rightarrow \text{hastail}(x)$, deduce `hastail(Tommy)`; map back to *Tommy has a tail*.
- **The backward map is where representations fail.** A formalism that cannot say its answer back in the user’s terms is useless in an expert system.
- Three **levels** a fact sits at: **knowledge** (what is true), **logical** (the sentence encoding it), **implementation** (the data structure holding the sentence).
- The **kinds of knowledge** to name if asked: objects, events, performance (how to do a task), meta-knowledge, facts.

Structured knowledge representation

- In logic, everything known about one object is **scattered across many separate clauses**. Nothing in the formalism ties them together.
- **Structured KR concentrates all of an object’s knowledge in one place:** one node of a graph, or one record of slots. The structure itself carries meaning.
- So the definition to write: **a representation in which knowledge is stored in named, linked units rather than in flat sentences**, each unit holding an object and everything known about it.
- The four structured schemes on this syllabus: **semantic nets, frames, conceptual dependency, scripts**.
- Rich and Knight call them **slot-and-filler** structures, *weak* where the slots carry no fixed meaning (nets, frames) and *strong* where they do (CD, scripts).
- **What it buys you:** inheritance, default values, and the ability to answer “what do you know about *X*” by looking in *one place*.

5.2 Approaches, models and schemes of knowledge representation

What are the different approaches / models / schemes of knowledge representation? How do you choose among them? 2 4 7 8

HOT 6/41 69 Bh 7, 80 Ba 2+5, 73 Bh 8, 79 Ash 4, 81 Bh 2, 80 Ch 2+2 · 73 Bh says “four types of KR scheme”, which is the **four-scheme list** below, not the five-method one · 80 Ch additionally asks *how you make a choice*

The four schemes (answer 73 Bh with these)

- **1. Logical** – knowledge as formal sentences, manipulated by rules of inference.
 - Eg propositional logic, first-order predicate logic (Ch 4).
 - ✓ Precise, sound, complete. × Rigid, and a single object’s facts end up scattered.
- **2. Procedural** – knowledge as procedures that *do* something.
 - Eg IF-THEN production rules, an expert system’s rule base.
 - ✓ Directly executable, control built in. × Knowledge and the code using it are inseparable, so nothing can be modified independently.
- **3. Network** – knowledge as a labelled graph.
 - Eg **semantic nets**, conceptual dependency, conceptual graphs.
 - ✓ Visual, inheritance is free. × No quantifiers, no negation.
- **4. Structured** – a network whose nodes are themselves *complex data structures*.
 - Eg **frames, scripts**.
 - ✓ Defaults, procedures and constraints per slot. × No inference engine of its own.
- **One sentence that ties them together:** 3 and 4 are the *structured* half of this chapter; 1 and 2 are Ch 4 and the expert system of Ch 7.

How to choose 80 Ch 2

- Score the candidates on the **four adequacies** of §5.3, against the domain you actually have.
- Then three practical questions:
 - Is the knowledge mostly **taxonomic**? Use a net or frames, and get inheritance for nothing.
 - Does it need **proof**, quantifiers or negation? Use logic – a net cannot express “*no bird is a mammal*”.
 - Does it need **defaults and exceptions**? Use frames; logic has no clean way to say “*birds fly, except penguins*”.
- **The examiner’s expected closing line:** no single scheme wins. Real systems are **hybrid** – frames for the objects, rules for the reasoning.

The five methods (the wording BJ Sir and PS Sir use)

- **Procedural** – facts plus the step-by-step operations on them, hard-coded.
- **Declarative** – states, facts, rules and relations declared *separately* from the algorithm that reasons over them.
 - This separation is the whole advantage: the knowledge can be changed without rewriting the reasoner, and one optimised reasoner serves many knowledge bases.
- **Relational** – facts as rows of a table, exactly as a database holds them. Simple, but carries no inference at all.
- **Hierarchical** – inheritable knowledge as a tree of classes, each level inheriting the level above.
- **Complex network graph** – the general labelled graph: semantic nets.

5.3 Evaluating a knowledge representation, and the issues in KR

How is a knowledge representation evaluated? What are the issues in knowledge representation? 2

3 4

HOT 7/41 80 Bh 3, 80 Ch 2+2, 79 Ash 2+2, 80 Ash 3, 82 Bh 2, 76 Bh 2, 79 Bh 2 · two different lists, and **papers mix them up**: *properties* is the four adequacies, *issues* is the five design questions · 76 Bh asks the issues inside an expert-system question

Evaluating a KR: the four adequacies

- **1. Representational adequacy** – can it represent *all* the kinds of knowledge the domain needs?
- **2. Inferential adequacy** – can its structures be manipulated to *derive new* structures, ie new knowledge from old?
- **3. Inferential efficiency** – can it carry extra guidance that **focuses** the inference engine in productive directions, rather than letting it search blindly?
- **4. Acquisitional efficiency** – can new knowledge be added **easily**, ideally automatically rather than by a human?
- Two practical criteria worth adding for the last mark: it must be **easy to use**, and **easy to modify and extend**.
- **The honest closing line**: no known representation scores well on all four at once. That is exactly why hybrid systems exist, and it is what 80 Ch's "*is it possible to evaluate a KR*" is fishing for – yes, on these four axes, but only relative to a domain.

The five issues in knowledge representation

- **1. Important attributes** – are there attributes so basic that they occur in almost every domain, and so deserve special handling?
 - **Yes: isa and instance**. Both carry **property inheritance**, and every scheme in this chapter is built on them. This is the expected answer, not a shrug.
- **2. Relationships among attributes** – four to name:
 - **inverses** (*owns / owned-by*), **existence in an isa hierarchy** (*height* generalises *batting-height*), **techniques for reasoning about values** (types, ranges, rules), **single-valued attributes** (a person has one date of birth).
- **3. Level of representation, ie granularity** – how fine should the primitives be?
 - Coarse: few facts, but similar sentences look different. Fine: identical meanings collapse together, but the store explodes. CD (§5.7) is the extreme fine-grained answer.
- **4. Representing sets of objects** – by **listing members**, or by **stating the defining property**? Only the second can say "*there are lots of sheep*", which is a property of the set and of no member.
- **5. Finding the right structure** – given a huge store, how do you **select** the relevant structure and **revise** it when it turns out wrong? Indexing, matching and frame selection all live here.
- Two more the decks add: **interpretation is informal** in a net, and **complex relationships are hard** to draw with binary links.

5.4 Semantic nets

What is a semantic net? Explain with an example. When do we use one? 2 5 6 7 8

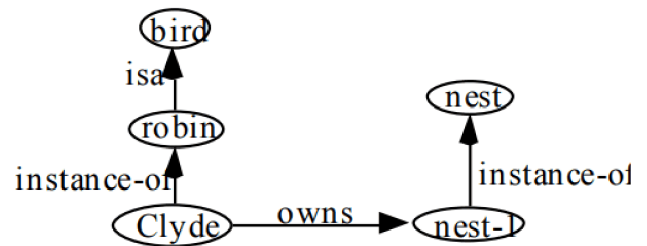
TOP 9/41 70 Bh 8, 72 Ma 7, 82 Ba 2, 75 Bh 6, 71 Bh 7, 69 Bh 7, 80 Ba 5, 76 Bh 5, 80 Bh 5 · a further 12 papers ask you to draw one, all worked in the companion · 72 Ma asks for a net about the AI course itself

What it is

- A **directed labelled graph** that stores knowledge as **nodes and arcs**.
 - **Nodes** = objects, concepts, situations or events.
 - **Arcs** = the relationship between two nodes, and the arc is labelled with that relationship.
- A **declarative graphic representation**: it states what is true and supports automated reasoning over it, but contains no procedure.
- Proposed as **an alternative to predicate logic**. Same facts, but all of one object's facts meet at one node instead of being scattered over clauses.
- **Oldest KR in the subject**: the Tree of Porphyry, 3rd century AD. Worth a line in an 8-mark answer.

The two links every paper wants named

- **isa / a-kind-of** — **class to class**. A cat is a mammal. Set inclusion: $Cats \subset Mammals$.
- **instance / member** — **individual to class**. Tom is a cat. Set membership: $Tom \in Cats$.
- **Getting these two the wrong way round is the commonest lost mark** in the drawing questions. Sakti Gauchan is an instance of Person; Person is isa Mammal.
- Everything else is an ordinary relation arc: has-part, owns, likes, team, uniform-color.



isa runs class to class, instance-of runs individual to class, and owns is an ordinary relation. Source: BA Sir CH-05.

The canonical net, and its FOPL

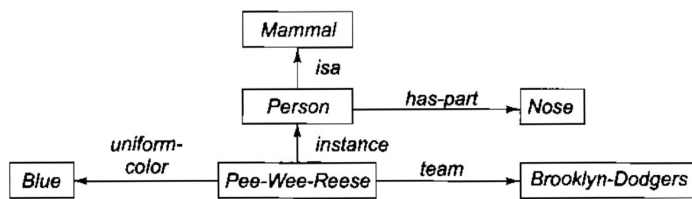


Fig. A Semantic Network

isa(Person, Mammal)
 instance(Pee-Wee-Reese, Person)
 team (Pee-Wee-Reese, Brooklyn-Dodgers)
 uniform-color(Pee-Wee-Reese, Blue)

Rich and Knight's figure. Every "convert these sentences" question in the papers is this net with the names changed: a class chain at the top, an individual hung off it by instance, and its own properties fanning out.
 Source: PS Sir ch5.

Why a net can do things logic cannot

- **Inheritance is free**. A property put on Person is automatically available to Pee-Wee-Reese by following instance then isa upward. No inference rule is needed; the arcs are the rule.
- **Intersection search**. Spread activation outward from two nodes and see where the two wavefronts meet. That is how a net answers "what connects the Brooklyn Dodgers and blue?" — a question logic can only answer by proving a conjecture you already had to guess.
- **This is the advantage to quote**, because it exploits the entity-based organisation that slot-and-filler structures have and logic does not.

Advantages and limitations of a semantic net; how do you represent a three-place relation? 2 4

part of the *advantages and limitations* question in 73 Ma, 76 Ba, 78 Ch, 78 Ba, 81 Bh · chip sits on \$5.6, where the comparison is

Advantages

- **Natural and transparent:** close to how people link ideas, so it is readable by a non-programmer.
- **Easy to visualise,** and easy to check by eye.
- **Inheritance** of properties comes for free down *isa* chains.
- **Space-efficient:** each object appears *once*, however many facts mention it.
- Related facts sit next to each other, so **relevant knowledge is found by following links** instead of by searching the whole store.

Limitations

- **Cannot express quantification, negation or disjunction.** There is no way to draw “every mammal has some parent”, “no bird is a mammal” or “Tom is black or white”. This is the limitation to write first.
- **No standard meaning for link names.** *has-part* means whatever the person who drew it meant, so two nets are not comparable.
- **No inference mechanism of its own:** it stores, it does not deduce.
- **Slow at query time** – answering may need a traversal of the whole graph.
- **Does not scale:** a net that really modelled human memory cannot be built by hand.
- Interpretation is **informal**, so the net depends entirely on its creator.

Three-place relations: use an event node

- Arcs are **binary**, so “John gave the book to Mary” cannot be one arc: it has three arguments.
- **The fix:** create a **new node for the event itself**, then hang one binary arc on it per argument.

John gave the book to Mary.

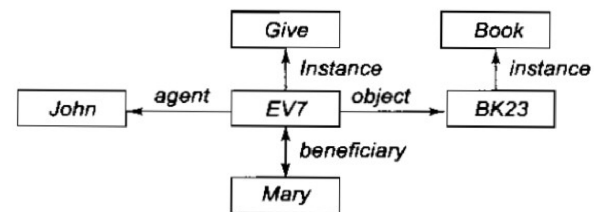


Fig. A Semantic Net Representing a Sentence

EV7 is the giving-event node. *agent* → John, *object* → the book, *beneficiary* → Mary, and *instance* says EV7 is a Give. Source: PS Sir ch5.

- Same trick for a baseball game with two teams and a score: *isa*(G23, Game), *home_team*(G23, Dodgers), *visiting_team*(G23, Cubs), *score*(G23, 5-3).
- **Say why it works:** it converts an *n*-place predicate into *n* binary ones by *reifying* the relation as an object.

What do the arcs of a semantic net actually mean? 2

not asked on its own · included because it is the **one line that separates a 5-mark answer from a 7-mark one** on every “explain semantic nets” question

Link Type	Semantics	Example
$A \xrightarrow{\text{Subset}} B$	$A \subset B$	Cats $\subset$ Mammals
$A \xrightarrow{\text{Member}} B$	$A \in B$	Felix $\in$ Cats
$A \xrightarrow{R} B$	$R(A,B)$	Felix $\xrightarrow{\text{Friend}}$ Opus
$A \xrightarrow{\boxed{R}} B$	$\forall x \ x \in A \Rightarrow R(x,B)$	Birds $\xrightarrow{\boxed{\text{Legs}}}$ 2
$A \xrightarrow{\boxed{R}} B$	$\forall x \ x \in A \Rightarrow \exists y \ y \in B \wedge R(x,y)$	Bird $\xrightarrow{\boxed{\text{Parent}}}$ Birds

- Subset arc = **set inclusion**, $A \subset B$. Eg *Cats* $\subset$ *Mammals*.
- Member arc = **set membership**, $A \in B$. Eg *Felix* $\in$ *Cats*.
- A named arc between two *individuals* = the ground fact $R(A, B)$. Eg *Friend*(Felix, Opus).
- A named arc drawn out of a *class* box means $\forall x (x \in A \Rightarrow R(x, B))$: **every** member has that value. Eg *Legs* out of *Birds* → 2.
- Same arc with an existential reading: $\forall x (x \in A \Rightarrow \exists y (y \in B \wedge R(x, y)))$. Eg *Parent* out of *Bird* → *Birds*: every bird has *some* bird parent.
- **Why it matters:** the last two are the only quantification a net can express, and only because the reader agrees to read the picture that way. Nothing in the drawing says which of the two is meant. That ambiguity is the “lack of formality” limitation.
- Source: BJ Sir [AI_Chapter5_new](#).

5.5 Frames

What is a frame? Explain with a suitable example. What is its significance in AI? 1 5 7 8

HOT 5/41 69 Po 7, 74 Bh 7, 71 Bh 8, 82 Ka 1, 80 Ash 5 · 82 Ka wants **one sentence** for 1 mark · 80 Ash asks for the *importance* of frames, so end on the advantages

Definition

- A frame is a **collection of attributes and their values that describes one entity**, together with that entity's relationship to other frames.
 - **Slot** = the attribute name. **Filler** = its value.
 - So a frame is a **record structure with meaning attached**.
- **A single frame is almost useless**. The power is in a **frame system**: many frames, connected because **the value of one frame's slot is another frame**.
- **The line that earns the mark**: a semantic net is a **two-dimensional** representation, nodes and arcs; **frames add a third dimension by letting a node have internal structure**.
- A frame is designed to represent a **stereotypical** object or situation: what is normally true, with defaults for what has not been observed.
- Frames are **static**. A script (\$5.7) is the same idea made *sequential*.

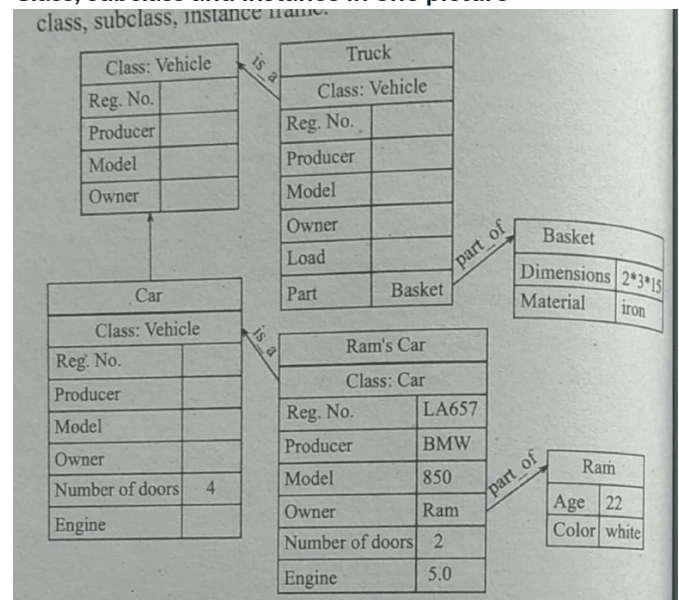
Types of frame

- **Class frame** – the generic frame others inherit from. Eg *Vehicle*.
- **Subclass frame** – inherits a class frame and adds slots. Eg *Car*.
- **Instance frame** – one individual. Eg *Ram's Car*.
- An instance or subclass frame **inherits every slot above it** and may add its own. Generic frames sit at the top of the hierarchy.

The four parts of a frame

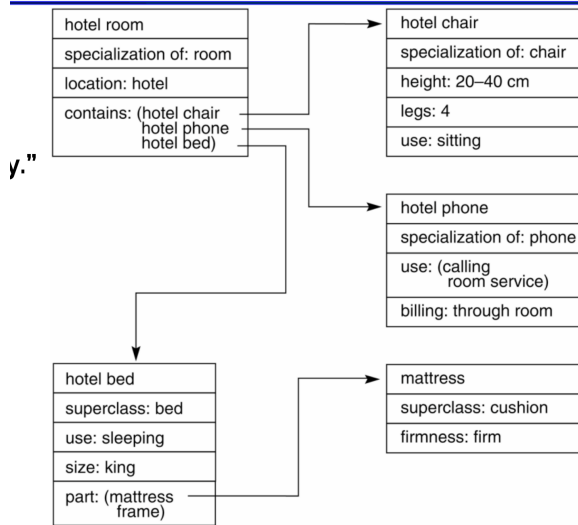
- **1. Frame identification name** – what entity this frame is about.
- **2. Relationship to other frames** – *isa*, *instance*, *part-of*, *superclass*.
- **3. Slot and filler** – the attribute–value pairs themselves.
- **4. Default information** – values taken as true *until contradicted*. **Logic has no clean way to do this**; frames do, and it is the strongest single argument for them.

Class, subclass and instance in one picture



Vehicle (class) → Truck and Car (subclass, *is-a*) → Ram's Car (instance), with *part_of* links out to Basket and Ram. Note the instance frame fills in the slots it inherited empty. Source: *Insights* book p136.

Part of a frame description of a hotel room



Part of a frame description of a hotel room. Source: BA Sir CH-05.

What can a slot hold, and what are demons? 2 4

inside the frame question in 71 Bh 8 and 69 Po 7 · a slot holds more than a value, and saying so is the difference between a 5-mark frame answer and an 8-mark one

Example Frame: STUDENT

Slot	Value (Current Filler)	Default (Default Filler)	Cardinality (Min..Max)	Type (Type Restriction)	Procedure (Attached Procedure)	Salience (Importance)	Constraint (Rule / Axiom)
Name	Ram	–	1 .. 1	String	–	High	Cannot be empty
Age	20	18	1 .. 1	Integer	If-added: Verify age	Medium	Age ≥ 16
Department	Computer Engineering	Computer Engineering	1 .. 1	Department	–	High	Must be a valid department
Phone Number	9841234567	–	1 .. 3	String	If-added: Validate format	Low	Must contain 10 digits
CGPA	3.75	0.0	1 .. 1	Float	If-needed: Calculate from marks	High	0 ≤ CGPA ≤ 4.0

• Value: current value stored in the slot.
 • Default: value used if no value is provided.
 • Cardinality: minimum and maximum number of fillers.
 • Type: type restriction on the fillers.
 • Procedure: attached procedures (if-needed, if-added, if-removed).
 • Salience: measure of importance.
 • Constraint: rule that the value must satisfy.

One STUDENT frame with every facet filled in. Source: PS Sir ch5.

Slots	Fillers
name	property
specialization_of	a_kind_of object
types	(car, boat, house) if-added: Procedure ADD_PROPERTY
owner	default: government if-needed: Procedure FIND_OWNER
location	(home, work, mobile)
status	(missing, poor, good)
under_warranty	(yes, no)

A generic property frame carrying two demons:

if-added: ADD_PROPERTY on types,
if-needed: FIND_OWNER on owner, which also shows a default of government. Source: PS Sir ch5.

A frame system: four frames pointing at each other

- hotel room carries three ordinary slots (*specialization of*, *location*, *contains*), and the third one's filler is **three pointers, not three strings**. That is what makes this a frame system.
- Each frame it points at repeats the pattern. *hotel chair* has *specialization of: chair*, so it inherits everything a chair has and **overrides** only *height*, *legs* and *use*.
- *hotel bed* adds a *part* slot filled by the *mattress* frame, which is **part-of**, not **is-a**. **Confusing those two is the commonest error in a frame answer**: *is-a* inherits, *part-of* does not.
- **The line to write under your own drawing**: the diagram is a graph whose nodes happen to be records. That is structured knowledge representation in one picture, and it is why 80 Bh can ask for a net and a frame in the same breath.

Facets: what a slot carries besides its value

- **Value** – the current filler.
- **Default** – the filler used when nothing has been observed.
- **Cardinality** – minimum and maximum number of fillers allowed.
- **Type** – a restriction on what a filler may be, usually another frame.
- **Procedures** – attached code, see demons below.
- **Salience** – how important this slot is to the frame.
- **Constraints** – axioms the value must satisfy, eg $0 \leq CGPA \leq 4.0$.

Demons (attached procedures, Insights §5.4.3)

- A demon is a **procedure stored in a slot and fired automatically when the slot is touched**. The frame does not just hold data, it reacts.
- **IF-NEEDED** – fires when a value is required and none is stored. *Compute it now*.
- **IF-ADDED** – fires when a value is put into the slot. *Propagate the consequences*.
- **IF-REMOVED** – fires when a value is deleted.
- **IF-CHANGED** – fires when a stored value is replaced.
- **Why this matters for the exam**: demons are what make a frame system *active*, and they are the honest answer to “frames have no inference mechanism” – they have no *general* one, but each slot can carry its own.
- **Frames Vs objects**: slots are data members, demons are methods, *isa* is inheritance and *part-of* is containment. **A frame system is object-oriented programming, invented for AI a decade earlier**. One sentence, and it is often worth a mark.

Advantages and limitations of frames 2 4

part of the *advantages and limitations* question, chip on \$5.6 · **write these as a pair with the semantic net list**, because every paper that asks for one asks for the other

Advantages

- **Groups related knowledge together**, so programming and retrieval are easier: everything about an object is in one record.
- **Defaults**, and therefore the ability to reason with incomplete information and to detect missing values.
- **Easy to extend**: adding a slot, a property or a relation costs one line.
- **Inheritance** down the class hierarchy, exactly as in a semantic net.
- **Procedural attachment**: a slot can compute or validate itself.
- **Flexible and widely used** – vision and natural language processing both run on frames.
- Good at **causal knowledge**, because the information is organised by cause and effect.

Limitations

- **No inference mechanism of its own**. Frames store and inherit; they do not deduce. Any reasoning has to be bolted on.
- **Modification is dangerous**: change one frame and every frame that inherits from it is affected.
- **Redundancy** – the same fact easily ends up in several frames.
- **No standard structure**: the frame for a room and the frame for a person look nothing alike, so nothing generic can be written over them.
- **Defaults can be wrong**, and nothing in the formalism flags it when they are.
- **The examiner's expected trade-off line**: frames win on organisation and defaults, logic wins on proof, nets win on visibility.

5.6 Semantic nets Vs frames, and converting one into the other

Explain semantic nets and frames with examples. Compare them, and list their advantages and limitations 2 4+4 6+2 7

TOP 13/41 72 Ash 7, 72 Ma 7, 73 Ma 4+4, 74 Ma 2+5, 74 Bh 7, 74 Bh 4+3, 75 Ba 2+6, 76 Ba 6+2, 78 Ba 6+2, 78 Ch 4+4, 79 Jth 5, 81 Ba 3+5, 81 Bh 2 · **the second-most-asked question in the chapter** · 72 Ash and 78 Ba ask the **"why are they important in AI"** follow-up, so keep two lines for it

	Semantic net	Frame
Shape	labelled directed graph	collection of records , slots and fillers
Dimension	two : node and arc	three : node, and structure <i>inside</i> the node
Unit of knowledge	one arc = one fact	one frame = one whole object
Node holds	a name only	slots, values, defaults, types, constraints, procedures
Relations	drawn as arcs	written as slot values that are other frames
Inheritance	✓ down <i>isa</i> / <i>instance</i> arcs	✓ down class → subclass → instance
Defaults	× cannot express	✓ first-class, with <i>if-needed</i> to compute one
Procedures	× none	✓ demons on any slot
Best at	seeing the relationships at a glance	holding everything about one object in one place
Weakest at	complex or structured facts about a single node	showing the shape of the knowledge as a whole
Reasoning	intersection search, link following	inheritance and slot lookup only
Both fail at	quantification, negation, disjunction, and having no inference engine of their own	

How frames are useful in semantic nets 72 Ash 7

- The question sounds like two topics. It is one: a **frame** is what you get when you let a semantic-net node have contents.
- A net node called *Tom* can only be named. Make it a frame and it can carry *colour: ginger*, *age: default 2*, *owner: John*, plus a demon that computes age from a birth date.
- So frames **fix the net's weakest point**: they let one node hold structured, defaulted, constrained knowledge instead of forcing every detail out onto its own arc.
- And the net **fixes the frame system's weakest point**: it shows the shape of the whole knowledge base, which a pile of records does not.
- **Answer in one sentence**: frames give a semantic net depth at each node, and the net gives a frame system its map.

Why both matter in AI 78 Ba 2

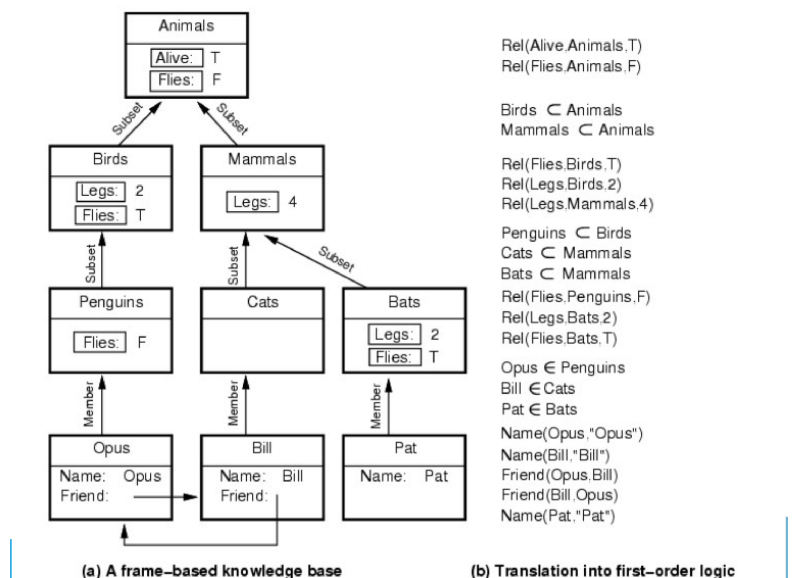
- They give **inheritance and defaults**, which is how an agent reasons with incomplete knowledge.
- They are **human-readable**, so a domain expert can check the knowledge base without knowing logic – the bottleneck in every expert system (Ch 7).
- They organise knowledge by **object** rather than by *sentence*, so retrieval is local instead of a search of the whole store.

The same knowledge three ways

- **Frame, net and FOPL are interchangeable** for taxonomic knowledge, and 75 Ba 6 asks for exactly that: *examples of both, with FOPL statements*.
- Read the figure left to right: a frame-based knowledge base, then the first-order sentences that say the same thing.
- `Subset(Birds, Animals)` is the *isa* arc. `Member(Opus, Penguins)` is the *instance* arc. `Rel(Legs, Birds, 2)` is a slot with a value.
- **Write the FOPL line beside every arc you draw** in that question. It is 2 free marks and most answers skip it.

Converting a semantic net into a frame 81 Ba 5

- **Step 1.** Every node becomes a **frame**, named after the node.
- **Step 2.** Every **outgoing arc** of that node becomes a **slot** in its frame. **The arc label is the slot name, the arc's target is the filler.**
- **Step 3.** *isa* and *instance* arcs become the frame's **class link**, not ordinary slots: *isa* → a subclass frame, *instance* → an instance frame.
- **Step 4.** Any slot whose filler is a node that has its own frame is **left as a pointer to that frame**, which is what makes the result a frame system.
- **Step 5.** Push each property to the **highest** class that has it, and let everything below inherit. A property drawn on three individuals in the net belongs on their class in the frame system.
- **Check:** no information may be lost. Count the arcs in the net; count the filled slots in the frames. They must match.
- 81 Ba is worked end to end in the companion, on this net.



(a) a frame-based knowledge base and (b) its translation into first-order logic, the same nine facts twice. Source: PS Sir ch5.

5.7 Conceptual dependency and scripts

What is conceptual dependency? Explain the common primitives used in it 2+5 3

PIN 2/41 71 Bh 2+5, 77 Ch 3+5 · only 2 papers, but **both times CD and scripts together carry the whole 8-mark question** · 77 Ch pairs it with "how is knowledge represented using scripts"

What it is

- Schank, 1973–75, built for **natural language understanding**.
- A representation of the **meaning** of a sentence, built from a **small fixed set of primitive acts** instead of from the sentence's own words.
- **Three goals**, and the papers want all three:
 - to **help draw inferences** from a sentence;
 - to be **independent of the words** used in the input;
 - so that **any two sentences identical in meaning have exactly one representation**.
- Hence *I gave the man a book, the man got a book from me* and *the book was given to the man by me* all draw the **same** diagram. That is the point of the whole theory.
- A **strong** slot-and-filler structure: unlike a frame, the slots have **fixed, agreed meanings**.

The four conceptual categories

- **ACT** – an action, always one of the primitives below.
- **PP** – *picture producer*, an object or actor.
- **AA** – *action aider*, a modifier of an ACT (how, when, where).
- **PA** – *picture aider*, a modifier of a PP (colour, size, quality).
- **Dependency rule**: PP and ACT are **governors**, PA and AA are **dependents**. A dependent cannot stand alone; a conceptualization needs at least two governors.
- Eg *Sally stroked her fat cat*: PP = Sally, cat; ACT = stroke; PA = fat.

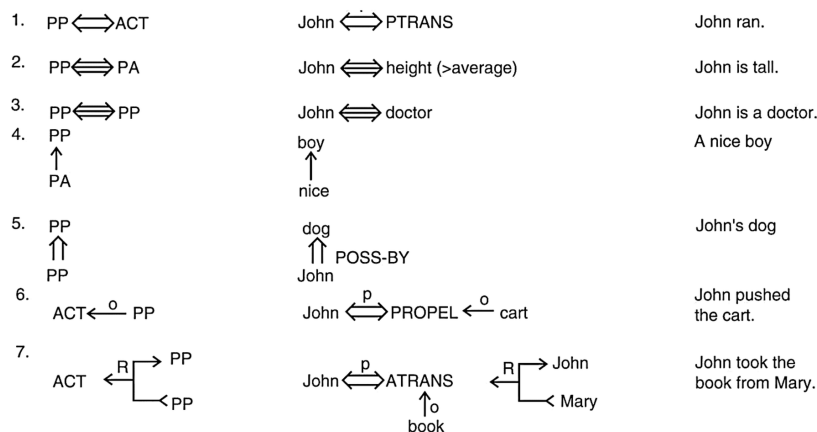
The eleven primitive acts

Primitive	Transfer or action, and a verb that maps to it
ATRANS	transfer of an abstract relationship , ie possession. <i>give, lend, borrow</i>
PTRANS	transfer of the physical location of an object. <i>go, come, went</i>
PROPEL	application of a physical force to an object. <i>push, pull</i>
MOVE	movement of a body part by its owner. <i>kick, punch</i>
GRASP	an actor grasping an object. <i>clutch, catch</i>
INGEST	an animal taking something in . <i>eat, drink</i>
EXPEL	an animal forcing something out . <i>cry, sweat, throw</i>
MTRANS	transfer of mental information . <i>tell, ask, read</i>
MBUILD	building new information out of old. <i>decide, infer, conclude</i>
SPEAK	producing a sound . <i>say, talk, play music</i>
ATTEND	focusing a sense organ on a stimulus. <i>listen, watch</i>

- **Learn the first three and MTRANS**. Every example in every paper and every scene of the restaurant script is built from those four.

Draw the conceptual dependency of a sentence 2 5

inside 71 Bh 5 · the notation cannot be typed, so **learn it as a picture**



The seven dependency forms, then each one used on a sentence. Row 6 is *John pushed the cart* (ACT $\xleftarrow{o}$ PP, PROPEL); row 7 is *John took the book from Mary* (ATRANS with a recipient pair). Source: BA Sir CH-05.

The notation

- ↔ **two-way link** between an actor and the action. Every conceptualization has exactly one.
- **direction of dependency**: the dependent points at its governor.
- o** = objective case, what the act is done to.
- R** = recipient case, drawn as a **fork**: the arrow up is the receiver, the arrow down is the giver.
- I** = instrumental case, what it was done with.
- D** = directive case, from where to where.
- p** = past tense, **f** future, **t** transition, **?** interrogative, **/** negative. Tense sits on the ↔.
- Worked example** – *I gave a book to Ram*:
 - $I \leftrightarrow ATRANS$, with **p** on the link;
 - $\xleftarrow{o}$ book;
 - $\xleftarrow{R}$ forking up to *Ram* and down to *I*.

Problems with CD

- The **original sentence cannot be reconstructed** from the diagram; meaning survives, wording does not.
- Choosing the primitive is hard**: one verb may map to several ACTs depending on context.
- The **structures get huge**. “*John bet Mike that Nepal will win the World Cup*” needs an enormous diagram for one sentence.
- Rules must be **hand-written per primitive**, so the theory does not scale to a general-purpose KR.

How is knowledge represented using scripts? Explain with an example 5

77 Ch 5 · 1 paper at 5 marks, so no chip of its own · **always asked as the second half of the CD question**, never alone · the restaurant script is the only example any deck or the textbook uses, so learn that one

What a script is

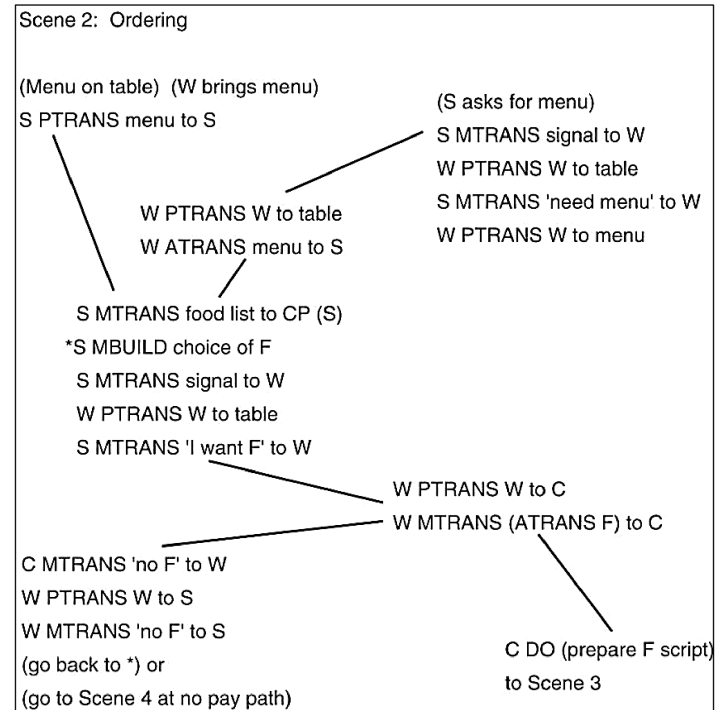
- Schank and Abelson, 1977**, written in the CD notation.
- A **structured description of a stereotyped sequence of events in a known context**: going to a restaurant, going to the theatre, visiting a doctor.
- It **extends a frame** by making the slot values **ordered in time** and by representing **expectations** about what happens next.
- Justified by three facts about the world: events occur in known patterns, events cause one another, and entry conditions decide whether a pattern applies at all.

The six components (write all six)

- Entry conditions** – must be true before the script can be invoked. Eg *S is hungry*, *S has money*.
- Results** – true once it has run. Eg *S is not hungry*, *O has more money*.
- Props** – the objects involved. Eg tables, menu, food, check, money.
- Roles** – the participants and what each does. Eg *S* = customer, *W* = waiter, *C* = cook, *M* = cashier, *O* = owner.
- Track** – a variation on the general pattern. Eg *coffee shop* Vs *fancy restaurant*: same script, different track.
- Scenes** – the ordered sequence of events, each written in CD primitives.

The RESTAURANT script, header and scene 1

- **Script:** RESTAURANT. **Track:** coffee shop.
- **Props:** tables, menu, F = food, check, money.
- **Roles:** S = customer, W = waiter, C = cook, M = cashier, O = owner.
- **Entry conditions:** S is hungry, S has money.
- **Results:** S has less money, O has more money, S is not hungry, S is pleased (optional).
- **Scene 1, entering:** S PTRANS S into restaurant · S ATTEND eyes to tables · S MBUILD where to sit · S PTRANS S to table · S MOVE S to sitting position.



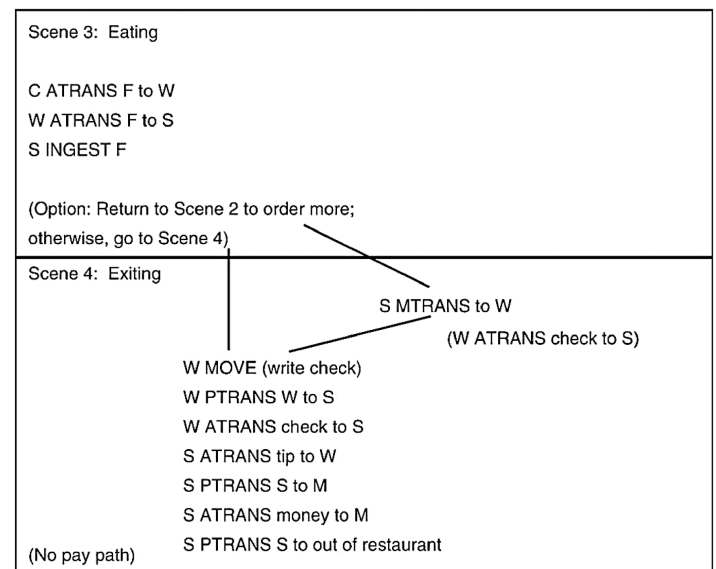
Scene 2, ordering, with both tracks: menu already on the table, or the waiter brings it. Source: BA Sir CH-05.

Invocation, and why it is the hard part

- A script is opened when its topic is **significant** in the input; if merely mentioned, only a pointer is kept.
- “John enjoyed the play in the theatre” → invoke, and every implicit question is answered: did he go, did he buy a ticket, did he have money.
- **The failure case to quote:** “John went to the theatre to pick up his daughter”. Same words, low significance, and invoking the script produces confident wrong answers.
- There is no reliable rule for judging significance, only heuristics. That is the honest limitation to close on.

Advantages and disadvantages

- ✓ **Predicts implicit events:** it fills in what the text never said, which is exactly what language understanding needs.
- ✓ Builds **one coherent interpretation** out of scattered observations.
- × **Far more specific and less general than a frame.** One script covers one situation.
- × Not suitable for knowledge that is not event-sequenced.
- × Wrong invocation gives confidently wrong answers.
- The cure for the inflexibility: **MOPs**, memory organization packets – smaller reusable modules recombined to fit the situation.



Scene 3 eating and scene 4 exiting, including the *no pay* path. Source: BA Sir CH-05.

Source notes. *Insights* §5.4.2 (book p135) converts the Tom-the-cat semantic net into frames, but its figure prints like John and type Ginger where the net says *is-owned-by* and *is-coloured*, and gives the John frame an Age 22 / Color white pair carried over from the car example on the facing page. The net-to-frame recipe in §5.6 above follows the book's *method* and Rich and Knight's *figures*, and the companion works the conversion on a net

that is drawn correctly. · The syllabus files this chapter as five sub-topics but the papers only ever set four: representations and mappings is never asked on its own, always as the two-mark opener to *what is knowledge representation*.

6. Machine Learning

6 hrs · asked in **40 of 41 papers** · 6 syllabus sub-topics · 436 marks, 12.9% of every mark ever set · the ID3 data tables, the GA generations and the fuzzy arithmetic are worked in *Ch6 – Machine Learning – Solved Problems*.

Read this first:

- **Two topics carry this chapter, and both are drawings. Genetic algorithm** **TOP 21/41** and **fuzzy logic** **TOP 16/41**. Between them they are asked in **31 of the 41 papers** and they take roughly two thirds of the chapter's marks. Learn the GA flowchart and the fuzzy-inference block diagram cold; everything else here is a bonus.
 - **The examiner pairs them with each other.** 82 Ka, 82 Bh and 81 Ba all ask fuzzy logic and GA in the *same* question. Answering one well and the other badly costs the whole 8 marks.
 - **Third is supervised Vs unsupervised** **TOP 12/41**, always with the rider “*clustering falls under which category? Justify, stating algorithms*”. Name **K-means** and **hierarchical clustering** or the justification scores nothing.
 - **ID3 is the only real arithmetic in the chapter** **HOT 6/41**, and it is always the same shape: entropy of the whole set, entropy of each attribute's split, gain = difference, pick the largest. Three different data tables appear, all worked in the companion.
 - **The syllabus line says “fuzzy learning” and no paper has ever asked that.** Every one of the sixteen asks the **fuzzy inference system** – membership functions, rules, fuzzification, defuzzification. §6.7 teaches the inference system and says where the two names meet.
 - Reliable short notes, each a fixed list you can write in four minutes: **GA** **3 4**, **fuzzy logic** **3 4**, **Boltzmann machine** **4**, **supervised Vs unsupervised** **4**, **mutation Vs crossover** **3**, **induction Vs deduction** **3**.
-

6.1 What machine learning is, and the learning framework

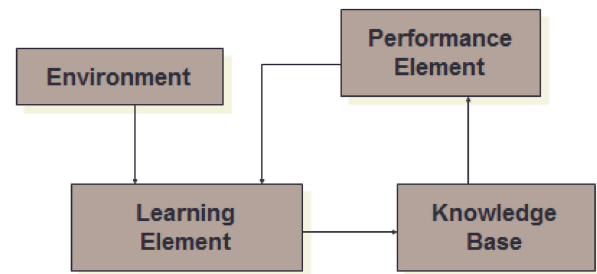
What is machine learning? Explain the learning framework with a block diagram. 1 6 2+6

PIN 2/41 78 Ba 2+6, 69 Bh 6 · the definition alone opens eleven more papers, always as the 1 or 2 lead-in to a GA, fuzzy or Boltzmann question (80 Ash 1 asks nothing else) · so learn the definition as one sentence and the framework as four boxes

Definition

- Machine learning is the branch of AI in which a program improves its performance on a task from experience, without being explicitly programmed for that task.
- Tom Mitchell's form, worth quoting: a program **learns** from experience E w.r.t. task T and measure P if **its performance at T , measured by P , improves with E** .
- In AI terms: the agent **observes data, builds a model** of it, and uses that model as both a hypothesis about the world and a program that solves problems.
- Three things learning always does**, and the three-word answer to "what is learning":
 - Change** – the learner itself is altered.
 - Generalization** – performance improves on *similar* tasks, not only on the ones seen.
 - Improvement** – and the design must stop changes that make performance worse.
- Why an intelligent agent needs it** (asked as "learning is an essential characteristic of intelligent agents, comment", Ch 1): a designer cannot enumerate every situation; the environment changes after deployment; and some tasks have no known algorithm, only examples.

The learning framework: four components



Environment feeds the learning element; the learning element writes the knowledge base; the performance element uses it and reports back.

- 1. Environment** – supplies the information. Judge it on:
 - Level:** high level = abstract, covers a broad class of problems; low level = detailed, covers one problem.
 - Quality:** noise free, reliable, ordered.
- 2. Learning element** – turns that information into knowledge. Rote learning, learning by example, by analogy, explanation-based (§6.2).
- 3. Knowledge base** – stores what was learnt. Must be **expressive, modifiable** and **extendible** (holds meta-knowledge, so its own structure can change).
- 4. Performance element** – uses the knowledge to act, measures how well it did, and **feeds the error back** to the learning element. That feedback arrow is the whole point of the diagram: draw it.

6.2 Ways of learning, and induction Vs deduction

What are the different ways / methods of learning? Describe each. 8 3+5

PIN 2/41 71 Bh 8, 81 Ba 3+5 · 81 Ba asks it as the 3 half of a GA question, so give the list, not essays · 78 Ch 2 asks the narrower “*algorithms inspired by biological phenomena and social behaviour*”: genetic algorithms, neural networks, swarm intelligence (ant colony, particle swarm), artificial immune systems

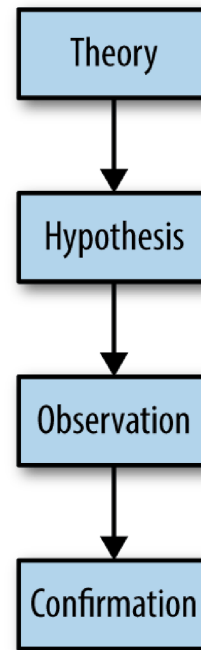
The five ways, easiest to hardest

- **1. Rote learning** – memorise and recall.
 - Solved problem → store → look up next time.
 - No inference at all: nothing new is derived.
 - **Only worth it if retrieval is cheaper than recomputing.**
 - Eg Samuel's checkers player, which stored board positions it had seen.
- **2. Learning by example (inductive)** – specific cases → general rule. Eg decision trees, ID3 (§6.5).
- **3. Learning by analogy** – transfer a solution from a known similar problem (§6.4).
- **4. Explanation-based learning (EBL)** – use existing domain knowledge to *explain* one example, then generalise the explanation into a rule.
 - Learns from **a single example**, because the prior knowledge does the work the extra examples would have done.
 - Eg see one cup once, know what makes it a cup, recognise every cup after.
- **5. Learning by deduction** – general rules → specific facts, or new general rules from old ones. $A = B$, $B = C$, so $A = C$.

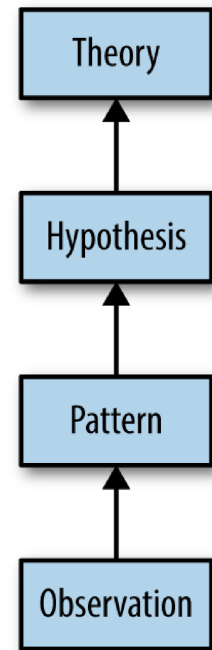
Induction Vs deduction

asked as a short note by 73 Bh 3.

Deductive Reasoning



Inductive Reasoning



Read the left ladder downwards and the right one upwards: that direction is the whole distinction.

	Inductive	Deductive
Direction	observation → hypothesis	hypothesis → observation
Goes from	specific	general
Produces	a general rule from raw data	a specific fact from known rules
Truth	not guaranteed , only probable	guaranteed if the premises hold
Reasoning	probabilistic	logical (Ch 4)
Example	1000 white swans → “swans are white”	all men are mortal; Socrates is a man; so Socrates is mortal

- **The one-line answer:** induction generalises and can be wrong; deduction specialises and cannot.
- Both appear in §6.2's list of ways of learning, and the papers ask them as a 3 short note, so three rows of that table plus this line is a full answer.
- **Where each one lives in this subject:** deduction is Chapter 4 (resolution, forward and backward chaining, all truth-preserving); induction is this chapter (ID3, §6.5) and every learning algorithm in it.
- **Abduction** is worth one extra line if you have room: reasoning from an observation to its *most likely cause*. Medical diagnosis. Also not truth-preserving.

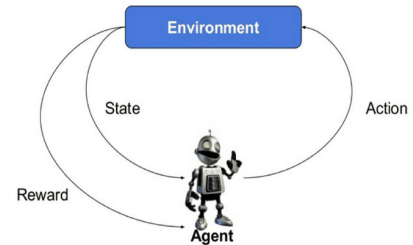
6.3 Supervised, unsupervised and reinforcement learning

Differentiate supervised and unsupervised learning. Clustering falls under which category? Justify, stating algorithms. 4 5 2+4 4+4 2+6

TOP 12/41 74 Ma 2+4, 72 Ash 2+4, 74 Bh 4+4, 79 Jth 4+4, 76 Bh 5, 78 Po 2+6, 69 Po 4, 74 Bh 4, 78 Ba 4, 77 Ch 4, 81 Ash 4, 75 Bh 4 · **third-most-asked topic in the whole subject** · two riders recur: “clustering falls under which?” (74 Ma, 72 Ash) and “learning is essential for intelligent agents, comment” (74 Bh, 79 Jth) · 78 Po wants all three types with examples

	Supervised	Unsupervised
Training data	labelled: every input carries its correct output	unlabelled: inputs only
Aim	learn the input → output mapping, then predict for new inputs	find hidden structure, groups and patterns in the data
Feedback	direct: error = predicted – actual, at every sample	none; the model is judged only by the structure it finds
Tasks	classification, regression	clustering, association
Algorithms	linear + logistic regression, SVM, naive Bayes, KNN, decision tree, back-propagation ANN	K-means, hierarchical (agglomerative), DBSCAN, Apriori, SOM
Human effort	high: someone has to label everything	low
Accuracy	higher, and measurable	lower, and hard to measure
Example	spam filter trained on mails already marked spam / not spam	segment customers by purchase behaviour, with no idea of the segments beforehand

Reinforcement learning



- **Feedback-based:** the agent acts, the environment returns a **reward** or a **penalty**, and the agent adjusts.
- No labelled data and no supervisor. The agent **discovers** which actions pay by trying them.
- Goal = maximise **cumulative** reward over the long run, not the next reward.
- **Four elements to name** (78 Po):
 - **Policy** – state → action mapping.
 - **Reward signal** – one number per step, defines the goal.
 - **Value function** – total reward expected from a state: good in the long run, where reward is good in the short run.
 - **Model** of the environment (optional).
- Example: a robot that gets +10 for reaching the goal square and –50 for falling in a pit. Also game playing, robot navigation, self-driving control.
- **Semi-supervised** is the fourth type worth one line: a small labelled set plus a large unlabelled one, used when labelling everything is too costly.

The rider: which category is clustering? (74 Ma, 72 Ash, 4 of the 6)

- **Clustering is unsupervised**, because the data carries **no class label** – the algorithm both invents the groups and assigns points to them.
- **Justify by naming algorithms**, which is what the question asks for and where the marks are:
 - **K-means** – choose k , assign every point to the nearest centroid, recompute the centroids, repeat until nothing moves. Nobody ever tells it a right answer.
 - **Hierarchical (agglomerative)** – start with every point its own cluster, repeatedly merge the two nearest, stop at k clusters or one.
 - **DBSCAN** – groups by density, and finds the number of clusters itself.
 - **SOM / Kohonen** (Ch 7) is a clustering network, and is unsupervised for exactly the same reason.
- **Contrast with classification**, which draws the same picture but is supervised: the classes exist *before* training and every training point already carries one.

Deciding the type of any example the examiner invents

What does the data carry?

Then it is

a correct output for every input	supervised
nothing but inputs	unsupervised
a few labelled, most not	semi-supervised
no outputs, but a reward after acting	reinforcement

- Worked on the examples the papers use: spam filter = supervised (mails already marked); customer segmentation = unsupervised; game-playing agent = reinforcement; handwriting recognition trained on labelled digits = supervised; market-basket association = unsupervised.
- **One sentence that is always worth writing:** supervised learning is told the answer and learns the mapping; unsupervised learning is told nothing and learns the structure; reinforcement learning is told only whether it did well, and learns the policy.

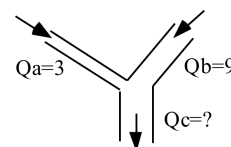
6.4 Learning by analogy

What is learning by analogy? Give an example, with a semantic network. 3 2+6 1+7

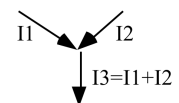
PIN 3/41 71 Bh 1+7, 76 Ba 2+6, 71 Ma 3 · 71 Bh demands the semantic network, so the answer is a *drawing plus the mapping table* · 71 Ma asks it as *analogy Vs inductive learning*

- **Learning by analogy = acquiring knowledge about a new entity by transferring it from a known, similar entity.**
- It carries a solution **across domains**: find the analogous states and operators in the two domains, then reuse the old solution.
- **One example is enough**, which is what makes it different from inductive learning (which needs many).
- **The five steps** (write these for a 6 or 7 mark answer):
 - **1. Recognition** – notice the new problem resembles a past one.
 - **2. Access and recall** – the resemblance indexes memory; retrieve one or more candidate analogues.
 - **3. Selection and mapping** – pick the relevant parts and map them from the **base** (known) domain to the **target** (new) one.
 - **4. Extension** – modify and extend the mapped solution to fit the target.
 - **5. Validation and generalization** – test it; if it holds, form a general rule covering both situations.
- **Analogy Vs induction** (71 Ma 3): analogy maps *one* case onto *one* case and reasons by similarity; induction generalises *many* cases into a rule. Analogy transfers, induction abstracts.

First example: hydraulics and Kirchhoff



Simple Hydraulics Problem



Kirchoff's First Law

- Flow into a junction equals flow out; current into a node equals current out. Knowing $Q_a = 3$, $Q_b = 9$ gives Q_c the same way $I_3 = I_1 + I_2$ does.
- **The mapping:** pressure drop $\leftrightarrow$ voltage drop, pipe $\leftrightarrow$ wire, flow rate $\leftrightarrow$ current, junction $\leftrightarrow$ node.
- So a student who knows Kirchhoff's law can solve the hydraulics problem **without being taught hydraulics**. That sentence is the definition made concrete, and it is what the examiner wants.

- **Second example: the atom and the solar system**, the one to draw as a **semantic net** for 71 Bh: two small nets, one arc list mapped onto the other.
 - Base net: Sun --greater-mass--> Earth,
Sun --attracts--> Earth,
Earth --revolves-around--> Sun.
 - Target net: Nucleus --greater-mass--> Electron,
Nucleus --attracts--> Electron, and the arc analogy **supplies**: Electron --revolves-around--> Nucleus.
 - Draw the two nets side by side and a **dashed arrow between the matching nodes**: Sun ↔ Nucleus, Earth ↔ Electron. That dashed mapping is the answer.

The mapping table, which is the marked half

Role	Base: solar system	Target: atom
Heavy central body	Sun	Nucleus
Light orbiting body	Earth	Electron
Relation known	attracts, greater mass	attracts, greater mass
Relation transferred	Earth revolves around Sun	Electron revolves around Nucleus

- The last row is the whole point: it is **not observed, it is carried across** by the analogy, and then validated.
- Say in one line what analogy risks: the mapping may not hold (electrons do not really orbit like planets), so step 5 validation is not optional. Naming that limitation reliably earns a mark.

6.5 Inductive learning, decision trees and ID3

Define inductive learning. How is the best attribute selected in a decision tree? Explain ID3 with an example. 8 1+7 2+6 2+7

HOT 6/41 74 Bh 2+7, 74 Ma 8, 72 Ash 8, 79 Jth 1+7, 77 Ch 8, 74 Bh 2+6 · five of the six print a data table and want the root attribute computed, so this section is theory only – all three tables are worked in the companion · 77 Ch disguises it as the *mushroom island* story

Inductive learning, and the tree

- Inductive learning = drawing a general hypothesis from a set of specific input–output examples, chosen so that it classifies the examples well *and* generalises to new ones.
- A decision tree is the standard hypothesis form:
 - each **internal node** tests one attribute,
 - each **branch** is one value of that attribute,
 - each **leaf** carries a class label.
- To classify, walk root to leaf and read off the label.
- ID3 (Iterative Dichotomiser 3, Quinlan) builds that tree **greedily, top down**: at every node it puts the attribute that separates the classes best, and never goes back.

The two formulas – memorise these

$$H(S) = - \sum_i p_i \log_2 p_i$$

entropy of set S : p_i is the fraction of S in class i . $H = 0$ when S is pure, $H = 1$ when a two-class set is split 50/50.

$$\text{Gain}(S, A) = H(S) - \sum_{v \in A} \frac{|S_v|}{|S|} H(S_v)$$

the second term is the *weighted average* entropy after splitting on A . Gain is the uncertainty removed.

- **Highest gain = lowest weighted entropy = best attribute.**
The two rules are the same rule.

Why it works, in one figure

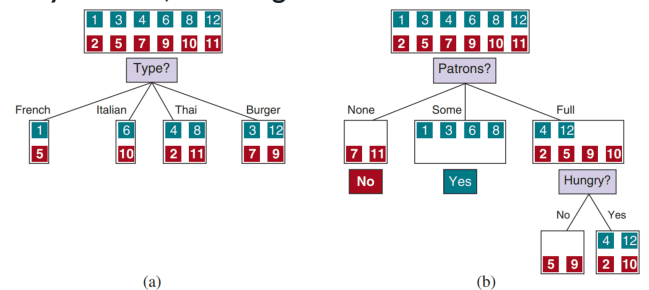


Figure 19.4 Splitting the examples by testing on attributes. At each node we show the positive (light boxes) and negative (dark boxes) examples remaining. (a) Splitting on *Type* brings us no nearer to distinguishing between positive and negative examples. (b) Splitting on *Patrons* does a good job of separating positive and negative examples. After splitting on *Patrons*, *Hungry* is a fairly good second test.

Splitting on *Type* leaves every branch as mixed as the parent: gain 0, useless. Splitting on *Patrons* produces two pure branches and one mixed one: high gain. That comparison is the answer to “how is the best attribute selected”.

The algorithm, as five steps

- 1. Compute $H(S)$ for the current set.
- 2. For each unused attribute A , split S by its values and compute the weighted entropy of the split.
- 3. $\text{Gain}(S, A) = H(S) -$ that weighted entropy.
- 4. Make the attribute with the **largest gain** the node, and branch on its values.
- 5. Recurse on each branch with the remaining attributes. Stop a branch when: it is **pure** (leaf = that class); **no attributes are left** (leaf = majority class); or it is **empty** (leaf = parent's majority).
- Three arithmetic checks that save marks:** H of a pure branch is 0, not 1; H of an even two-class split is exactly 1; and every gain must be ≥ 0 , so a negative one is an error in your weighted sum.

Characteristics to list if asked

- Greedy, so **no guarantee of the optimal tree** – it can settle in a local optimum.
- Produces small trees, but not always the smallest.
- Overfits** the training data; prefer smaller trees, and prune.
- Awkward on **continuous** attributes: too many possible split points.
- Handles noise and missing values badly. C4.5 fixes all four.

6.6 Genetic algorithm

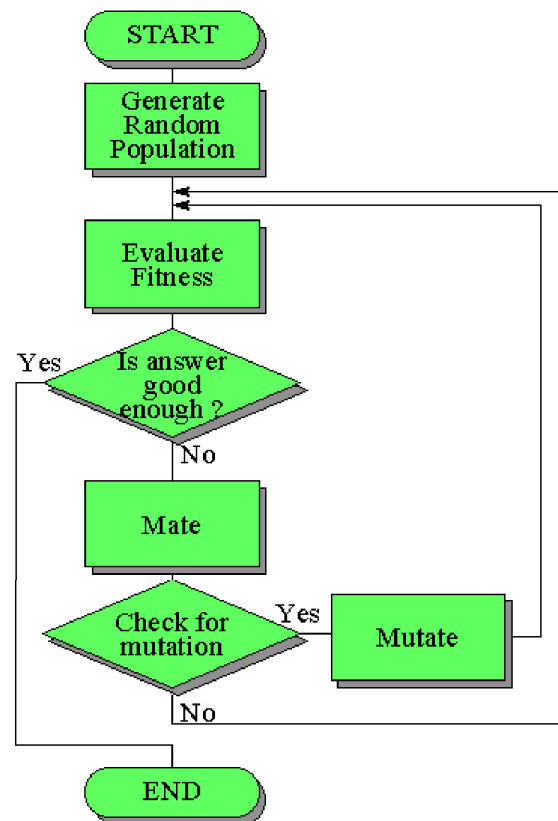
What is a genetic algorithm? Explain all its steps with a block diagram, and its operators. 3 4 5 7 8

10 2+8

TOP 21/41 69 Po 10, 71 Ma 2+8, 72 Ma 2+6+2, 75 Ba 8, 77 Ch 8, 81 Bh 2+6, 79 Ch 2+6, 80 Ba 8, 76 Bh 5, 81 Ba 3+5, 72 Ma 7, 82 Ka 2+6, 82 Bh 2+6, 81 Ash 3+5, 81 Ch 7, 78 Ch 2+6, 74 Bh 4, 73 Bh 3, 70 Bh 2+6, 74 Bh 4+4, 75 Bh 2+8 · the single most-asked topic in the chapter and the second most-asked in the subject · it has been set in every year from 2069 to 2082 · the wording rotates but the answer never does: steps + block diagram + the three operators + a worked example

Definition and idea

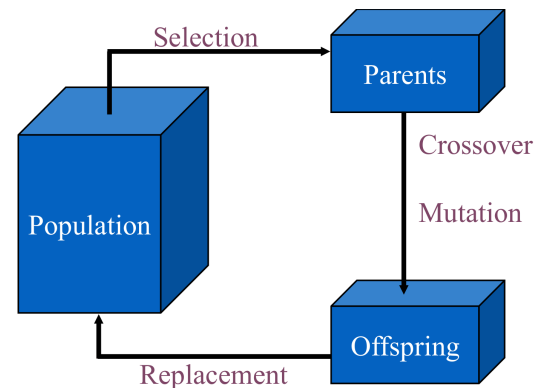
- A genetic algorithm is a search and optimization technique that imitates natural selection: it keeps a population of candidate solutions and evolves it, generation by generation, towards better ones.
- Darwin in one line: **select the best, discard the rest.**
- Evolutionary computing** (asked by name in 79 Ch) is the family; GA is its best-known member, alongside genetic programming, evolutionary strategies and swarm methods.
- Formally a variant of **stochastic beam search**: many states kept at once, successors generated by combining pairs rather than mutating one.
- Vocabulary** – the examiner marks these:
 - Gene** – one position in the string; controls one characteristic.
 - Chromosome** (individual) – a chain of genes = **one candidate solution**, usually a binary string.
 - Population** – the set of chromosomes alive in one generation.
 - Fitness function** – scores a chromosome. **Problem dependent; it is the only thing that knows what “good” means.**
 - Generation** – one full cycle of the loop.



The flowchart 81 Ba asks for by name. Draw START → generate random population → evaluate fitness → good enough? → (no) select and mate → mutate → back to evaluate; (yes) END.

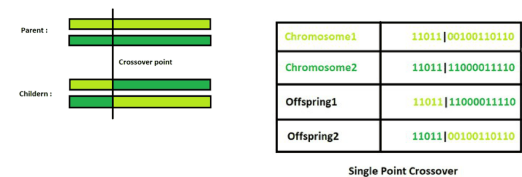
The five steps, which are also the five operators

- **1. Initialization** – generate the starting population at random.
 - Encode a solution as a string first (**chromosome encoding**); binary is usual, but integer and permutation encodings exist.
 - Population size trades diversity against run time. Seed it around a promising region if you have one.
- **2. Fitness evaluation** – score every chromosome with the fitness function. Infeasible chromosomes get fitness 0.
- **3. Selection** – choose parents, with probability rising with fitness.
 - **Roulette wheel**: slice proportional to fitness. **Tournament**: pick k at random, take the best. **Rank**: order them, select on rank. **Elitism**: copy the best one or two through untouched.
 - **Keep a few weak ones on purpose** – they hold the diversity the next crossover needs.
- **4. Crossover (recombination)** – cut two parents and swap the tails, so each child inherits from both. Probability typically 0.6 to 0.9.
- **5. Mutation** – flip a random gene with a small probability, typically 0.001 to 0.01.
- **6. Replacement, then termination** – offspring replace the weakest members; stop on a **fixed number of generations**, a **fitness threshold**, or when the population has **converged** (no improvement for several generations).

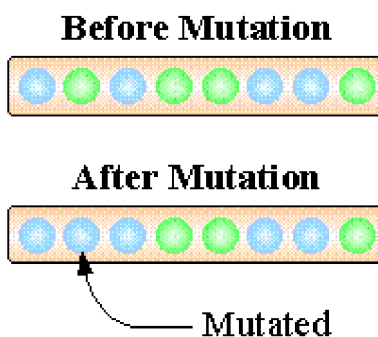


The same algorithm as one cycle, which is the “block diagram” most papers want: population → selection → parents → crossover, mutation → offspring → replacement → population.

Crossover, drawn



Single-point: one cut, tails swapped. **Two-point**: two cuts, the middle segment swapped. **Uniform**: each gene taken from either parent by a coin toss.



Mutation Vs crossover (73 Bh 3)

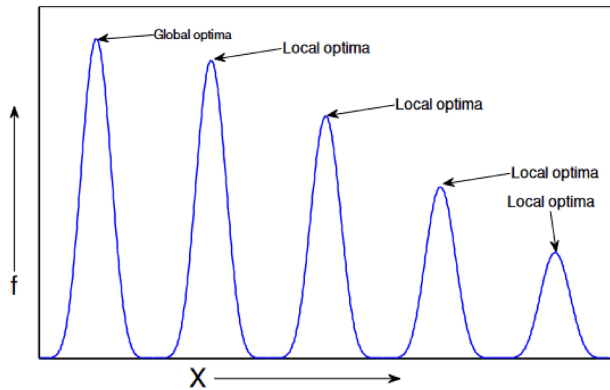
	Crossover	Mutation
Parents	two	one
Does	exchanges segments between two chromosomes	flips a gene in one chromosome
Rate	high, 0.6–0.9	low, 0.001–0.01
Role	exploitation : combines what already works	exploration : introduces what is missing
If absent	population stagnates as copies of a few parents	search sticks in a local optimum

Mutation flips one gene of one child.

Why mutation matters (81 Ba asks exactly this)

- Crossover only **recombines genes already in the population**. If every chromosome carries a 0 in some position, no amount of crossover will ever produce a 1 there.
- Mutation is the **only source of new genetic material**. It keeps diversity up and stops the search converging early on a **local optimum**.
- So the rate must be *small*: too high and the search degenerates into a random walk, losing the good solutions selection just found.

When shall we use a genetic algorithm? (81 Ash, 80 Ch)



One global and four local optima. A hill-climber stops on whichever bump it started under; a population searches all five at once.

- The search space has **many local optima** and gradient methods get trapped.
- The objective function is **not smooth**, not differentiable, or is only available as a black box, so no derivative method applies.
- The number of parameters is **very large** and exhaustive search is impossible.
- The objective is **noisy or stochastic**.
- A **good-enough** answer is acceptable: GA gives no optimality guarantee.
- **Do not use it** when an exact algorithm exists, when a single evaluation of the fitness function is expensive, or when the answer must be provably optimal.

Where it is applied (72 Ma 2)

- Optimization: travelling salesman, knapsack, job-shop scheduling, timetabling.
- Engineering design, antenna and circuit layout, PID controller tuning.
- Neural network weight and architecture search; feature selection in ML.
- Robotics path planning, vehicle routing, bioinformatics (gene sequencing).
- **Why the study of genes belongs in AI (74 Bh 4):** evolution is a **proven search procedure** that solved design problems no designer could; GA borrows the mechanism, not the biology.

Advantages and limitations

- ✓ Simple concept; works on the **encoded** parameters, so it needs no derivative and no auxiliary information, only a fitness score.
- ✓ Searches many points at once, so it escapes local optima; inherently **parallel**; the answer improves with run time and can be stopped anywhere.
- × No guarantee of the global optimum; × many fitness evaluations, so slow when each is costly; × the encoding, population size and rates all need tuning; × can converge prematurely if diversity is lost.

6.7 Fuzzy logic and the fuzzy inference system

What is fuzzy logic? When do we use it? Explain fuzzy inference / the steps, with an example. 2 3

4 8 2+6 3+7 1+2+5

TOP 16/41 69 Bh 4, 70 Ma 2+6, 80 Ash 1+2+5, 73 Ma 3+7, 79 Ash 1+2+5, 80 Ch 2+5, 82 Ba 2+6, 80 Bh 8, 79 Bh 2+6, 81 Ba 4, 78 Po 4, 77 Ch 4, 81 Ash 4, 80 Ba 3, 82 Ka 2, 82 Bh 2 · the syllabus calls this “fuzzy learning” and no paper ever has: they ask fuzzy logic and fuzzy inference, which is reasoning, not learning · 73 Ma is the only one to name Mamdani · 82 Ka and 82 Bh bolt it onto a GA question

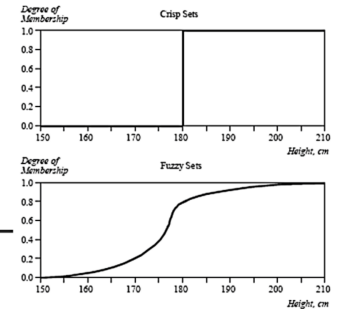
What it is

- Fuzzy logic is a many-valued logic in which a proposition has a degree of truth anywhere in $[0, 1]$, instead of only 0 or 1.
- It exists because human concepts have **no sharp boundary**: *tall, hot, slow, near, honest*. A crisp rule must answer *is 179 cm tall?* yes or no; a fuzzy one answers 0.78.
- Introduced by **Lotfi A. Zadeh**, UC Berkeley, **1965**.
- **Fuzzy set**: for a universe U , a fuzzy set A is the set of pairs $\{(x, \mu_A(x)) : x \in U\}$ where the **membership function**

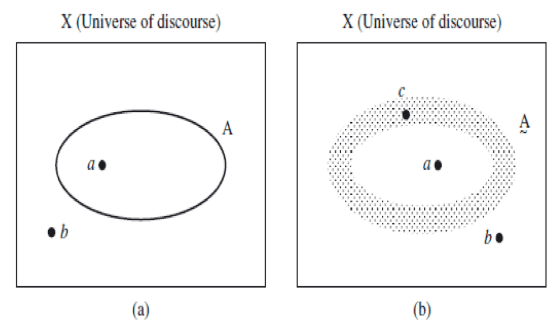
$$\mu_A(x) : U \rightarrow [0, 1]$$

- $\mu_A(x) = 1 \rightarrow x$ is fully in A ; $\mu_A(x) = 0 \rightarrow$ fully out;
 $0 < \mu_A(x) < 1 \rightarrow$ partly in.
- **Linguistic variable** – a variable whose values are words: Temperature = {very cold, cold, warm, hot, very hot}. Each word is a fuzzy set with its own membership curve, and the curves **overlap** on purpose.

Name	Height, cm	Degree of Membership	
		Crisp	Fuzzy
Chris	208	1	1.00
Mark	205	1	1.00
John	198	1	0.98
Tom	181	1	0.82
David	179	0	0.78
Mike	172	0	0.24
Bob	167	0	0.15
Steven	158	0	0.06
Bill	155	0	0.01
Peter	152	0	0.00



Crisp Vs fuzzy on the same data. Membership of *tall*: the crisp column steps from 0 to 1 at 180 cm, so Tom at 181 is tall and David at 179 is not. The fuzzy column grades them 0.82 and 0.78 – which is what a person would say. This one table answers “how does fuzzy logic differ from classical binary logic” (82 Bh) outright.



Diagrams for (a) crisp set boundary and (b) fuzzy set boundary.

The same point as sets: b is simply outside crisp A ; in fuzzy A the boundary is a band, so b and c have partial membership.

Fuzzy set operations

all defined pointwise on the membership values; worked on real numbers in the companion.

Operation	Membership
Union $A \cup B$	$\max(\mu_A, \mu_B)$
Intersection $A \cap B$	$\min(\mu_A, \mu_B)$
Complement A'	$1 - \mu_A$
Bold union (X-OR)	$\min(1, \mu_A + \mu_B)$
Bold intersection	$\max(0, \mu_A + \mu_B - 1)$
Equality	$\mu_A(x) = \mu_B(x)$ for every x

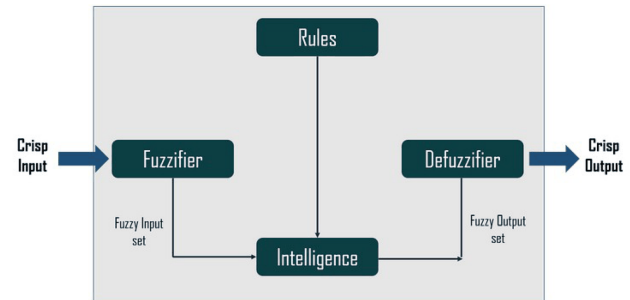
Fuzzy logic Vs probability

- Not the same thing, and papers reward saying so.
- Ram's membership of "old" is 0.9 – Ram is 90% old. A statement about **vagueness**, and it stays 0.9 after you meet him.
- $P(\text{Ram is old}) = 0.9$ – Ram either is or is not old and we do not know which. A statement about **ignorance**, and it collapses to 0 or 1 once you meet him.

The steps of "fuzzy learning" (80 Bh 8, 79 Bh 2+6, 69 Bh 4)

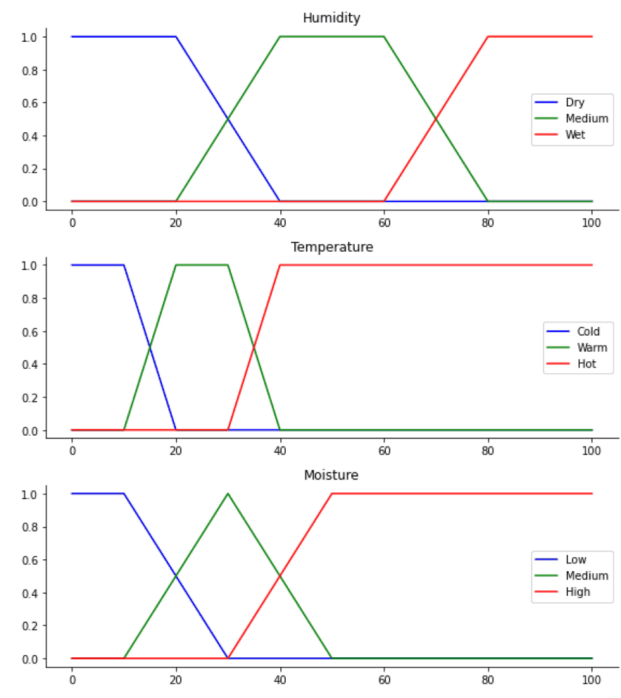
- 1. Define the linguistic variables and terms. Temperature = {very cold, cold, warm, very warm, hot}.
- 2. Construct a membership function for each term. Triangular and trapezoidal are normal; they must overlap.
- 3. Build the rule base of IF-THEN rules, usually as a matrix of every input combination against the action to take.
- 4. Fuzzify: convert the crisp readings into membership degrees.
- 5. Evaluate the rules in the inference engine, and aggregate the results into one fuzzy output.
- 6. Defuzzify into a crisp control value.
- Worked end to end on a fan-speed controller in the companion, §6.N.12; §6.N.13 builds that controller's variables, membership functions and rule base from a blank sheet, which is what 80 Ch and 82 Ba ask for instead of a run.

Architecture of a fuzzy inference system



The block diagram 82 Ba and five other papers ask for: crisp input → fuzzifier → fuzzy input set → inference engine (driven by the rule base above it) → fuzzy output set → defuzzifier → crisp output.

- Rule base – the IF-THEN rules supplied by domain experts. *IF temperature is cold AND target is warm THEN heater is on.*
- Fuzzification – read the crisp sensor value off each membership curve to get a degree for every linguistic term.
- Inference engine – match the fuzzified inputs against the rules, compute each rule's **firing strength**, and combine the consequents.
- Defuzzification – collapse the aggregated fuzzy output back to one crisp number. **Centroid** (centre of gravity) is the usual method; also mean of maxima, bisector, weighted average.



Step 2 for three linguistic variables of a soil-moisture controller.

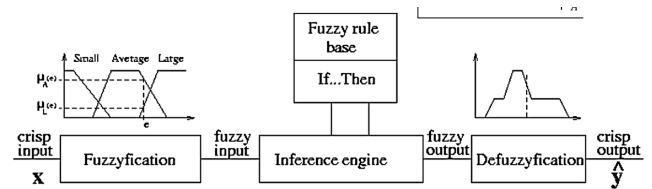
Note the overlaps: at 45 the humidity is *both* medium and wet, to some degree.

Mamdani fuzzy inference (73 Ma 3+7)

- Built by **Ebrahim Mamdani**, London, **1975**, to control a steam engine and boiler from rules given by human operators.
- Its six operations, in order: **determine the rules**, **fuzzify** the inputs, combine them to get each rule's **strength** (fuzzy AND = min), apply **implication** to clip the output set, **aggregate** the clipped sets, then **defuzzify**.
- **Mamdani Vs Sugeno** in one line: Mamdani's rule consequent is a *fuzzy set* and needs defuzzifying; Sugeno's is a *function of the inputs*, so the output is a weighted average and no defuzzification is needed. Mamdani is more intuitive, Sugeno cheaper.

Advantages

- Mimics human reasoning, so the rules are **readable and can be written by a domain expert** who knows no mathematics.
- Needs no exact mathematical model of the plant – decisive where one does not exist.
- **Robust**: imprecise, noisy or partly missing inputs still give a sensible output, so it degrades gracefully when a sensor fails.
- Simple, cheap to implement on small hardware; handles nonlinearity of arbitrary complexity.
- Proven in real products: washing machines, air conditioners, camera autofocus, anti-lock brakes, subway train braking.



The same architecture with the membership curves drawn on it, so “fuzzy input” and “fuzzy output” are visible as shapes rather than words.

Disadvantages, and when not to use it

- **No systematic design method.** Rules and membership functions come from expert opinion, so two designers get two systems.
- Accuracy depends entirely on the quality of that expert knowledge.
- Validation and verification need extensive hardware testing; there is no proof of stability the way there is for a classical controller.
- Not suited to problems needing high precision, nor to pattern recognition where a neural network learns the mapping from data.
- **Do not use it** when the input space does not map naturally to an output space, when a crisp model already works, or where common sense or a simple threshold would do.

6.8 Boltzmann machine

Define the Boltzmann machine. Explain it with a suitable algorithm. 4 1+7 2+8

PIN 3/41 70 Ma 1+7, 72 Ash 2+8, 76 Ba 4 · only three papers, but two of them put 7 or 8 marks on it · 70 Ma pairs it with the semantic network (Ch 5), 72 Ash with “what is machine learning”

- A Boltzmann machine is a stochastic, energy-based, recurrent neural network of symmetrically connected binary units that switch on and off at random, with a probability set by the network's energy.
- Invented by Hinton and Sejnowski, 1985. Named for the Boltzmann distribution it samples from.
- Structure:
 - Visible units V carry the data in and the reconstruction out.
 - Hidden units H capture features and dependencies.
 - Every pair is symmetrically connected, $w_{ij} = w_{ji}$, and there are **no self connections**.
 - **There is no output layer** – the same visible nodes that take the input return the reconstruction.
- State and energy. Each unit is 0 or 1; the network's energy is

$$E = - \sum_{i < j} w_{ij} s_i s_j - \sum_i \theta_i s_i$$

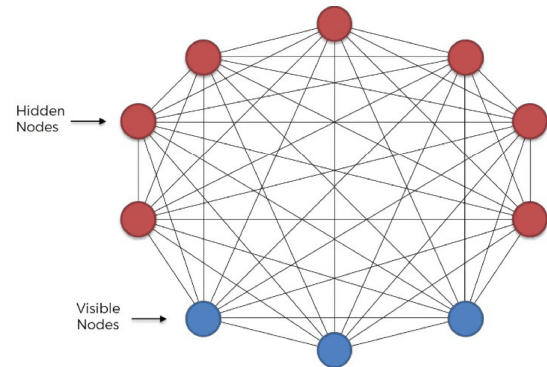
and the probability of a state falls exponentially with its energy:

$$p_i = \frac{e^{-\varepsilon_i/kT}}{\sum_j e^{-\varepsilon_j/kT}}$$

- So the machine **prefers low-energy states but can still climb out of one**, which is how it escapes local minima – the same idea as simulated annealing (Ch 3), with T lowered as it runs.

The learning algorithm, in six steps

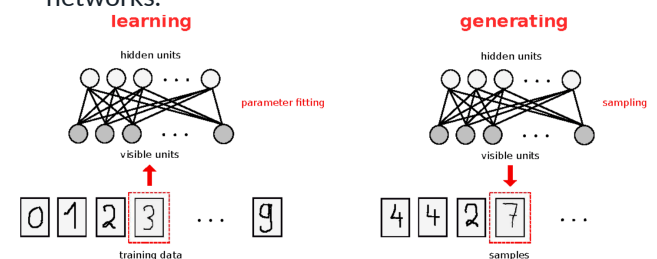
- 1. Initialise the weights randomly and set a high temperature T .
- 2. **Positive phase (clamped)**: fix the visible units to a training vector, let the hidden units settle, record the correlation $\langle s_i s_j \rangle_{data}$.
- 3. **Negative phase (free)**: release every unit, let the network run to equilibrium, record $\langle s_i s_j \rangle_{model}$.
- 4. **Update** $\Delta w_{ij} = \eta (\langle s_i s_j \rangle_{data} - \langle s_i s_j \rangle_{model})$.
- 5. Lower T a little (annealing) and repeat over the training set.
- 6. Stop when the weights stop changing.
- **What the two phases mean**: the clamped phase measures how units co-fire when the world is showing them the truth, the free phase how they co-fire when the network is dreaming on its own. Learning pushes the second towards the first.



Visible nodes below, hidden above, every pair connected both ways. Compare with a feed-forward net: no layers, no direction, no output side.

Restricted Boltzmann machine (RBM)

- A full Boltzmann machine is **far too slow**: every unit depends on every other, so the weights must be updated one at a time.
- RBM **forbids connections inside a layer**. Visible-to-hidden only.
- Units within a layer are then independent given the other layer, so a whole layer updates **in parallel**. Trained by **contrastive divergence**.
- Used for dimensionality reduction, classification, regression, collaborative filtering (recommender systems), and as the building block of deep belief networks.



6.9 Every Chapter 6 question, grouped by the answer it wants

Every Chapter 6 question in the 41 papers, grouped by the answer it wants. **Bold** = regular exam, plain = back. Monospace = CT 710 (BEI), *italic* = CT 78506 (BEX), upright = CT 653 (BCT).

- **Genetic algorithm 21 papers** → §6.6, worked in the companion. The wordings, all one answer: *explain GA and its operators* (69 Po, 71 Ma, 72 Ma); *all steps with block diagram* (75 Ba, 77 Ch, 81 Bh, 80 Ba, 76 Bh, 78 Ch); *steps with a flowchart, and why mutation matters* (81 Ba); *working principle with examples* (72 Ma, 81 Ch); *when shall we use it* (81 Ash); *what is evolutionary computing* (79 Ch); *the algorithm of GA* (70 Bh); *justify that the study of genes is part of AI* (74 Bh); short notes (74 Bh, 73 Bh as mutation Vs crossover); and bolted to a fuzzy question in 82 Ka and 82 Bh.
- **Fuzzy logic and fuzzy inference 16 papers** → §6.7. *Fuzzy inference with an example* (70 Ma, 80 Ash); *Mamdani method* (73 Ma); *when do we use it, and the steps* (79 Ash, 80 Ch); *architecture and working mechanism* (82 Ba); *all the steps in fuzzy learning with a block diagram* (80 Bh, 79 Bh, which also asks *what is membership of an element*); *what is fuzzy learning, with a practical example* (69 Bh); and short notes at 81 Ba, 78 Po, 77 Ch, 81 Ash, 80 Ba.
- **Supervised Vs unsupervised, and the three types 12 papers** → §6.3. The *clustering rider* is in 74 Ma and 72 Ash; the “*learning is essential for intelligent agents*” rider in 74 Bh and 79 Jth (answered in Ch 1); all three types with examples in 78 Po.
- **ID3 and the best attribute 6 papers** → §6.5 for the method, companion for all three data tables: *play golf* (74 Bh, 74 Ma, 72 Ash), *play cricket* (79 Jth), *poisonous mushrooms* (77 Ch). 74 Bh asks the ID3 process without a table.
- **Ways of learning 2 papers** → §6.2 (71 Bh 8, 81 Ba 3). *Induction Vs deduction* as a short note at 73 Bh 3. *Algorithms inspired by biology and social behaviour* at 78 Ch 2.
- **Learning framework 2 papers** → §6.1 (78 Ba, 69 Bh). *Define machine learning* alone at 80 Ash 1, and as the lead-in to eleven other questions.
- **Learning by analogy 3 papers** → §6.4. 71 Bh wants it *with a semantic network*; 71 Ma wants *analogy Vs inductive learning* 3.
- **Boltzmann machine 3 papers** → §6.8 (70 Ma, 72 Ash, 76 Ba).
- **Swarm intelligence** 2 verified/added – asked once, 75 Bh, and **no lecture deck in this subject mentions it**. Answer: a family of optimization methods in which many simple agents following local rules produce intelligent group behaviour with no central control – ant colony optimization (pheromone trails, shortest path), particle swarm optimization (each particle steers towards its own best and the swarm’s best), bee colony, flocking. Like GA it is population based and nature inspired; unlike GA the individuals **are not replaced, they move**. The other half of that question, the GA fitness numerical, is worked in the companion.
- **Reinforcement learning in detail** – named by 78 Po only, and the one lecture deck that covers it is PS Sir’s. §6.3 carries the four elements (policy, reward signal, value function, model).
- **Not in this chapter:** *neural networks* appear on the syllabus line for Chapter 6 but every paper sets them under Applications of AI – perceptron, back-propagation, Hopfield, Kohonen and Hebbian learning are all in **Ch 7**. *Learning in a neural network* short notes (69 Po, 74 Bh, 78 Ba, 77 Ch, 81 Ash) are answered there.

Source notes

- Spine: BA Sir CH-06 Machine Learning, PS Sir 6. Machine Learning, SG Sir AIChapter_6, and Insights on Artificial Intelligence ch 6 for the worked method.
- **Two corrections carried into the companion.** Insights p167 prints $\mu_{A'}(30) = 0.5$ for a complement whose correct value is 0.6, and p166 labels a fuzzified pair (10, 0.3) when the element is 9, not 10. Both are shown correctly there, with a note.
- **Reinforcement learning and Mamdani inference are in PS Sir’s deck only**, which is labelled *old syllabus*. Both are on the current CT 710 syllabus by way of the papers, so both are kept, checked against the standard sources.
- Swarm intelligence is verified/added – researched and written for 75 Bh, because no deck in Notes\ contains the word.

7. Applications of AI

14 hrs · asked in **all 41 papers** · 3 syllabus groups, 12 leaf topics · 899 marks, **26.5% of every mark ever set** · the gate networks, the Hopfield matrix, the forward propagation and the parse tree are worked in Ch7 – Applications of AI – Solved Problems.

Read this first:

- **This one chapter is a quarter of the paper.** It is bigger than chapters 1, 2, 3 and 5 put together, and **no paper has ever skipped it**. Expect three or four questions from it every time.
- **Four questions dominate, and each is nearly certain:**
 - The steps of NLP, with a block diagram **TOP 23/41** – the most-asked question in the entire subject.
 - Perceptron, and design a net for AND / OR / XOR **TOP 17/41**.
 - Expert system architecture, with a block diagram **TOP 17/41**.
 - Machine vision: role, steps, applications **TOP 12/41**.
- **Three drawings are worth more than anything else here:** the five-box NLP pipeline, the four-box expert system, and a two-input perceptron. Practise all three until you can draw them in one minute each.
- **Then the neural networks:** Kohonen **TOP 9/41**, Hopfield **TOP 8/41**, back-propagation **TOP 8/41**, the neuron model and activation functions **TOP 8/41**, Hebbian learning **HOT 4/41**.
- **The examiner pairs across sub-topics**, so revise them together: *machine vision + back-propagation* (71 Ma, 72 Ash), *NLP challenges + machine vision* (79 Bh), *declarative Vs procedural + expert system architecture* (70 Ma, 72 Ma, 73 Ma).
- **Chapter 6's neural-network syllabus line is answered here.** The syllabus files neural networks under Machine Learning, but every paper sets them under Applications of AI, so perceptron, back-propagation, Hopfield, Kohonen and Hebbian learning are all in this chapter.

7.1 The artificial neuron: model, activation function, ANN Vs the brain

What is an ANN? Explain the McCulloch-Pitts model with a diagram. What is the role of the activation function? 4 3+4 4+3 1+7 3+5 8 10

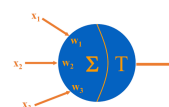
TOP 8/41 71 Bh 3+5, 81 Ch 3+4, 82 Ka 4+3, 75 Ba 8, 76 Ba 10, 80 Bh 2+2+4, 79 Bh 1+7, 69 Bh 4 · 71 Bh and 69 Bh want the brain comparison, 81 Ch and 82 Ka the activation function, 75 Ba and 76 Ba the MP neuron on AND and XOR (worked in the companion), 80 Bh the mathematical model and what training means

Definition and the mathematical model

- An artificial neural network is a parallel computational system of many simple processing elements (neurons), connected in a specific way, that learns a task from data rather than from a program.
- Modelled on the brain: massively parallel, and **tolerant of noise and of the failure of individual units**.
- **One neuron does three things**, and that is the whole model:
 - **1. Weight** each input. w_1x_1 up to w_nx_n . The weight is the strength of that connection, and **the weights are what the network learns**.
 - **2. Sum** them, plus a **bias** b (equivalently, subtract a **threshold** θ):

$$net = \sum_{i=1}^n w_i x_i + b$$

- **3. Apply an activation function** f to that sum: $y = f(net)$.



The neuron fires if the following inequality is true:

$$x_1w_1 + x_2w_2 + x_3w_3 > T$$

The McCulloch-Pitts neuron and its firing condition. Weight, sum, threshold: the same three operations every neuron in this chapter performs.

- The five elements to name if the question says “elements of a neural network”: **inputs, weights, threshold or bias, activation function, learning rate** α .
- What “**training a network**” means (80 Bh 2): presenting examples and adjusting the weights and biases so the output moves towards the target. **Nothing else in the network changes.**
- **Learning rate** α sets the step size. Too large and the weights oscillate and never settle; too small and training takes forever. Typical 0.01 to 0.5.

The McCulloch-Pitts neuron (1943)

the first neuron model, and the one four papers name.

- Output is **binary**: 1 (fires) or 0. Nothing in between.
- Every input is weighted; the neuron **fires when the weighted sum exceeds the threshold** T :

$$x_1w_1 + x_2w_2 + x_3w_3 > T \Rightarrow y = 1$$

- **Excitatory** inputs carry positive weights, **inhibitory** inputs negative ones, and in the original model **one active inhibitory input vetoes the neuron outright.**
- **The weights are fixed by the designer, not learnt.** That is the difference from a perceptron, and it is the sentence the examiner is looking for.
- It can realise **AND, OR and NOT** – so any Boolean function, if you are allowed several layers – but **one MP neuron cannot do XOR.** Both worked in the companion.

The activation function, and how to choose one (81 Ch 3, 82 Ka 3)

- What it is for, three answers, all worth marks:
 - **Decides whether the neuron fires**, by mapping an unbounded sum onto a bounded output.
 - **Introduces non-linearity.** Without it, a stack of layers collapses into one linear map, and the network can only ever learn a straight line – this is the answer to 79 Bh’s “how can non-linearity be modelled through an ANN”.
 - Must be **differentiable** for gradient descent, which is why back-propagation needs sigmoid or tanh, not a step.
- **Choosing one for a project** – answer with the rule, then an example:
 - **Binary classification output** → **sigmoid**, because it gives a value in $(0, 1)$ readable as a probability.
 - **Multi-class output** → **softmax**.
 - **Regression output** → **linear**, so the value is unbounded.
 - **Hidden layers of a deep net** → **ReLU**: cheap, and it does not saturate, so it does not kill the gradient. Its cost is **dead units** – a neuron whose input stays negative has zero gradient and never recovers, which is exactly what happens in the numerical worked in the companion.
 - **Zero-centred hidden layer** → **tanh**, which converges faster than sigmoid for that reason.
 - **Hard threshold** (step, signum) → only for a perceptron or an MP neuron, never where gradients are needed.

Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = \max(0, z)$	Multi-layer Neural Networks	
Rectifier, softplus	$\phi(z) = \ln(1 + e^z)$	Multi-layer Neural Networks	

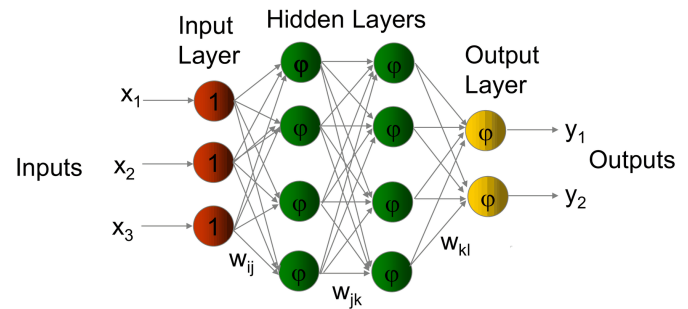
Copyright © Sebastian Raschka 2016
(<http://sebastianraschka.com>)

ANN Vs the human brain (71 Bh 5, 69 Bh 4)

	Human brain	ANN
Units	about 86 billion neurons	a few tens to a few billion
Connections	about 100 trillion synapses	set by the architecture
Speed	slow: a neuron fires at a few hundred Hz	fast: gigahertz
Processing	massively parallel , all at once	parallel in principle, usually serial on one machine
Memory	about 2.5 petabytes, content addressable	limited, address based
Fault tolerance	very high : neurons die daily and nothing is lost	high within a net, none if the machine fails
Learning	continuous, from few examples	needs many examples and retraining
Power	about 20 W	kilowatts to train

– **The contrast worth quoting**: for vision the brain is worth a thousand supercomputers; for arithmetic a pocket calculator is worth ten brains. Structure, not speed, is what differs.

Network structures



- **Feed-forward** – signals travel one way, input to output, with **no loops**. A layer's output never affects that same layer.
 - **Single layer**: input straight to output. Only linearly separable problems (§7.3).
 - **Multilayer (MLP)**: one or more **hidden layers** between them. **One hidden layer is enough to represent any Boolean function**, and with enough units, any continuous function.
- **Feedback (recurrent)** – loops allowed, so the network has **state, and therefore memory**. More powerful, harder to train. Hopfield (§7.5) is the standard example.
- **Competitive** – output units compete and only the winner fires. Kohonen (§7.6).

7.2 Perceptron, linear separability and the XOR problem

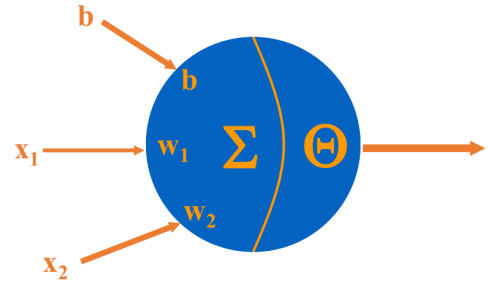
What is a perceptron? Design a neural network that acts as an AND / OR / XOR gate. 4 6 7 10 1+7
2+6 1+8

TOP 17/41 72 Ma 2+7, 73 Ma 1+7, 74 Ma 10, 74 Bh 1+8, 82 Bh 2+6, 79 Ch 2+6, 79 Ash 1+8, 77 Ch 2+6, 79 Jth 1+6, 73 Bh 2+6, 78 Po 2+2+4, 77 Ch 2+6, 72 Ma 7, 76 Bh 6, 81 Bh 2+6, 69 Po 4, 72 Ash 2+4+4 · second most-asked question in the subject, and eight of the seventeen name XOR specifically · all four gates are worked in the companion

The perceptron

- A perceptron is a single artificial neuron with adjustable weights, a bias and a threshold activation, trained by an iterative rule that is guaranteed to converge when the classes are linearly separable. Rosenblatt, 1958.
- The difference from a McCulloch-Pitts neuron: the MP neuron's weights are chosen by hand, the perceptron's are *learnt*.
- Activation, with threshold θ :

$$f(\text{net}) = \begin{cases} 1 & \text{net} > \theta \\ 0 & -\theta \leq \text{net} \leq \theta \\ -1 & \text{net} < -\theta \end{cases}$$



- The perceptron learning rule. For each training pair, compute y ; if $y \neq t$, update, otherwise change nothing:

$$w_i(\text{new}) = w_i(\text{old}) + \alpha t x_i \quad b(\text{new}) = b(\text{old}) + \alpha t$$

- Weights change only when there is an error. That single sentence separates the perceptron rule from Hebbian learning (§7.4), which updates on every pattern.
- Convergence theorem: if a separating line exists, the rule finds one in a finite number of steps, whatever the starting weights.

Linear separability, and why it decides everything

- A single neuron computes $w_1x_1 + w_2x_2 + b$ and thresholds it. The set of inputs where that sum is zero is a straight line (a hyperplane in n dimensions). Everything on one side outputs 1, everything on the other outputs 0.
- So a perceptron can learn a function if and only if one straight line separates its 1s from its 0s. That is the whole of 81 Bh's "perceptron works as a linear classifier".
- Plot the four corners of the unit square:
 - AND – only (1, 1) is 1. One line cuts it off. ✓
 - OR – only (0, 0) is 0. One line cuts it off. ✓
 - XOR – the two 1s sit at opposite corners, with the two 0s at the other opposite corners. No straight line separates them. ✗
- Draw that square. Marks are given for the picture, and it takes ten seconds.

Limitations (76 Bh 6 asks precisely this)

- Learns only linearly separable classes.
- If the data is not separable the algorithm does not converge and its behaviour is not defined – it cycles.
- If it is separable there are usually many valid answers, and the rule gives no way to tell whether the one it found is a good one (no margin, no confidence). That is what SVMs later fixed.

The algebraic proof, which is what a "justify" question wants

- Suppose weights w_1, w_2 and bias b realise XOR with the rule output 1 iff $w_1x_1 + w_2x_2 + b > 0$. The four rows give:
 - (0, 0) → 0: $b \leq 0$ (i)
 - (0, 1) → 1: $w_2 + b > 0$ (ii)
 - (1, 0) → 1: $w_1 + b > 0$ (iii)
 - (1, 1) → 0: $w_1 + w_2 + b \leq 0$ (iv)
- Add (ii) and (iii): $w_1 + w_2 + 2b > 0$, so $w_1 + w_2 > -2b$.
- From (iv): $w_1 + w_2 \leq -b$. Together: $-2b < -b$, i.e. $b > 0$, which contradicts (i).
- Therefore no single perceptron can implement XOR. ■
- The fix: a multilayer perceptron. One hidden layer computes OR and NAND, and the output neuron ANDs them, so the two hidden units bend the decision boundary into two lines. Built and checked row by row in the companion.

7.3 Learning in a neural network: Hebbian, delta and back-propagation

What is the back-propagation algorithm? When do we need it? Explain with an example. 4 2+2+4 3+5

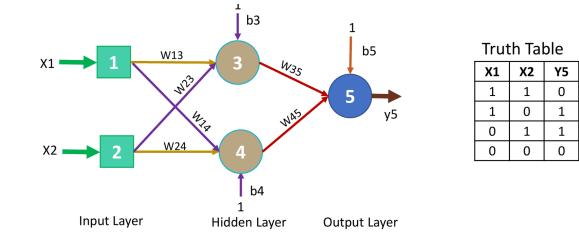
4+4 2+6 2+4+4

TOP 8/41 70 Bh 4+4, 72 Ash 2+4+4, 70 Ma 3+5, 80 Bh 2+2+4, 74 Bh 4, 79 Ch 2+6, 71 Ma 3+5, 72 Ash 3+5 · 71 Ma and 72 Ash attach it to machine vision (§7.9), so revise the two together · one full step is computed in the companion

The three learning rules, in one place

Rule	Update	Updates when
Hebb	$w_i += x_i t$	every pattern
Perceptron	$w_i += \alpha t x_i$	only on an error
Delta (LMS, Widrow-Hoff)	$w_i += \alpha (t - y_{in}) x_i$	every pattern, by how wrong it was

- The delta rule uses the *net input* y_{in} , not the thresholded output, so it measures the size of the error and not just its sign. That is what makes it differentiable, and therefore what back-propagation generalises.
- Adaline** (Adaptive Linear Neuron, Widrow and Hoff 1960) is a single unit trained by the delta rule: identity activation while training, threshold afterwards. **On the syllabus, never yet examined**, so know it in four lines and no more.



Back-propagation

- Short for “backward propagation of errors”: a supervised algorithm that trains a **multilayer** network by gradient descent, pushing the output error backwards through the layers to get a gradient for every weight.
- When do we need it** (79 Ch 2): as soon as there is a **hidden layer**. A hidden unit has no target of its own, so the perceptron rule cannot say what its error is. Back-propagation manufactures one from the chain rule. No hidden layer, no XOR (§7.2), so no useful network.
- Why sigmoid:** $f(x) = \frac{1}{1+e^{-x}}$ has the derivative $f'(x) = f(x)(1 - f(x))$, so the forward pass already computed everything the backward pass needs. That is the only reason it is the default in every textbook example.

The back-propagation algorithm

- Six steps:**
 1. Initialise all weights and biases to small random values, pick η .
 2. **Forward pass:** layer by layer, compute $net_j = \sum w_{ij}x_i + b_j$ and $h_j = f(net_j)$ to the output y .
 3. **Error at the output,** $E = \frac{1}{2}(y - t)^2$.
 4. **Backward pass:** output delta $\delta_{out} = (y - t)y(1 - y)$; hidden delta $\delta_j = h_j(1 - h_j)\sum_k \delta_k w_{jk}$.
 5. **Update every weight,** $w \leftarrow w - \eta \delta$ (input to that weight), and every bias, $b \leftarrow b - \eta \delta$.
 6. Repeat over the training set until E is small enough or the epoch limit is reached.
- Problems to name:** slow; **gets stuck in local minima**; **vanishing gradient** through many sigmoid layers; sensitive to η and to the initial weights; can overfit.

Hebbian learning HOT 4/41 73 Bh 3+7, 81 Ba 2+6, 78 Ba 1+7, 78 Ch 6+3

- Hebb's postulate, 1949: *when cell A repeatedly takes part in firing cell B, the connection from A to B is strengthened*. Usually quoted as “**neurons that fire together wire together**”.
- Algorithm** (four steps, and this is the whole answer):
 - Step 0. Set all weights and the bias to 0.
 - Step 1. For a training pair (s, t) , set the inputs $x_i = s_i$.
 - Step 2. Set the output to the target, $y = t$.
 - Step 3. $w_i(new) = w_i(old) + x_i y$ and $b(new) = b(old) + y$.
 - Repeat for every pair. **One pass. No iteration, no learning rate, no error term.**
- Is Hebbian learning supervised?** (78 Ch 3) – **yes, as used here:** the update needs the target t , so the network is told the right answer. Hebb's biological postulate is unsupervised; the *Hebb net* for pattern association is not. Say both halves.
- It needs **bipolar** data. With binary inputs a 0 contributes nothing to any weight, so the AND net cannot be learnt at all – a real trap, and the reason every worked example uses ± 1 .

7.4 Hopfield network

What is a Hopfield network? Explain its features and all its steps with an example. 3 4 6 8 3+5

TOP 8/41 69 Bh 8, 75 Bh 8, 71 Bh 8, 80 Ash 3+5, 82 Bh 6, 80 Ch 4, 82 Ba 3, 71 Ma 3 · 82 Bh wants the weight matrix and a recall computed, which is the companion · 80 Ash and 71 Ma want it compared — with a feed-forward net and with Kohonen

- A Hopfield network is a single-layer, fully connected, recurrent network of binary units with symmetric weights, which settles from any starting state into the nearest stored pattern. Hopfield, 1982.
- **Structure**, and every word of it earns a mark:
 - One layer. Every unit is connected to **every other** unit, and each unit's output feeds back as an input.
 - Weights are **symmetric**, $w_{ij} = w_{ji}$, with **no self connection**, $w_{ii} = 0$.
 - Units are **binary** $\{0, 1\}$ or **bipolar** $\{-1, +1\}$.
 - **There is no separate output layer**: the same units take the input and hold the answer.
- **Hopfield Vs feed-forward** (80 Ash 5): recurrent Vs one-way; symmetric weights Vs arbitrary; one layer Vs many; associative memory Vs function approximation; converges to a stored pattern Vs computes an output in one pass; trained in **one shot** Vs trained by many epochs of back-propagation.

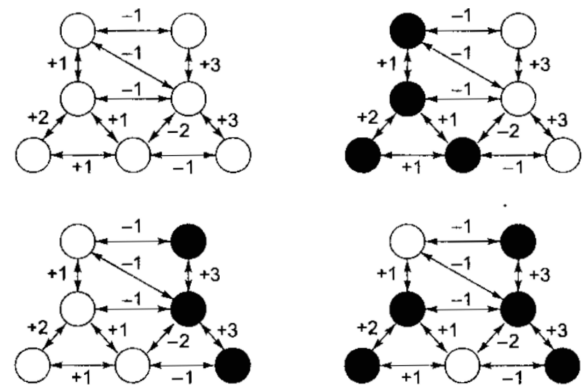


Fig. 18.2 The Four Stable States of a Particular Hopfield Net

Four stable states of one small net. Supply any part of one and the network falls into it.

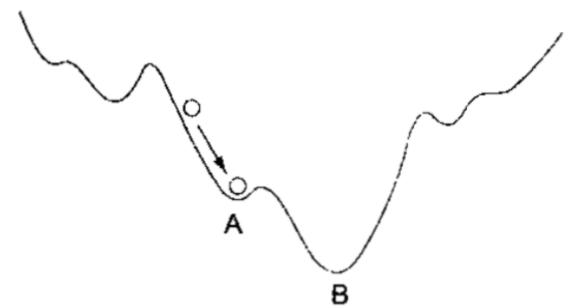


Fig. 18.3 A Hopfield Net as a Model of Content-Addressable Memory

Every update walks downhill on this surface, so the network always stops somewhere. The valleys are the stored patterns.

- **The four features to list** (71 Bh asks for them by name):
 - **Content-addressable memory** – retrieve a whole pattern by supplying *part* of it. This is the headline property.
 - **Distributed representation** – a pattern is stored in all the weights at once, not in one cell.
 - **Asynchronous control** – units update one at a time, in any order, with no central clock.
 - **Fault tolerance** – damage a few weights and recall degrades gracefully rather than failing.
- **Energy.** Each state has an energy

$$E = -\frac{1}{2} \sum_{i \neq j} w_{ij} s_i s_j - \sum_i \theta_i s_i$$

and every update either lowers it or leaves it alone. So the network **always converges**, to a local minimum of E . The stored patterns are those minima.

- **Capacity:** about $p \leq 0.15N$ patterns for N units. Store more and the patterns interfere; the net then settles into **spurious states** that were never taught.

The algorithm

- **Training** (one pass, no iteration). For P bipolar patterns:

$$w_{ij} = \sum_{p=1}^P s_i^{(p)} s_j^{(p)} \quad (i \neq j), \quad w_{ii} = 0$$

i.e. sum the outer products and zero the diagonal.

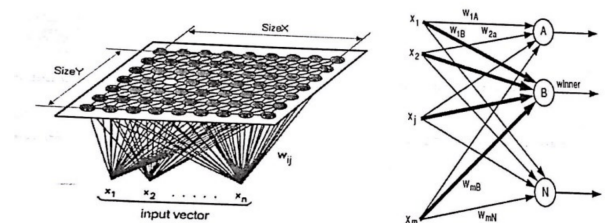
- **Recall / testing**, given a noisy input x :
 - **1.** Set the state $y = x$.
 - **2.** Pick one unit i at random and compute $y_{in,i} = x_i + \sum_j y_j w_{ji}$.
 - **3.** Apply the threshold: $y_i = 1$ if $y_{in,i} > \theta_i$, unchanged if equal, 0 (or -1) if less.
 - **4. Broadcast the new y_i immediately** – the next unit uses it. This is what *asynchronous* means, and updating all units at once is the commonest mistake.
 - **5.** Repeat until no unit changes. The state is then stable.
- **Worked in full in the companion:** three patterns stored, the weight matrix built, and a corrupted pattern recalled unit by unit.

7.5 Kohonen network and the self-organizing map

What is a self-organizing map? Explain all the steps involved with an example. 3 4 5 2+5

TOP 9/41 74 Bh 2+5, 80 Bh 4, 79 Bh 4, 75 Bh 4, 74 Ma 4, 72 Ash 4, 79 Jth 5, 81 Bh 3, 71 Ma 3 · seven of the nine ask it as a short note, so the answer is a definition, the algorithm as five steps, and one application · 71 Ma wants it compared with Hopfield

- A self-organizing map (Kohonen network) is an unsupervised, competitive neural network that maps high-dimensional input onto a low-dimensional grid of neurons while preserving the topology of the data. Kohonen, 1982.
- **Structure:** an input layer, and **a grid of output neurons** (usually 2-D). Fully connected between them and **nothing else** – no hidden layer, no bias, no activation function.
- **Competitive learning:** for each input, exactly one output neuron **wins**, the one whose weight vector is nearest the input. It and its *neighbours* are pulled towards that input; everyone else is left alone.
- **Topology preserving** is the property to name: inputs that are similar end up firing neurons that are close together on the grid. That is what makes an SOM a visualisation tool and not just a clustering algorithm.
- **Used for:** clustering, dimensionality reduction, data visualisation (the poverty map is the classic), feature extraction, image compression.



The grid of neurons, and the same net drawn as connections from the input vector to every output neuron with one marked winner.

Hopfield Vs Kohonen (71 Ma 3)

	Hopfield	Kohonen
Learning	supervised (patterns given)	unsupervised
Structure	one layer, fully recurrent	input layer → output grid, feed-forward
Weights	symmetric, $w_{ii} = 0$	one vector per output neuron
Rule	Hebbian outer product, one shot	competitive, winner takes most
Output	a stored pattern	the winning neuron's position
Used for	associative memory, optimization	clustering, visualisation

– ✓ Easy to interpret; handles large, complex data. × Computationally expensive; the grid size and the initial weights are guesses; it assumes nearby points should behave alike.

The algorithm, five steps

- **Step 1. Initialise** every neuron's weight vector w_j randomly, and set the learning rate α and the neighbourhood radius σ .
- **Step 2. Present** an input vector x .
- **Step 3. Find the winner** (the Best Matching Unit), the neuron whose weight vector is nearest:

$$D(j) = \sum_i (x_i - w_{ij})^2, \quad j^* = \arg \min_j D(j)$$

the *squared* Euclidean distance is enough — the square root does not change which one is smallest, and skipping it saves time in an exam.

- **Step 4. Update the winner and its neighbours**, moving them towards the input:

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha (x_i - w_{ij}(\text{old}))$$

for every j inside the neighbourhood of j^* .

- **Step 5. Shrink** α and σ , and repeat from step 2 until they reach their floor or the map stops moving.
- **Two phases worth naming:** an early **ordering** phase with a large radius that lays the map out, and a long **convergence** phase with a small radius that fine-tunes it.

7.6 Expert systems: what they are, and the architecture

What is an expert system? Explain its general architecture with a block diagram. 4 6 8 2+4 1+7 2+5 3+5

TOP 17/41 69 Po 8, 71 Ma 2+4, 76 Ba 1+7, 71 Bh 8, 80 Ba 2+5, 81 Ba 3+5, 77 Ch 6+2, 78 Ch 1+4+3, 76 Bh 5+2+2, 70 Ma 2+6, 72 Ma 3+6, 73 Ma 3+5, 75 Ba 2+6, 80 Ch 6, 82 Ba 8, 71 Bh 2+6, 74 Bh 2+5 · **joint most-asked question in the subject** · three papers dress it as a scenario — *disease diagnosis* (80 Ch), *a disaster-prone area of Nepal* (82 Ba), *create a scenario where we require one* (79 Ch) — and want the same block diagram with the domain renamed

Definition

- An expert system is a computer program that simulates the decision-making of a human expert in one narrow domain, using a body of high-quality specific knowledge rather than a general algorithm.
- **Narrow is the operative word.** An expert system is expert at one thing and useless at everything else.
- Knowledge comes from **human specialists**, and also from texts, journals and databases.

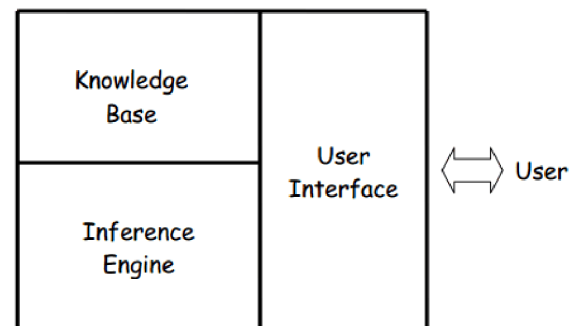


Fig: Block Diagram of expert system

Draw this. Four boxes and one double-headed arrow to the user is a full answer to the block-diagram half of every one of the seventeen papers.

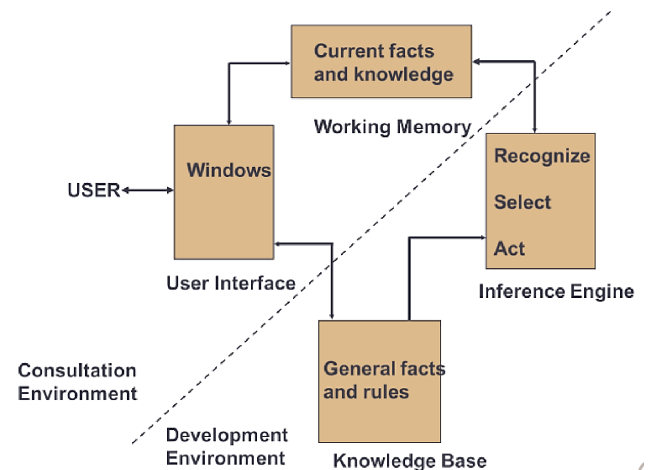
- **Expert system shell** (82 Ka 2): an expert system with the **knowledge base emptied out** – inference engine, user interface and explanation facility, ready for a new domain's rules. It is what turns building an expert system into writing rules instead of writing code. Eg EMYCIN, the shell left when MYCIN's medical rules were removed.
- **When is a problem suited to one** (82 Ka 3): the domain is **narrow and well bounded**; genuine **human experts exist** and can explain their reasoning; the task is **symbolic and heuristic**, not numerical; it takes an expert minutes to hours, not seconds or months; and no ordinary algorithm solves it.

How the inference engine works (81 Ba 5, 82 Bh 5)

- It applies one rule of logic, **modus ponens**: if $A \rightarrow B$ is in the knowledge base and A is in working memory, add B .
- **Two directions**, both from Chapter 4:
 - **Forward chaining (data driven)** – start from the facts, fire whatever matches, add the conclusions, repeat. Use it when there are few facts and many possible conclusions. *Monitoring, planning, design.*
 - **Backward chaining (goal driven)** – start from a hypothesis and look for rules that would prove it, asking the user for what is missing. Use it when there are few plausible conclusions. *Diagnosis.* MYCIN works this way, and it is why the system asks you questions.
- **The recognize-select-act cycle**, which is the answer if the question says *how is inference performed*:
 - **1. Match** – compare every rule's condition against working memory, producing a **conflict set** of rules that could fire.
 - **2. Conflict resolution** – choose *one* of them, by specificity, recency, rule priority or a **meta-rule**.
 - **3. Act** – fire it, updating working memory. Repeat until the goal is reached or nothing matches.
- **Rule types to name**: **relationship** (*IF the battery is dead THEN the car will not start*), **recommendation** (*... THEN take a cab*), **directive** (*... AND the fuel system is ok THEN check the electrical system*), **heuristic**, and **meta-rules**, which are rules about which rules to use.

The components

- **Knowledge base** – the domain knowledge, almost always as **IF-THEN production rules**: *IF the battery is dead THEN the car will not start*. This is what makes it an expert system rather than a program.
- **Working memory** – the facts about *this* case, built up during the consultation. Sometimes called the database or the fact base.
- **Inference engine** – matches rules against working memory and fires them. Also called the control structure or rule interpreter.
- **User interface** – two-way: it takes the user's queries and answers, and it presents conclusions.
- **Explanation facility** – answers *why are you asking* and *how did you conclude that*. **Name it: it is the one component most answers forget**, and it is what makes an expert system acceptable to a professional.
- **Knowledge acquisition subsystem** – how new rules get in (§7.7).



The fuller picture, worth drawing when the question is worth 6 or more: the dashed line splits the consultation environment the end user sees from the development environment the knowledge engineer works in.

Two real systems to name

- **MYCIN** (Stanford, 1970s) – diagnoses blood infections and recommends antibiotics from about **500 production rules**. Uses **backward chaining**, and will explain its reasoning on request. Matched human specialists.
- **DENDRAL** (Stanford, late 1960s) – the **first** expert system. Infers molecular structure from mass spectra using chemists' heuristics. **Forward chaining**.
- Others worth one word each: **XCON** (configuring VAX computers), **PROSPECTOR** (mineral exploration), **CADUCEUS** (internal medicine).

7.7 Expert system development, and knowledge acquisition

Explain the stages of expert system development, with an example. 6 8 2+5 2+6 4+4

TOP 8/41 69 Bh 8, 70 Bh 4+4, 73 Bh 2+6, 75 Ba 2+6, 79 Ch 2+6, 80 Ch 6, 82 Ba 8, 80 Ba 2+5 · always paired with the block diagram, so answer both halves · 80 Ch casts you as the knowledge engineer for a disease-diagnosis system, 82 Ba in a disaster-prone area of Nepal

The six stages

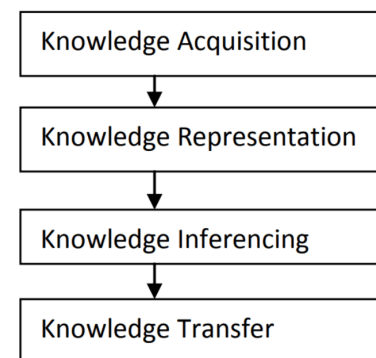
- **1. Identify the problem domain.** Is the problem suitable at all? Are there real experts, and will they cooperate? Is it cost-effective?
- **2. Design the system.** Choose the technology and the shell; decide how the domain knowledge will be *represented*; settle the integration with existing databases.
- **3. Develop the prototype.** The **knowledge engineer** acquires the domain knowledge from the expert and encodes it as IF-THEN rules. This is the long stage.
- **4. Test and refine.** The knowledge engineer runs sample cases; end users try the prototype; rules are corrected and added. **Iterate 3 and 4 until it performs like the expert.**
- **5. Develop and complete.** Integrate with the real environment, document it, train the users.
- **6. Maintain.** Keep the knowledge base current as the domain changes, and keep the interfaces working as the surrounding systems evolve.

Answering the scenario version

- **Name the domain, the expert and three real rules.** For 80 Ch's disease diagnosis: expert = a physician; rules like *IF fever > 102 °F AND rash AND joint pain THEN suspect dengue, certainty 0.8*; working memory = this patient's symptoms; backward chaining, because you are testing a hypothesis.
- Then draw the block diagram with the boxes *relabelled for that domain*, and walk the six stages in one line each. That is a full 8 marks.

Knowledge acquisition PIN 2/41 71 Bh 2+6, 74 Bh 2+5

- Knowledge acquisition is the process of extracting knowledge from human experts and other sources and encoding it into the knowledge base.
- **It is the bottleneck of the whole field.** Experts know more than they can say, disagree with each other, and are expensive; and what they do say has to be turned into rules by someone who is not an expert.
- **The methods:** structured and unstructured interviews; observation of the expert working; questionnaires; protocol analysis (the expert thinks aloud); case studies; and automatic acquisition, i.e. machine learning from records (Ch 6), which is what removes the bottleneck when data exists.
- **Sources:** the expert (primary), then secondary and tertiary experts, then literature – reports, guidelines, books, manuals – then end users.
- **The four kinds of knowledge to name:** declarative (facts), procedural (how to do it), episodic (remembered cases), meta-knowledge (knowledge about the knowledge).
- **Example to give:** interviewing a cardiologist about how she reads an ECG, recording each judgement as a rule, then replaying old cases to see where the rules disagree with her.



A second, coarser framing of the same thing, and the one BA Sir's deck gives. It is not a rival to the six stages: acquisition and representation are stage 3, inferencing is stage 4, transfer is stage 5. · **Either list is accepted.** Give the six stages, and draw this beside them if the question says "with a diagram" – it is the only flow in the answer, the block diagram in §7.6 being a *structure*.

7.8 Expert systems: characteristics, advantages, limits, and Vs human experts

What are the characteristics of an expert system? Compare it with a human expert. Give its advantages and disadvantages. 3 4 7 8 2+2+3

TOP 11/41 72 Ash 8, 81 Ch 4+3, 80 Ash 8, 81 Ash 4, 79 Bh 4, 78 Ch 1+4+3, 76 Bh 5+2+2, 77 Ch 6+2, 82 Ka 2+2+3, 80 Ba 2+5, 82 Bh 2+5 · 80 Ash 8 asks for the comparison alone, with practical examples for each point, so the table below is that answer

Pro Expert system	Human expert	Expert system
	Perishable(decay)	Permanent
	Slow processing	Fast processing
	Unpredictable	Consistent
	Slow reproduction	Quick replication
	Expensive	Affordable
Pro Human	Broad focus	Narrow focus
	Inspiration	Lacks creative ability
	Adaptive needs	Instruction
	Common sense	Machine knowledge

Which side wins each point. The top block is where the machine is better, the bottom block where the person is.

Characteristics (80 Ba 2, 82 Ka 2)

- **High performance** in its narrow domain, at expert level.
- **Understandable** – takes input and gives output in the user's language.
- **Reliable and highly responsive.**
- **Explains itself**, and separates **knowledge from the program that uses it**, so the knowledge can be edited without touching the code.
- Uses **symbolic** representation and **heuristics**, not an algorithm.
- Handles **uncertainty**, with certainty factors or probabilities (Ch 4).

Advantages

- **Permanent**: it does not retire, forget, or die. A human expert's knowledge leaves with them.
- **Reproducible** – copy it to a hundred sites at no extra cost, so scarce expertise reaches places that have none. This is the argument for 82 Ba's disaster scenario.
- **Consistent**: same input, same answer, every time. A tired human is not.
- **Fast**, available **24 hours**, and **cheaper** than the expert.
- Works in **hazardous environments**; keeps a full audit trail; and is a **training tool**, since it explains its reasoning.

Disadvantages and limitations

- **No common sense**. It has knowledge, not understanding, and will give a confidently absurd answer just outside its domain.
- **Narrow**: cannot transfer anything it knows to a neighbouring problem.
- **Cannot learn from experience** on its own – every new rule has to be put in by a knowledge engineer.
- No **creativity** or intuition, and no ability to recognise that the case in front of it is unusual.
- **Expensive and slow to build**, because of the knowledge-acquisition bottleneck; and **only as good as the expert it copied**, errors included.
- Cannot take moral responsibility for a decision, which is why medical expert systems advise rather than decide.

Declarative Vs procedural knowledge HOT 4/41 70 Ma 2+6

, 72 Ma 3+6 , 73 Ma 3+5 , 72 Ma 6

all four ask it *with* the expert system architecture, so the two are one answer.

	Declarative	Procedural
Is	knowing what	knowing how
Content	facts about objects, events, situations	the steps to carry a task out
Form	statements: logic, semantic nets, frames (Ch 4, Ch 5)	rules, programs, procedures
Stored	separately from how it is used	inseparably from the code using it
Changed	easily, one fact at a time	only by rewriting the procedure
Reused	by any inference method	only for its own purpose
Efficiency	lower: control must be worked out at run time	higher: control is built in
Example	<i>Kathmandu is the capital of Nepal;</i> <code>dog(Tommy)</code>	<i>how to ride a bicycle;</i> a sorting routine

In an expert system, both are present (this is the half the papers actually mark)

- The **knowledge base** is **declarative**: it states what is true in the domain and says nothing about how to use it.
- The **inference engine** is **procedural**: it knows how to search and fire rules, and nothing about the domain.
- **That separation is the whole design**, and the reason a shell exists at all: swap the declarative half and the same procedural half serves a new domain.
- An IF–THEN rule sits on the line, which is worth a sentence: it *states* a relationship, so it reads declaratively, but firing it *does* something, so it behaves procedurally. That is why production rules are the standard expert system representation.

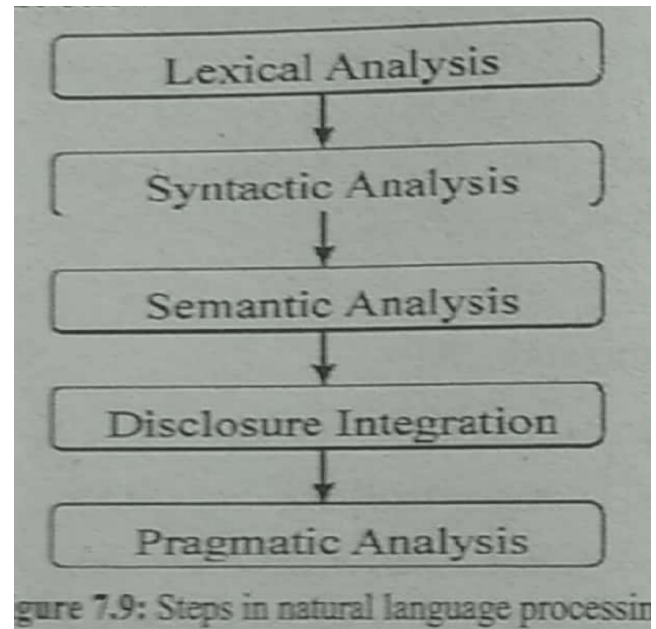
7.9 Natural language processing: NLU, NLG and the five steps

What is NLP? Explain the different steps involved, with a block diagram and examples. 5 6 7 8 9 2+6 3+5

TOP 23/41 69 Bh 6, 69 Po 8, 73 Ma 2+4, 79 Ch 3+5, 72 Ash 2+6, 75 Ba 8, 72 Ma 8, 73 Bh 7, 73 Bh 2+7, 74 Ma 9, 74 Bh 2+6, 82 Ba 2+6, 79 Jth 1+6, 82 Ka 2+5, 81 Ch 2+5, 77 Ch 2+6, 78 Po 3+5, 79 Ash 5+2, 75 Bh 6+2+2, 76 Bh 5+4, 78 Ba 2+6, 78 Ch 2+4+2, 80 Ch 2+6 · the most-asked question in the whole subject, in 23 of the 41 papers, and it has been set at least once in every year from 2069 to 2082 · eight of the twenty-three also want NLU and NLG defined · 80 Ch asks it as *build a Nepali chatbot*

What NLP is

- Natural language processing is the branch of AI that analyses written or spoken human language and converts it into a representation a computer can use, and back again.
- Two halves, and the papers ask for both by name:
 - NLU, natural language understanding – text in, meaning out. Maps the input to a useful representation: entities, concepts, keywords, relations, sentiment, semantic roles. **This is the hard half.**
 - NLG, natural language generation – meaning in, text out. Its own three stages: **text planning** (what to say), **sentence planning** (how to phrase and combine it), **text realisation** (produce grammatical output).
- Input is text or speech, and **the quality of the input bounds everything downstream**: a bad transcription cannot be recovered by a good parser.
- Applications to name: machine translation, chatbots and virtual assistants, sentiment analysis, information retrieval and search, spam filtering, text summarisation, question answering, speech recognition, spelling and grammar correction, optical character recognition.



Draw these five boxes in this order. It is the single most valuable minute of drawing in the subject. (The published figure prints *Disclosure Integration*; the term is **Discourse Integration** – a misprint in the book, not a different step.)

The five steps, each with the example to give

- **1. Lexical / morphological analysis.** Break the text into paragraphs, sentences and **tokens**, and analyse the structure of each word.
 - “I love NLP, it’s fascinating!” →
[I] [love] [NLP] [,] [it’s] [fascinating] [!]
 - Includes **stemming** (rule-based chopping: *closed, closing, closer* → *clos*) and **lemmatization** (dictionary-based: *better* → *good*). Name the difference; it is a reliable extra mark.
 - Morphology: *unhappily* = *un-* + *happy* + *-ly*.
- **2. Syntactic analysis (parsing).** Check the words against the grammar and build a **parse tree** showing how they combine.
 - “The school goes to boy” is **rejected here** – every word is a real word, but the arrangement is not English.
 - Includes **part-of-speech tagging**: *book* is a noun in *the book on the table* and a verb in *to book a flight*.
 - A worked parse tree is in the companion.

The four levels, and the ambiguity at each TOP 8/41 71 Ma 2+2+2+2

, 71 Bh 8 , 72 Ma 6 , 78 Ba 2+6 , 72 Ash 1+7 , 82 Bh 3+4 , 80 Bh 3+5 , 80 Ba 2+5

eight papers ask for the *levels* rather than the steps, usually with “give an example of ambiguity at each”. Same material, different cut: give one example per level and you have the marks.

Level	Concerns	An ambiguity
Phonetic	sounds → words	“ice cream” against “I scream”; “grey day” against “grade A”
Lexical	which sense of a word	<i>bank</i> : riverbank or institution
Syntactic	which grammatical structure	“I saw the man with a telescope”: who has the telescope?
Semantic	what the sentence means	“The chicken is ready to eat”: is it the eater or the meal?
Pragmatic	what the speaker intended	“Can you open the window?”: request, not a question

Steps 3 to 5

- **3. Semantic analysis.** Turn the parse into meaning.
 - **Word sense disambiguation**: *bank* the riverbank or *bank* the institution; *Mary had a bat in her office*.
 - Sentence-level meaning: “Colourless green ideas sleep furiously” is perfect syntax and **rejected here**, which is the clean example of what step 3 does that step 2 cannot.
- **4. Discourse integration.** The meaning of a sentence depends on the sentences around it.
 - “The thieves stole the paintings. They were subsequently sold / caught / found.” **One word changes who “they” is.**
 - “John wanted it” means nothing without the previous sentence.
- **5. Pragmatic analysis.** What was actually *meant*, using real-world knowledge.
 - “Can you pass the salt?” is a **request**, not a question about your arm. Answering yes is grammatically perfect and pragmatically wrong.
 - Covers sarcasm, idiom and implied meaning.
- **Phonetic is the level every deck skips**, and six papers list it. It is the level at which *sound* is turned into *words*, and its ambiguity is that one stream of sound splits into words in more than one way.
- **Syntactic and semantic analysis compared** (80 Bh, 80 Ba, 82 Bh ask exactly this): syntax asks *is this arrangement legal, and what is its structure* and answers with a parse tree; semantics asks *what does that structure mean* and answers with a meaning representation. A sentence can pass one and fail the other, in both directions.

7.10 Issues and challenges in NLP, and machine translation

What are the problems associated with NLP? Why is it hard? 2 3 5 8 1+7

TOP 13/41 70 Bh 1+7, 70 Ma 2+6, 78 Ch 2+4+2, 82 Ka 2+5, 81 Ch 2+5, 77 Ch 2+6, 78 Po 3+5, 79 Ash 5+2, 75 Bh 6+2+2, 79 Bh 2+5, 81 Ash 8, 80 Bh 3+5, 80 Ba 2+5 · almost always the 2 or 3 opener to the steps question · 81 Ash 8 and 80 Ch 2+6 set it as a Nepali-language problem and want the issues related to Nepali

Why natural language is hard

- **Ambiguity at every level** – §7.9's table. This is the root cause and should be your first sentence.
- **Context dependence**: “Where’s the water?” means something different in a chemistry lab, when you are thirsty, and under a leaking roof.
- **Many ways to say one thing**: “Ram’s birthday is October 11” and “Ram was born on October 11”.
- **The language will not stay still**: new words, new senses, slang. No lexicon is ever complete.
- **Hidden and figurative meaning**: idiom, metaphor, sarcasm.
- **Pronouns and reference**: “Ravi went to the supermarket. He found his favourite coffee. He paid for it and left.” What is it?
- **Ungrammatical and incomplete input**: “He rice eats.” People still understand it; a parser does not.
- **Ellipsis and conjunction**: “Ram and Hari went to a restaurant. Ram had coffee, Hari tea.” – the second verb is simply missing.
- **Speech adds its own**: accents, homophones, and **no spaces between spoken words**.

Machine translation (70 Bh 1+7)

- **Machine translation is the automatic conversion of text or speech from one natural language into another, preserving meaning.**
- **Three approaches**, in historical order:
 - **Rule based** – dictionaries plus hand-written grammar rules. Accurate where the rules cover, brittle everywhere else.
 - **Statistical** – learn word and phrase probabilities from a **parallel corpus**. Needs vast aligned data.
 - **Neural** – an encoder-decoder network trained end to end. The current standard, and the reason translation improved suddenly after 2016.
- **Its challenges**: word-sense ambiguity; idioms that must not be translated literally; different **word order** (English SVO, Nepali SOV); grammatical gender and honorifics with no counterpart in the other language; named entities and untranslatable culture-specific words; and no parallel corpus for low-resource pairs.

The Nepali version of the answer (81 Ash 8, 80 Ch 6)

- **Low-resource language**: little annotated text, no large treebank, few parallel corpora. Everything below follows from this.
- **Devanagari script**: transliteration and code-mixing (Nepali typed in Roman letters), inconsistent Unicode encoding, no standard spelling for many loanwords.
- **Morphologically rich**: heavy inflection and agglutination, so one root yields dozens of surface forms and a lexicon lookup fails.
- **Honorifics and levels of respect** (*timi / tapai / hajur*) change the verb, and carry meaning a translation must preserve.
- **Free word order**, and **subject-object-verb**, so English-trained parsers transfer badly.
- **Dialects** and heavy loan vocabulary from English and Hindi.
- **Mitigation, which is the half that carries the marks**: build and release an annotated corpus; use transfer learning from a multilingual model rather than training from scratch; augment data by back-translation; write a morphological analyser for the inflections; standardise the encoding; keep a human in the loop for evaluation.

Tools NLP needs (the “knowledge bases required”, 80 Ch 2)

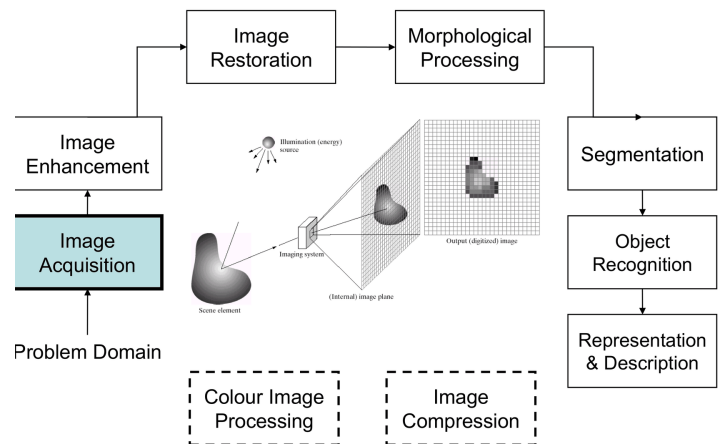
- **Lexicon / dictionary** – the vocabulary, with meanings, pronunciations and syntactic categories.
- **Morphological analyser** – splits words into prefix, root and suffix and derives their grammatical features.
- **Grammar** – the rules that say which arrangements are legal, used by the parser.
- **Semantic and world knowledge** – an ontology or knowledge base (Ch 5) that says what the words refer to and what is plausible.
- **Annotated corpus** – tagged and parsed text to train and to test on.
- **Discourse and pragmatic model** – to resolve pronouns and intent.

7.11 Machine vision

What is machine vision? Why is it important in AI? List its steps and applications. 3 4 2+4 2+5 2+6 3+5
1+5+2

TOP 12/41 71 Ma 3+5, 72 Ash 3+5, 80 Ash 2+6, 81 Ash 2+4, 81 Bh 2+5, 79 Ash 1+5+2, 69 Bh 4, 80 Ch 4, 80 Ba 3, 82 Ba 3, 79 Bh 2+5, 70 Bh 2+6 · two papers pair it with back-propagation (71 Ma, 72 Ash) and one with GA (70 Bh), so it is often the other half of a neural-network question · 80 Ash wants a scenario and the steps to solve it

- Machine vision is the field of AI that builds a model of the real world from images – recovering useful information about a three-dimensional scene from its two-dimensional projections.
- The hard part in one line:** the world is 3-D and the image is 2-D, so information has been thrown away and must be put back using knowledge about the objects and the projection geometry.
- An image is a 2-D array of intensity values. Each element is a pixel. Grey scale $f = f(x, y)$; colour is three such functions, $\{R(x, y), G(x, y), B(x, y)\}$.



anitha Acharya (aolunthazas@gmail.com)

Why it matters to AI (71 Ma 3, 79 Bh 5)

- Vision is how an agent perceives**, and Chapter 1's agent model begins with sensors. Without it an agent is limited to whatever a human types in.
- It is the only practical channel for **unstructured** information about the physical world: no one can hand-code a description of every scene.
- It is what makes autonomy possible – robots, vehicles, inspection – and it is where neural networks work best, which is why **back-propagation** (§7.3) and machine vision are asked together.

The stages, in order

- 1. Image acquisition** – capture the scene with a camera or sensor and digitise it. Lighting is a design decision, not an accident.
- 2. Image enhancement** – make features easier to see: contrast stretching, sharpening, noise filtering, edge detection.
- 3. Image restoration** – undo a *known* degradation (blur, sensor noise) using a model of how it happened. Enhancement is subjective, restoration is objective; say so.
- 4. Morphological processing** – operate on shape: erosion, dilation, opening, closing, to clean up the regions.
- 5. Segmentation** – **partition the image into objects and background**, grouping pixels that share intensity, colour, texture or motion. The hardest step, and the one everything after depends on.
- 6. Representation and description** – turn each region into **features**: boundary or area, size, shape, moments, texture.
- 7. Object recognition** – assign a label to each region by **pattern recognition**: statistical (feature vectors into a classifier) or structural (decompose into primitives).
- 8. Interpretation** – say what the scene *means*: how many, where, moving how, and what to do about it.
- Colour processing** and **image compression** sit beside the pipeline rather than in it; the block diagram draws them as separate boxes and so should you.

Applications

- **Industry:** automated inspection on a production line, reading part numbers, robot guidance and pick-and-place, barcode and label verification.
- **Medicine:** X-ray, CT and MRI analysis, tumour detection, cell counting.
- **Transport:** self-driving cars, lane and sign detection, number-plate recognition, traffic monitoring.
- **Security:** face recognition, fingerprint and iris biometrics, surveillance, intrusion detection.
- **Agriculture:** crop and disease monitoring, fruit grading, yield estimation.
- **Documents:** OCR, handwriting recognition, cheque processing.
- **Remote sensing and astronomy; AR and VR.**

Answering the scenario version (80 Ash 6, 81 Ash 4)

- **Pick a concrete scenario and walk the eight stages through it.** That is the answer; a generic list of stages is half marks.
- Worked example – **sorting apples on a conveyor:**
 - **Acquire:** overhead camera, diffuse lighting to kill shadows.
 - **Enhance:** median filter to remove sensor speckle.
 - **Restore:** deblur for the belt's motion, which is known and constant.
 - **Morphology:** close small gaps so each apple is one solid region.
 - **Segment:** threshold on colour to separate fruit from belt.
 - **Describe:** for each region take area, roundness, mean hue, and the fraction of dark pixels.
 - **Recognise:** a classifier sorts each into grade A, grade B or reject.
 - **Interpret and act:** fire the diverter arm for the right bin.
- Say what would break it – overlapping apples defeat segmentation, changing daylight defeats the colour threshold – and how you would fix it. That sentence is what separates a 6 from a 4.

7.12 Every Chapter 7 question, grouped by the answer it wants

Bold = regular exam, plain = back. *Monospace* = CT 710 (BEI), *italic* = CT 78506 (BEX), upright = CT 653 (BCT).

- **Steps of NLP 23 papers** → §7.9. Wordings, all one answer: *explain the steps* (69 Po, 73 Ma, 79 Ch, 75 Ba, 72 Ma, 73 Bh); *define NLU and NLG, then the steps* (73 Bh, 74 Ma, 74 Bh, 82 Ba, 79 Jth); *the problems, then the steps* (82 Ka, 81 Ch, 77 Ch, 78 Po); *why is NLP hard* (79 Ash); *steps and a parse tree for "A girl has a name"* (76 Bh, worked in the companion); *a Nepali chatbot* (80 Ch).
- **NLP levels and their ambiguities 8 papers** → §7.9's second half (71 Ma, 71 Bh, 72 Ma, 78 Ba, 72 Ash, 82 Bh, 80 Bh, 80 Ba). Phonetic is the level to make sure you can define.
- **NLP issues, challenges, machine translation 13 papers** → §7.10, including both Nepali-language questions (81 Ash, 80 Ch).
- **Perceptron and the logic gates 17 papers** → §7.2, all worked in the companion. **OR** (72 Ma); **AND** (73 Bh, 78 Po, 77 Ch); **XOR** (73 Ma, 74 Ma, 74 Bh, 82 Bh, 79 Ash, 77 Ch); *any logic gate* (79 Jth); *linear separability* (76 Bh, 81 Bh); *MLP* (72 Ma, 77 Ch); short note (69 Po).
- **Expert system architecture 17 papers** → §7.6. **Development stages 8 papers** → §7.7. **Advantages, limits, Vs human experts 11 papers** → §7.8. **Declarative Vs procedural 4 papers** → §7.8. **Knowledge acquisition 2 papers** → §7.7.
- **Machine vision 12 papers** → §7.11.
- **Kohonen / SOM 9 papers** → §7.5. **Hopfield 8 papers** → §7.4, with the weight matrix and recall worked in the companion for 82 Bh.
- **Back-propagation 8 papers** → §7.3, one full step worked in the companion. **Hebbian learning 4 papers** → §7.3, the AND net worked in the companion.
- **The neuron model, McCulloch-Pitts, activation function 8 papers** → §7.1.
- **Forward propagation through the printed network** (PIN 2/41 82 Ka 8, 81 Ch 8) – the only genuine numerical in the chapter, and **the same question printed twice with the same figure**. Worked in the companion, including why the ReLU layer's weights cannot be updated at all for the given input.
- **Short notes that live here:** *learning in a neural network* (69 Po, 74 Bh, 78 Ba, 77 Ch, 81 Ash), *learning rate and learning based on statistics* (72 Ma), *human brain Vs neural network* (69 Bh).
- **Adaline** – on the syllabus, **never examined in 41 papers**. Four lines in §7.3 and no more.
- **Pattern recognition** – in the textbook's §7.4 but not on the CT 710 syllabus and never asked on its own. It appears here only as step 7 of the machine-vision pipeline.

Source notes

- Spine: BA Sir *CH-07 Applications of AI* (the fullest single source in *Notes* for this chapter), PS Sir 7.1 *Neural Network*, 7.2 *Expert Systems* and 7.3 *NLP and Machine Vision*, and *Insights on Artificial Intelligence* ch 7 for the worked method.
- **One correction carried into the figure caption:** the published NLP figure prints *Disclosure Integration*; the step is **Discourse Integration**. The same misprint runs through several decks.
- **Phonetic-level ambiguity is** verified/added. Six papers ask for the four levels of analysis and no deck in *Notes* explains the phonetic one; the examples here were researched and written.
- **The Nepali-language answers are** verified/added for 81 Ash and 80 Ch: no source covers them, and both questions are worth 6 or 8 marks.
- **Hebbian learning** is not on the syllabus under any name but is asked in four papers, so it is written in full in §7.3 and worked in the companion.

8. Last-Minute Revision

Built from the frequency analysis of all 41 papers, 2069–2082 BS, across CT 653, CT 710 and CT 78506. Nothing here is new: every line points back into a chapter. · **Short on time?** §R.5 is a one-day plan; §R.4 is the fifteen questions to revise if you have an evening.

R.1 Must Know (every TOP-tier topic — these carry the paper)

A topic is TOP tier when 8 or more of the 41 papers ask it. There are 27, and every chapter has at least two, so no chapter is safe to skip.

Ch	Topic	Papers	The one thing to be able to do
4	Resolution refutation	33/41	Negate the goal, convert to CNF, resolve to the empty clause. The single most examined skill
2	Crypt-arithmetic by constraint satisfaction	31/41	Column by column with carries, state the constraints, show the map, verify the sum
7	The steps of NLP, with a block diagram	23/41	Lexical, syntactic, semantic, discourse, pragmatic — five boxes and one example each
6	Genetic algorithm	21/41	Flowchart + selection, crossover, mutation + one worked generation
7	Perceptron; design a net for AND / OR / XOR	17/41	Train the gate in bipolar form; prove XOR needs two layers, then build it
7	Expert system architecture	17/41	Four boxes: knowledge base, working memory, inference engine, user interface. Add the explanation facility
6	Fuzzy logic and fuzzy inference	16/41	Fuzzifier → inference engine + rule base → defuzzifier, with membership curves
4	Bayes' theorem and statistical reasoning	15/41	$P(A B) = P(B A)P(A)/P(B)$, and the denominator by total probability
5	Semantic nets Vs frames, with examples	13/41	One comparison table plus two small drawings; list advantages and limitations
7	Problems and challenges in NLP	13/41	Ambiguity at every level, then context, variation, new words, pronouns
2	What is a CSP? List its constraints	12/41	Variables, domains, constraints, then unary / binary / higher-order
6	Supervised Vs unsupervised learning	12/41	The comparison table, then clustering is unsupervised , naming K-means
7	Machine vision: role, steps, applications	12/41	Acquisition to interpretation, eight stages, walked through one real scenario
4	Forward Vs backward chaining	11/41	Data driven Vs goal driven, one worked example each, when to use which
7	Expert system advantages, limits, Vs human experts	11/41	Permanent, consistent, reproducible against no common sense, cannot learn

Ch	Topic	Papers	The one thing to be able to do
1	Define AI, and its subfields	10/41	The four-quadrant definition, then six subfields with one line each
1	The Turing test, and passing it	10/41	The three-room setup, the total Turing test, and the standard objections
3	BFS Vs DFS, on the performance criteria	10/41	Complete, optimal, time, space – one table, with b^d against bm
4	Why CNF, and the steps to convert	9/41	Eight steps in order; the two that reorder are Skolemization and dropping $\forall$
5	What is a semantic net, with an example	9/41	Nodes, labelled arcs, isa between classes, instance to an individual
7	Kohonen network / self-organizing map	9/41	Five steps, winner by Euclidean distance, neighbours updated too
1	Applications of AI in daily life	8/41	Two named fields, worked, not a list of buzzwords
7	The neuron model, McCulloch-Pitts, activation	8/41	Weight, sum, threshold; MP weights are <i>chosen</i> , a perceptron's are learnt
7	Back-propagation	8/41	Six steps; needed the moment there is a hidden layer
7	Hopfield network	8/41	Symmetric weights, $w_{ii} = 0$, asynchronous update, content-addressable memory
7	Expert system development stages	8/41	Six stages, and name the knowledge engineer
7	The four levels of NLP analysis and their ambiguity	8/41	Phonetic, syntactic, semantic, pragmatic – one ambiguity example each

Where the marks actually are. Chapter 7 is **899 marks, 26.5 % of everything ever set**, and chapter 4 is **638 marks from only 4 syllabus hours – 2.4 times its hour share**. Those two chapters alone are **45 % of the paper**. But chapters 1, 2 and 5 are cheap and reliable: 1022 marks between them for topics that are mostly fixed lists. **Revise 4 and 7 first, then 1, 2 and 5 for the easy marks, and give 3 whatever is left** – it is the only chapter that is under-asked for its hours.

R.2 Must Memorize (the lists and formulas you cannot derive in the hall)

Ch 1 – Introduction

- **Four definitions of AI:** thinking humanly | thinking rationally | acting humanly | acting rationally. **Turing test sits in “acting humanly”**
- **PEAS** = Performance measure, Environment, Actuators, Sensors
- **Agent types**, simplest first: simple reflex, model-based reflex, goal-based, utility-based, **learning**
- **Environment properties:** fully / partially observable, deterministic / stochastic, episodic / sequential, static / dynamic, discrete / continuous, single / multi agent
- **Turing test** – human interrogator, human, machine, text only, 5 minutes, 30 % deception. **Total** Turing test adds vision and robotics

Ch 2 – Problem Solving

- **CSP** = $\langle X, D, C \rangle$: variables, domains, constraints (unary, binary, higher-order)
- **Crypt-arithmetic constraints:** one digit per letter, all distinct, no leading zero, carries are **0 or 1**, at most 10 distinct letters
- **Well-defined problem** = initial state, actions / operators, transition model, goal test, path cost
- **Production system** = rule base + working memory + control strategy (recognize, resolve conflict, act)

Ch 3 – Search

- **BFS** complete, optimal for unit cost, time and space $O(b^d)$
- **DFS** not complete, not optimal, time $O(b^m)$, space $O(bm)$ – the one advantage
- **A***: $f(n) = g(n) + h(n)$; optimal iff h is **admissible** ($h \leq h^*$), and optimal on graphs iff h is **consistent** ($h(n) \leq c(n, n') + h(n')$)
- **Greedy** uses h alone $\rightarrow$ fast, not optimal
- **Alpha-beta** best case $O(b^{d/2})$, i.e. **twice the depth in the same time**; α is Max's best so far, β is Min's; prune when $\alpha \geq \beta$
- **Hill-climbing traps:** local maximum, plateau, ridge. Cures: random restart, simulated annealing, sideways moves

Ch 4 – KR and reasoning

- **CNF in 8 steps:** remove $\leftrightarrow$; remove $\rightarrow$; push $\neg$ inwards; standardise variables apart; move quantifiers left; **Skolemize** ($\exists$); drop $\forall$; distribute $\vee$ over $\wedge$; split into clauses
- **Resolution refutation:** $KB \cup \{\neg \text{goal}\}$ to CNF, resolve complementary literals with the MGU, derive the **empty clause**
- **Modus ponens** $A, A \rightarrow B \vdash B$
- **Bayes** $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$,
 $P(B) = P(B|A)P(A) + P(B|\neg A)P(\neg A)$
- **Belief network joint**
 $P(x_1 \dots x_n) = \prod_i P(x_i \mid \text{parents}(x_i))$

Ch 5 – Structured KR

- **Class to class is isa; individual to class is instance** – the commonest lost mark in the subject
- **Four adequacies:** representational, inferential, inferential efficiency, acquisitional
- **Five issues in KR:** important attributes; relations among attributes; granularity; sets of objects; finding the right structure
- **Frame** = slots + facets (value, default, range, if-added, if-needed, cardinality)
- **CD primitives:** ATRANS, PTRANS, PROPEL, MOVE, GRASP, INGEST, EXPEL, MTRANS, MBUILD, SPEAK, ATTEND
- **Script** = entry conditions, roles, props, results, scenes

Ch 6 – Machine Learning

- **Entropy** $H(S) = -\sum p_i \log_2 p_i$; **Gain** $= H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$
- $H = 0$ pure, $H = 1$ at 50/50. **Highest gain wins**
- **GA steps:** initialise $\rightarrow$ evaluate fitness $\rightarrow$ select $\rightarrow$ crossover $\rightarrow$ mutate $\rightarrow$ replace $\rightarrow$ terminate
- Crossover rate **0.6–0.9**, mutation **0.001–0.01**. **Mutation is the only source of new genes**
- **Fuzzy ops:** union max, intersection min, complement $1 - \mu$, bold union $\min(1, \mu_A + \mu_B)$, bold intersection $\max(0, \mu_A + \mu_B - 1)$
- **Fuzzy inference:** fuzzify $\rightarrow$ rule strength (AND = min, OR = max) $\rightarrow$ implication (clip) $\rightarrow$ aggregate (max) $\rightarrow$ defuzzify (centroid)
- **Boltzmann** $p_i = e^{-\varepsilon_i/kT} / \sum_j e^{-\varepsilon_j/kT}$

Ch 7 – Applications

- **Neuron** $net = \sum w_i x_i + b$, $y = f(net)$
- **Perceptron rule** $w_i += \alpha t x_i$, **only on an error**
- **Hebb rule** $w_i += x_i t$, **every pattern, one pass**
- **Delta rule** $w_i += \alpha(t - y_{in})x_i$
- **Back-prop:** $\delta_{out} = (y - t)y(1 - y)$;
 $\delta_j = h_j(1 - h_j) \sum_k \delta_k w_{jk}$; $w \leftarrow w - \eta \delta$ (input)
- **Sigmoid** $\sigma(z) = 1/(1 + e^{-z})$, $\sigma' = \sigma(1 - \sigma)$
- **Hopfield** $w_{ij} = \sum_p s_i^{(p)} s_j^{(p)}$ for $i \neq j$, $w_{ii} = 0$; capacity $p \leq 0.15N$; update **one unit at a time**
- **SOM:** winner by $\min \sum (x_i - w_{ij})^2$, then $w_{ij} += \alpha(x_i - w_{ij})$ for the winner **and its neighbours**
- **Expert system** = knowledge base + working memory + inference engine + user interface + explanation facility
- **NLP** = lexical, syntactic, semantic, discourse, pragmatic
- **Machine vision** = acquisition, enhancement, restoration, morphology, segmentation, description, recognition, interpretation

R.3 Must Practice (the worked problems, ranked by how many papers set them)

85 problems across six companions. Every one opens with the question exactly as its paper prints it, so each can be attempted from the PDF alone. Chapter 1 sets no problems in 41 papers and has no companion.

Ch	Problem family	Papers	Problems	Do these first
4	Resolution refutation	33/41	10	The food / likes-peanuts set (7 papers), Charlie-the-horse (6), Colonel West (4)
2	Crypt-arithmetic	31/41	13	BASE+BALL=GAMES (8 papers), SEND+MORE=MONEY (6)
7	Design a net for a logic gate	17/41	5	Perceptron AND, then XOR both halves: the proof and the two-layer net
6	Genetic algorithm generations	21/41	3	The knapsack, where mutation, not crossover, finds the optimum
5	Sentences into a semantic net	13/41	12	Any three; six of the eleven sets are the same three figures renamed
6	Fuzzy arithmetic	16/41	3	The set operations, then one full Mamdani inference
4	Bayes numericals	15/41	12	B5 the rare-disease test, B7 the defective bulb, B10 the wet-grass network
4	CNF conversion	9/41	4	The four formulas the papers actually print
3	A*, best-first, minimax and alpha-beta on a printed graph	8/41	5	Both A* graphs, then alpha-beta on the 2079 Chaitra tree
6	ID3: pick the root attribute	6/41	4	Play golf AND play cricket – same-looking tables, different answers
7	Hebbian AND network	4/41	1	One pass to (2, 2, -2); know why binary fails
2	Water jug, river crossing, tic-tac-toe	5/41	5	The 4-and-3 jug, both directions
7	Forward propagation on the printed net	2/41	1	Both ReLU units are dead: only <i>a</i> and <i>b</i> can be updated
7	Hopfield weight matrix and recall	1/41	1	Three <i>orthogonal</i> patterns, or four units cannot hold them
7	Back-propagation, one step	8/41	1	Nine parameters; the output must move towards the target
7	Parse tree	1/41	1	Five words, eight nodes, three minutes

R.4 Most Repeated PYQs (if you revise only fifteen questions, revise these)

Ordered by how many of the 41 papers set them. Together these fifteen have carried well over half the marks ever awarded in this subject.

1. Assume the following facts. Convert to clause form and prove the goal by resolution refutation. **6 8 2+6** 33 papers
2. Solve the following crypt-arithmetic problem by defining it as a constraint satisfaction problem. **6 8 2+6** 31 papers
3. What is NLP? Explain the different steps involved, with a block diagram and suitable examples. **5 6 8 2+6** 23 papers
4. What is a genetic algorithm? Explain all its steps with a block diagram and its operators. **5 7 8 2+6** 21 papers
5. What is a perceptron? How can we design a neural network that acts as an AND / OR / XOR gate? **6 7 10 1+8** 17 papers
6. What is an expert system? Explain its general architecture with a block diagram. **4 6 8 1+7** 17 papers
7. What is fuzzy logic? When do we use it? Explain fuzzy inference with a suitable example. **4 2+6 3+7** 16 papers
8. Explain Bayes' theorem and its role in reasoning; then the given numerical. **4 2+6 8** 15 papers
9. Explain semantic nets and frames with examples; compare them and list their advantages and limitations. **4+4 7 8** 13 papers
10. What are the problems associated with NLP? **2 3 5** 13 papers
11. Differentiate supervised and unsupervised learning. Which category is clustering? Justify, stating algorithms. **4 5 2+4** 12 papers
12. What is a constraint satisfaction problem? Discuss it and list its constraints. **2 4 2+6** 12 papers
13. What is machine vision? List the steps involved and its applications. **3 4 2+5 2+6** 12 papers
14. Differentiate between forward and backward chaining with a suitable example. **4 2+4 6** 11 papers

15. What are the advantages and disadvantages of an expert system? Compare it with a human expert. 4 7 8 11 papers

The examiner's habit worth knowing. Questions arrive in *pairs*, and the same pairs recur: **crypt-arithmetic** + **well-defined problem**; **resolution** + **CNF**; **forward chaining** + **the Colonel West premises**; **issues in KR** + **draw the semantic net**; **fuzzy logic** + **genetic algorithm** in one question (82 Ka, 82 Bh, 81 Ba); **machine vision** + **back-propagation** (71 Ma, 72 Ash); **declarative Vs procedural** + **expert system architecture** (70 Ma, 72 Ma, 73 Ma). Revising one half of a pair and not the other is how whole 8-mark questions get half answered.

R.5 One-Day Revision Checklist

Morning — the two big chapters (about 4 hrs)

- ☐ **Ch 4 resolution** (75 min): **the highest-value hour in this subject**. Do three refutations end to end — the food set, Charlie-the-horse, Colonel West — writing the CNF steps out every time
- ☐ **Ch 4 rest** (40 min): the eight CNF steps from memory · forward Vs backward chaining · two Bayes numericals · one belief network
- ☐ **Ch 7 diagrams** (60 min): the **five-box NLP pipeline** · the four-box expert system · a two-input perceptron · the machine-vision pipeline. Draw each three times
- ☐ **Ch 7 networks** (45 min): perceptron AND in bipolar · the XOR proof and the two-layer net · the Hebb net · Hopfield and SOM algorithms

Afternoon — the reliable marks (about 3 hrs)

- ☐ **Ch 2 crypt-arithmetic** (50 min): BASE+BALL=GAMES and SEND+MORE=MONEY, both fully written out with the constraint list
- ☐ **Ch 6** (50 min): the GA flowchart and one worked generation · the fuzzy inference block diagram · one ID3 root attribute
- ☐ **Ch 5** (35 min): three semantic nets · the net Vs frame table · isa against instance
- ☐ **Ch 1** (25 min): the four definitions · PEAS for two systems · the Turing test and the objections
- ☐ **Ch 3** (25 min): BFS Vs DFS table · $f = g + h$ and admissibility · one alpha-beta tree

Evening — consolidation (about 1 hr)

- ☐ Re-read **R.2 Must Memorize** twice. Write the eight CNF steps, the GA operator rates and the back-propagation deltas from memory once
- ☐ Re-read every **orange** line in the chapters — they are the traps
- ☐ Read **R.4** and, for each of the fifteen, say out loud what the first three lines of your answer would be
- ☐ Check the **abbreviation chart** and the month codes — **Ash and Asa are different months**

In the hall

- ☐ **Draw first, then annotate**. An unlabelled correct block diagram still scores; three paragraphs with no diagram do not
- ☐ Resolution → **negate the goal before converting**, and say you did
- ☐ Crypt-arithmetic → substitute the digits back and **show the addition working**
- ☐ Semantic net → **isa between classes, instance to an individual**; count one arc per sentence
- ☐ ID3 → recompute; never carry a remembered root across from a similar table
- ☐ Any gate network → use **bipolar ± 1** , and finish with the four-row verification table
- ☐ Every numerical → write the formula before substituting
- ☐ A “scenario” question (Nepali chatbot, disaster expert system, machine-vision line) → **it wants the standard answer with the domain renamed**, plus two concrete rules or features

Abbreviations used in these notes

Only these are ever shortened. Anything not on this list is written out in full.

Short	Means	Short	Means
AI	artificial intelligence	ANI	artificial narrow intelligence
AGI	artificial general intelligence	ASI	artificial super intelligence
KR	knowledge representation	KRR	knowledge representation and reasoning
PEAS	performance measure, environment, actuators, sensors	GPS	General Problem Solver
NLP	natural language processing	NLU	natural language understanding
NLG	natural language generation	CAPTCHA	completely automated public Turing test to tell computers and humans apart
ML	machine learning	ANN, NN	artificial neural network
MLP	multilayer perceptron	BP	back-propagation
SOM	self-organizing map	GA	genetic algorithm
FIS	fuzzy inference system	ID3	Iterative Dichotomiser 3
CSP	constraint satisfaction problem	DFS	depth first search
BFS	breadth first search	DLS	depth limited search
IDS	iterative deepening search	UCS	uniform cost search
CNF	conjunctive normal form	WFF	well-formed formula
FOL	first-order logic	FOPL	first-order predicate logic
RRS	resolution refutation system	BBN	Bayesian belief network

Short	Means	Short	Means
ES	expert system	KB	knowledge base
CD	conceptual dependency	LISP	list processing (the language)
MOP	memory organization packet	isa	the class-to-class link of a semantic net (<i>instance</i> is the individual-to-class one)
attr(s)	attribute(s)	repo	repository
freq	frequency	config	configuration
info	information	no.	number
Eg	example	Vs	compare / differentiate
w/, w/o	with, without	+Fig	draw the figure
→	leads to, therefore	=	is, equals

Reading the year tags and chips

Exam-paper month codes: Ba = Baishakh · Jth = Jestha · Shr = Shrawan · Bh = Bhadra · Ash = Ashwin · Ka = Kartik · Po = Poush · Ma = Magh · Ch = Chaitra. Years are Bikram Sambat, so 80 Ch is 2080 Chaitra.

Bold year tags are Regular exams, plain are Back exams. A header printing “Regular / Back” in one box counts as Regular.

Three course codes, three faces. Plain = CT 653 (BCT, III-II, 26 papers) · monospace = CT 710 (BEI, IV-I, 7 papers) · *italic* = CT 78506 (BEX, IV-II, 8 papers). **The same year code names different papers under different codes:** 77 Ch and 77 Ch are two different exams, and only the font tells them apart.

Tier chips over all 41 papers, 2069–2082 BS: **TOP /41** = topic asked in **8 or more** papers · **HOT /41** = **4 to 7** papers · **PIN /41** = **1 to 3** papers but worth ≥ 6 marks.

Mark chips such as **2+6** are the marks that question actually carried. **syllabus, no PYQ yet** marks a syllabus topic never yet examined but written up anyway. **verified/added** marks anything researched and added that no source note in this folder covers.